

Development of Ontological Architecture for Capability-based Co-simulation Configuration

Master's Thesis

by

Anuja Chakraborty

Matriculation Number: **425838**

Email Id: **vyr58gad@rhrk.uni-kl.de**

Supervisors:

First Advisor: Prof. Dr.-Ing. Peter Liggesmeyer

Second Advisor: Siwara Schmitt, Fraunhofer IESE

31.05.2025

RPTU Kaiserslautern-Landau
Department of Computer Science
Kaiserslautern, Germany

Abstract

This thesis presents a layered ontology architecture for capability-based co-simulation configuration, particularly focusing on simulation models for automotive communication protocols. A four-layer ontology architecture — comprising upper-level, mid-level, domain-level, and use-case-level ontologies — was developed using OWL 2 DL and modular design principles grounded in industry standards such as Basic Formal Ontology (BFO: ISO 21838), VDI 3633, Capability-Skill-Services (CSS), Industrial Digital Twin Association (IDTA) Submodel Template (SMT) for "Provision of Simulation Models", and ISO 25010. Each protocol-specific domain-level ontology (Controller Area Network (CAN), Local Interconnect Network (LIN), Ethernet, FlexRay) was formally represented with semantic annotations and capability mappings aligned to simulation model metadata. A use-case-level ontology is instantiated for the Automated Emergency Braking (AEB) scenario to demonstrate the applicability of the architecture to real-world simulations. Semantic querying is supported through competency questions (CQs) and the query language SPARQL, enabling validation, discovery, and formal configuration of simulation environments. Implemented using the FERAL simulation framework, the architecture supports a scalable, reusable, and interoperable method for aligning simulation intent with domain semantics. This work lays the foundation for future automation in co-simulation configuration and semantic integration across heterogeneous tools.

Keywords: Ontology, Co-Simulation, Capability, Simulation Model, Communication Protocols, Semantic Configuration

Contents

List of Abbreviations	6
1 Introduction	7
2 Related Work	9
2.1 Systematic Mapping Study	9
2.1.1 Research Questions	9
2.1.2 Search strings for online database research	10
2.1.3 Manual target search	10
2.1.4 Inclusion and Exclusion Criteria for search results	10
2.2 Results	11
2.2.1 Results of the Database Search	11
2.2.2 Findings	18
2.2.3 Research Gaps	22
3 Ontological Framework for Capability-based Co-Simulation	26
3.1 Upper-level ontology	28
3.2 Mid-level ontology	28
3.3 Domain-level ontology	30
3.4 Use-case-level ontology	31
4 Implementation of the Ontological Architecture	32
4.1 Overview of the Implementation	32
4.2 Implementation of the Upper-level Ontology	35
4.3 Implementation of the Mid-level Ontology	35
4.4 Implementation of the Domain-level Ontology	39
4.4.1 CAN Simulation Model	46
4.4.2 LIN Simulation Model	51
4.4.3 Ethernet Simulation Model	55
4.4.4 FlexRay Simulation Model	59
4.5 Implementation of the Use-case-level Ontology	63
4.6 Implementation of the Capability based engineering approach	67

5	Evaluation	73
6	Discussion	79
7	Conclusion and Future Work	82
A	Appendix	90
A.1	Mid-level Ontology	90
A.2	Domain-level Ontology	97
A.3	CAN Simulation Model Ontology	97
A.4	CAN use-case-level Ontology	105
A.5	LIN Simulation Model Ontology	112
A.6	Ethernet Simulation Model Ontology	115
A.7	FlexRay Simulation Model Ontology	119
A.8	Additional Ontology files	122

List of Figures

1	The Systematic Mapping Process.[PFMM08]	9
2	Ontology Architecture: the Big Picture.	27
3	Ontology Architecture for Capability-based Co-simulation	28
4	Ontology Architecture for Capability-based Co-simulation shown as the example of Fraunhofer IESE's use case AEB.	32
5	Upper-level ontology.	35
6	Mid-level ontology.	36
7	Object property mapping in Protégé showing domain and range restrictions with semantic comments for automated reasoning and validation	37
8	Automated Emergency Braking Scenario. image source: https://ackodrive.com/car-guide/autonomous-emergency-braking/	63
9	Instance of CAN simulation model for sensoric system in its use-case-level ontology.	65
10	Instance of CAN simulation model for vehicle control system in its use-case-level ontology.	65
11	Process followed for the capability-based simulation configuration. Based on	67
12	Conceptual Flow for Capability and Feasibility Evaluation	68
13	SPARQL query and its result of First Capability Check for Simulation Goal 1	74
14	SPARQL query and its result of Second Capability Check for Simulation Goal 1	74
15	SPARQL query and its result of Feasibility Check for Simulation Goal 1	75
16	SPARQL query and its result of first Capability Check for Simulation Goal 2	75
17	SPARQL query and its result of Second Capability Check for Simulation Goal 2	76
18	SPARQL query of Feasibility Check for Simulation Goal 2	77
19	Result of the SPARQL query of Feasibility Check for Simulation Goal 2	77
20	Domain-Level Ontology of CAN Simulation Model.	80
21	Example of FaultInjection Capability of the CAN Simulation Model Ontology.	81

List of Tables

2	List of primary studies.	11
3	Ontology class alignment with BFO and related standards.	38
4	Ontology property alignment with domains and ranges.	39
5	Class Definitions and Semantic Justifications for CAN simulation model Ontology.	49
6	Object Properties and Their Roles in CAN Simulation model Ontology.	50
7	Datatype Properties for Simulation Configuration in CAN Simulation model Ontology.	51
8	Class Definitions and Semantic Justification in LIN Simulation Model Ontology.	54
9	Object Properties and Their Roles in LIN Simulation Model Ontology.	55
10	Datatype Properties for Simulation Configuration in LIN Simulation Model Ontology.	55
11	Class Definitions and Semantic Justification in Ethernet Simulation model Ontology.	58
12	Object Properties and Their Roles in Ethernet Simulation model Ontology	59
13	Datatype Properties for Simulation Configuration in Ethernet Simulation model Ontology	59
14	Class Definitions and Semantic Justification in FlexRay Simulation Model Ontology	62
15	Object Properties and Their Roles in FlexRay Simulation Model Ontology	62
16	Datatype Properties for Simulation Configuration in FlexRay Simulation Model Ontology	62

List of Abbreviations

Abbreviation	Full Form
--------------	-----------

ABox	Assertional Box
AEB	Automated Emergency Braking
BFO	Basic Formal Ontology
CAN	Controller Area Network
CQ	Competency Question
CSS	Capability-Skill-Services
DeMO	Discrete-Event Modeling Ontology
EC	Exclusion Criteria
ECU	Electronic Control Unit
IC	Inclusion Criteria
IDTA	Industrial Digital Twin Association
LIN	Local Interconnect Network
ODP	Ontology Design Pattern
RQ	Research Question
SMS	Systematic Mapping Study
SMT	Submodel Template
TBox	Terminology Box

1 Introduction

Simulation has become a cornerstone of modern engineering processes, particularly in domains such as manufacturing, automotive systems, and cyber-physical infrastructures [LFS⁺22], [HKK⁺20]. It plays a critical role in virtually mapping production lines, for instance, in the automotive industry, a virtual simulation of a car assembly line can help evaluate how modifications in robotic welding sequences or material flow affect the production efficiency or system safety — even before the physical equipment is installed. This can help in assessing the impact of changes, identifying system vulnerabilities, and reducing costly downtimes [LFS⁺22], [Mou20]. In both the design and operational phases, simulations offer a low-risk, cost-effective alternative to physical prototyping by enabling early detection of functional inconsistencies and performance bottlenecks. However, developing integrated and accurate simulation environments often requires coupling multiple heterogeneous models that differ in abstraction level, format, and simulation paradigm. No single simulation tool is capable of fully capturing the behavioral, spatial, and logical aspects of complex systems. For example, simulating the interaction between a robotic gripper and a product may involve discrete-event, continuous-time, and network communication models. This makes co-simulation, which combines heterogeneous simulation tools and models, increasingly important.

Despite its potential, the configuration and integration of co-simulation environments remains a highly manual, error-prone, and time-consuming process [SK22]. Experts must possess not only domain-specific knowledge, but also detailed familiarity with simulator interfaces, configuration formats, and model compatibility. Manual synchronization and tuning across models can lead to numerical errors, longer simulation times, and inconsistencies in results. Additionally, maintaining simulation setups as real systems evolve poses significant scalability challenges.

To address these limitations, semantic technologies — including ontologies and knowledge graphs — have emerged as promising tools for formally representing the structure, behavior, and capabilities of simulation models [LFW23], [JSHL19]. Ontologies provide a shared vocabulary and machine-interpretable semantics for describing domain concepts, simulation parameters, configuration constraints, and quality attributes. Through automated reasoning and semantic querying, they facilitate key configuration tasks such as capability matching, parameter validation, and scenario composition. The benefits of ontologies in simulation engineering are multifold. They enable the integration of heterogeneous models and tools, support model reuse, and allow the derivation of implicit knowledge through inferencing. For instance, OWL-based ontologies can infer transitive relationships or detect configuration inconsistencies that would otherwise require extensive manual validation. Semantic querying (e.g., via SPARQL) enables domain experts to retrieve simulation models that meet specific functional or quality criteria. Although prior research has applied ontologies in manufacturing and process simulation contexts — e.g., for model generation, reuse, or sequence configuration [LFS⁺22], [RJF23] — simulation of communication networks, especially in safety-critical automotive scenarios, remains underexplored.

To fill this gap, this thesis proposes a layered ontology architecture for capability-based co-simulation configuration, focusing on communication protocols commonly used in automotive systems. The architecture comprises four layers: a Basic Formal Ontology (BFO) (ISO 21838)-aligned upper-level ontology, a mid-level ontology integrating standard patterns (VDI 3633, Capability-Skill-Services (CSS), Industrial Digital Twin Association (IDTA) Submodel Template (SMT) "Provision of Simulation Models", ISO 25010), a domain-level ontology that contains communication protocol-specific ontologies of Controller Area Network (CAN), Local Interconnect Network (LIN), Ethernet, and FlexRay communication protocols, and a use-case-level ontology applied to emergency braking scenarios. It is important to note that the domain-level ontology is designed to be modular and extensible, with its content dependent on the domain goals and use cases of the deploying organization. In this thesis, the protocol-specific modeling at the domain level aligns with Fraunhofer IESE's focus on communication network simulation in the context of Virtual Testbeds. However, the same framework could accommodate alternative domain-specific models for different use-case scenarios depending on the organizational requirements.

Each layer in the architecture builds upon the previous to ensure semantic coherence, modularity, and reusability. This modular architecture enables semantic alignment between simulation goals and simulation model configurations. The overarching objective of this work is to pave the way for automated or semi-automated simulation configuration by enabling machine-readable descriptions of simulation models and their contextual requirements. While the current implementation remains partially manual, the architecture is designed to support future integration with intelligent agents and automation pipelines.

This thesis focuses on the development of an ontology architecture for capability-based co-simulation configuration, with an emphasis on describing and aligning communication protocols at the domain level. The primary scope includes the utilization of relevant communication protocol standards provided by the "Fraunhofer FERAL" simulation framework, with the aim of creating a semantic representation of capabilities and quality attributes of simulation models in relation to co-simulation configuration. As part of this thesis, we implement semantic reasoning mechanisms to perform capability and feasibility checks through SPARQL queries. These checks enable users to identify and evaluate simulation models that meet defined simulation goals, such as testing the effects of fault injection or measuring message transmission under specific conditions.

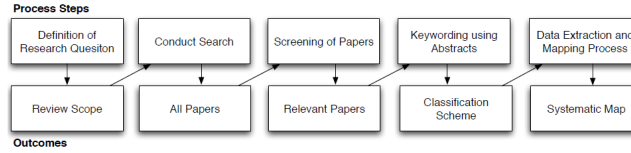


Figure 1: The Systematic Mapping Process.[PFMM08]

2 Related Work

2.1 Systematic Mapping Study

A Systematic Mapping Study (SMS) provides a structured approach to categorizing and analyzing existing research to understand the scope and trends of a specific field. As defined by [PFMM08], "A systematic mapping study provides a structure of the type of research reports and results that have been published by categorizing them." The overall process followed in this study is illustrated in Figure 1.

Initially, the core Research Questions (RQs) guiding the SMS were formulated, serving as a foundation for identifying relevant literature. The search process involved defining appropriate search strings and applying them to well-established scientific databases. Prior to assessing the papers for relevance, inclusion and exclusion criteria were established to filter relevant research contributions. And the final phase involved data extraction and mapping, leading to the construction of a systematic map [PFMM08].

2.1.1 Research Questions

The research questions formulated in this study aim to systematically explore the development of an ontology architecture for capability-based co-simulation configuration. These questions focus on ontology structuring, simulation model representation, and their associated capabilities and quality attributes.

- **RQ1:** How can a structured ontology architecture be developed to support capability-based co-simulation configuration?
- **RQ2:** How can simulation models be represented into the ontology architecture to guide capability-based co-simulation configuration?
- **RQ3:** How can capabilities and quality attributes be formally incorporated to enable automated configuration?
- **RQ4:** How can communication protocols at the domain level be semantically represented to ensure interoperability in co-simulation?

2.1.2 Search strings for online database research

To identify relevant literature, Boolean logic was employed to construct search strings that combined key terms using "AND" and "OR" operators. The following search strings were developed to ensure comprehensive coverage and they were applied to three prominent scientific databases- ACM Digital Library (ACM), IEEE Xplore (IEEE), and ScienceDirect (SD):

- ("ontology architecture" OR "ontology-based" OR "ontology") AND ("semantic interoperability" OR "semantic integration") AND ("simulation" OR "co-simulation" OR "simulation models") AND ("capabilit*") – IEEE: 6, SD: 0, ACM: 0.
- ("ontology architecture" OR "ontology-based" OR "ontology") AND ("semantic interoperability" OR "semantic integration") AND ("simulation" OR "simulation models" OR "co-simulation") AND ("capability") – IEEE: 2, SD: 287, ACM: 130.
- ("ontology architecture" OR "semantic ontology" OR "Ontology" OR "semantic description") AND ("simulation models" OR "simulation" OR "co-simulation" OR "model-based simulation") AND ("capabilit*" OR "functional capabilities" OR "attributes") – IEEE: 184, SD: NA, ACM: 189.
- ("ontology architecture" OR "ontology") AND ("cosimulation" OR "co-simulation" OR "co simulation") – IEEE: 7, SD: 183, ACM: 74.
- "Digital twins" AND "simulation" AND "ontology" – IEEE: 18, SD: 131, ACM: 227.

So, using these search strings the searches were conducted primarily in the "title" and "abstract" fields to ensure that the retrieved papers directly addressed the main research topics. If necessary, deeper investigations were conducted into introductions and other relevant sections.

2.1.3 Manual target search

In addition to database searches, a manual targeted search was performed to supplement the systematic study. Sources such as [Connected Papers](#) and ResearchGate provided relevant academic connections. Additionally, discussions with colleagues and supervisors, as well as internal Fraunhofer documentation, contributed to the selection of key research papers.

2.1.4 Inclusion and Exclusion Criteria for search results

To maintain the relevance and quality of selected papers, inclusion and exclusion criteria were defined.

Inclusion Criteria (IC):

- **IC1:** Papers that match the search string and describe approaches within the scope of this thesis, as defined in Introduction chapter.

Exclusion Criteria (EC):

- **EC1:** Papers not written in English.
- **EC2:** Papers not accessible through the selected databases (IEEE, ACM, SD).
- **EC3:** Papers published before 2004.

2.2 Results

The results of the SMS are presented in this section. First, an overview of the database search results is provided, followed by a detailed presentation of the findings from the identified papers. These findings were used to understand the current state of research and to help identify the research gap addressed in this thesis.

2.2.1 Results of the Database Search

The predefined search strings led to a total of 1438 research papers. These can be divided into 217 from IEEE, 620 from ACM, and 600 from SD. Following application of the IC and EC and removal of duplicates, 1410 papers were discarded. This resulted in 27 papers being identified as relevant to our research topic and included in the list of primary studies. During the course of the research, additional papers from ResearchGate that demonstrated significant relevance to our topic were also incorporated into our analysis.

The papers that have been found relevant as part of the primary studies are listed in Table 2.

Table 2: List of primary studies.

Year	Title	Search String	Source
2006	USING ONTOLOGIES FOR SIMULATION MODELING	("ontology architecture" OR "semantic ontology" OR "Ontology" OR "semantic description") AND ("simulation models" OR "simulation" OR "co-simulation" OR "model-based simulation") AND ("capabilit*" OR "functional capabilities" OR "attributes")	[BPM]

(continued on next page)

Year	Title	Search String	Source
2007	USING ONTOLOGIES FOR SIMULATION INTEGRATION	("ontology architecture" OR "semantic ontology" OR "Ontology" OR "semantic description") AND ("simulation models" OR "simulation" OR "co-simulation" OR "model-based simulation") AND ("capabilit*" OR "functional capabilities" OR "attributes")	[HBH ⁺]
2005	SIMULATION AND THE SEMANTIC WEB	("ontology architecture" OR "semantic ontology" OR "Ontology" OR "semantic description") AND ("simulation models" OR "simulation" OR "co-simulation" OR "model-based simulation") AND ("capabilit*" OR "functional capabilities" OR "attributes")	[MB]
2016	ONTOLOGY-BASED SEMANTIC MODEL OF SUPPLY CHAINS FOR MODELING AND SIMULATION IN DISTRIBUTED ENVIRONMENT	("ontology architecture" OR "ontology-based" OR "ontology") AND ("semantic interoperability" OR "semantic integration") AND ("simulation" OR "co-simulation" OR "simulation models") AND ("capabilit*")	[SLDIMG16]
2009	TOWARDS ONTOLOGY-DRIVEN INTEROPERABILITY FOR SIMULATION-BASED APPLICATIONS	("ontology architecture" OR "semantic ontology" OR "Ontology" OR "semantic description") AND ("simulation models" OR "simulation" OR "co-simulation" OR "model-based simulation") AND ("capabilit*" OR "functional capabilities" OR "attributes")	[BA09]

(continued on next page)

Year	Title	Search String	Source
2023	An ontology-based shop-floor digital twin configuration approach	"Digital twins" AND "simulation" AND "ontology"	[YLZ ⁺ 23]
2014	Building an Ontology for Intelligent Maintenance Systems and Spare Parts Supply Chain Integration	("ontology architecture" OR "ontology-based" OR "ontology") AND ("semantic interoperability" OR "semantic integration") AND ("simulation" OR "co-simulation" OR "simulation models") AND ("capabilit*")	[SP14]
2015	Integrating heterogeneous engineering knowledge and tools for efficient industrial simulation model support	("ontology architecture" OR "semantic ontology" OR "Ontology" OR "semantic description") AND ("simulation models" OR "simulation" OR "co-simulation" OR "model-based simulation") AND ("capabilit*" OR "functional capabilities" OR "attributes")	[NSM5]
2018	Ontology for Conceptual Modelling of Intelligent Maintenance Systems and Spare Parts Supply Chain	("ontology architecture" OR "ontology-based" OR "ontology") AND ("semantic interoperability" OR "semantic integration") AND ("simulation" OR "co-simulation" OR "simulation models") AND ("capabilit*")	[RP18]

(continued on next page)

Year	Title	Search String	Source
2018	Product Model ontology and its use in capability-based matchmaking	("ontology architecture" OR "ontology-based" OR "ontology") AND ("semantic interoperability" OR "semantic integration") AND ("simulation" OR "co-simulation" OR "simulation models") AND ("capabilit*")	[JSHL18]
2023	A role-based Metric to determine the Quality of Simulation Models	("ontology architecture" OR "semantic ontology" OR "Ontology" OR "semantic description") AND ("simulation models" OR "simulation" OR "co-simulation" OR "model-based simulation") AND ("capabilit*" OR "functional capabilities" OR "attributes")	[NSM5]
2020	An Ontology-based Metamodel for Capability Descriptions	("ontology architecture" OR "ontology-based" OR "ontology") AND ("semantic interoperability" OR "semantic integration") AND ("simulation" OR "simulation models" OR "co-simulation") AND ("capability")	[WBS ⁺ 20]
2022	Ontological Architecture for Knowledge Graphs in Manufacturing and Simulation	("ontology architecture" OR "semantic ontology" OR "Ontology" OR "semantic description") AND ("simulation models" OR "simulation" OR "co-simulation" OR "model-based simulation") AND ("capabilit*" OR "functional capabilities" OR "attributes")	[LFS ⁺ 22]

(continued on next page)

Year	Title	Search String	Source
2019	The development of an ontology for describing the capabilities of manufacturing resources	("ontology architecture" OR "ontology-based" OR "ontology") AND ("semantic interoperability" OR "semantic integration") AND ("simulation" OR "co-simulation" OR "simulation models") AND ("capabilit*")	[JSHL19]
2023	An Approach to Automating the Generation of Process Simulation Sequences	("ontology architecture" OR "semantic ontology" OR "Ontology" OR "semantic description") AND ("simulation models" OR "simulation" OR "co-simulation" OR "model-based simulation") AND ("capabilit*" OR "functional capabilities" OR "attributes")	[RJF23]
2011	Applications of Ontologies for Assembling Simulation Models of Industrial Systems	("ontology architecture" OR "semantic ontology" OR "Ontology" OR "semantic description") AND ("simulation models" OR "simulation" OR "co-simulation" OR "model-based simulation") AND ("capabilit*" OR "functional capabilities" OR "attributes")	[N1]
2019	Capability-based semantic interoperability of manufacturing resources: A BaSys 4.0 perspective	("ontology architecture" OR "ontology-based" OR "ontology") AND ("semantic interoperability" OR "semantic integration") AND ("simulation" OR "simulation models" OR "co-simulation") AND ("capability")	[PGL ⁺ 19]

(continued on next page)

Year	Title	Search String	Source
2012	Ontology-based Representation of Simulation Models	("ontology architecture" OR "semantic ontology" OR "Ontology" OR "semantic description") AND ("simulation models" OR "simulation" OR "co-simulation" OR "model-based simulation") AND ("capabilit*" OR "functional capabilities" OR "attributes")	[GCMS12]
2020	FIRST STEPS TOWARDS BRIDGING SIMULATION AND ONTOLOGY TO EASE THE MODEL CREATION ON THE EXAMPLE OF SEMICONDUCTOR INDUSTRY	("ontology architecture" OR "semantic ontology" OR "Ontology" OR "semantic description") AND ("simulation models" OR "simulation" OR "co-simulation" OR "model-based simulation") AND ("capabilit*" OR "functional capabilities" OR "attributes")	[RJMS ⁺ 20]
2020	Application Ontology for Multi-Agent and Web-Services' Co-Simulation in Power and Energy Systems	("ontology architecture" OR "ontology") AND ("cosimulation" OR "co-simulation" OR "co simulation")	[TSP ⁺ 20]
2023	An Architecture for Knowledge Graph based Simulation Support	("ontology architecture" OR "semantic ontology" OR "Ontology" OR "semantic description") AND ("simulation models" OR "simulation" OR "co-simulation" OR "model-based simulation") AND ("capabilit*" OR "functional capabilities" OR "attributes")	[LFW23]
2019	Towards an Assisted Simulation Planning for Co-Simulation of Cyber-Physical Energy Systems	("ontology architecture" OR "ontology") AND ("cosimulation" OR "co-simulation" OR "co simulation")	[SSL19]

(continued on next page)

Year	Title	Search String	Source
2004	Investigating Ontologies for Simulation Modeling	("ontology architecture" OR "semantic ontology" OR "Ontology" OR "semantic description") AND ("simulation models" OR "simulation" OR "co-simulation" OR "model-based simulation") AND ("capabilit*" OR "functional capabilities" OR "attributes")	[MBSF04]
2020	Design Ontology in a Case Study for Cosimulation in a Model-Based Systems Engineering Tool-Chain	("ontology architecture" OR "ontology") AND ("cosimulation" OR "co-simulation" OR "co simulation")	[LWT20]
2008	CODES: An Integrated Approach to Composable Modeling and Simulation	("ontology architecture" OR "semantic ontology" OR "Ontology" OR "semantic description") AND ("simulation models" OR "simulation" OR "co-simulation" OR "model-based simulation") AND ("capabilit*" OR "functional capabilities" OR "attributes")	[TS08]
2022	Flexible Model Exchange in Modelling Smart Mobility by Using Domain Ontologies	("ontology architecture" OR "semantic ontology" OR "Ontology" OR "semantic description") AND ("simulation models" OR "simulation" OR "co-simulation" OR "model-based simulation") AND ("capabilit*" OR "functional capabilities" OR "attributes")	[SK22]

(continued on next page)

Year	Title	Search String	Source
2006	ODSG: An Architecture of Ontology-based Distributed Simulation on Grid	("ontology architecture" OR "semantic ontology" OR "Ontology" OR "semantic description") AND ("simulation models" OR "simulation" OR "co-simulation" OR "model-based simulation") AND ("capabilit*" OR "functional capabilities" OR "attributes")	[DYW06]

2.2.2 Findings

The main objective of this study is to identify research gaps in existing literature. The four research questions formulated in Chapter 2.1.1 serve as a foundation for guiding the systematic review, with a focus on finding relevant insights from prior work.

The development of ontology architecture for capability-based co-simulation configuration, intersects several key research areas. This literature review examines the current state of research across these domains, identifying key developments, observed patterns, and gaps that collectively inform and justify the approach proposed in this thesis.

Foundations and Methodologies in Ontology Architecture and Semantic Modeling :

[LFS⁺22] presents a hierarchical ontological structure which is structured into four levels: upper ontologies, mid-level ontologies, domain ontologies, and use case ontologies. This hierarchical approach allows for modularity and flexibility in representing various simulation scenarios and production processes. It also emphasizes the use of established production and simulation standards, specifically ISA-95 and VDI guidelines to make the ontological architecture robust. [WBS⁺20] presents a structured approach to describe the capabilities of manufacturing resources using Semantic Web technologies. The architecture consists of an upper-level, domain level and application level ontology. The upper-level ontology serves as a foundational framework for capability descriptions where the modules in domain-level can describe concrete capabilities relevant to specific industries or applications, such as assembly, handling, or robotics. It also emphasizes the use of formal logic in ontology modeling, which enables the derivation of implicit facts from explicitly modeled knowledge. This feature enhances the usability and performance of capability descriptions.

[SP14] explores multi-layered ontology structures (Meta Ontology, Domain Ontology, Application Ontology) where The Meta Ontology layer uses Manufacturing Semantic ONtology (MASON) as its basis. The Domain Ontology layer includes concepts from Condition-Based Maintenance/Intelligent Maintenance Systems (CBM/IMS) and Supply Chain ontologies. The

Application/Instantiation layer allows for specific details and instances for each industry area. This approach bridges gaps between various domains, enabling cohesive co-simulation setups in systems like cyber-physical energy systems. [RP18] discusses the development of an ontology as a multi-layered structure, consisting of a meta-ontology layer, a domain ontology layer, and an application ontology/instance layer. It builds upon existing ontologies in manufacturing, maintenance, and supply chain domains, synthesizing concepts from various sources.

[HTB⁺18] discusses an initial method for ontology building which is structured into multiple methodological building blocks (MBBs) designed to address specific requirements identified in the industry and this is an iterative approach that incorporates both lightweight and heavy-weight ontologies, ensuring that the resulting ontologies are tailored to meet the specific needs of collaborative embedded systems. [HKK⁺20] paper presents a structured approach to ontology building that addresses the specific needs of Cyber-physical Systems(CPSs), aiming to facilitate the reuse of existing ontologies and information resources (IRs) within the domain. This method emphasizes the reuse of existing Terminology Box (TBox) to enhance efficiency in ontology development and presents a set of Ontology Design Patterns (ODPs) developed through the proposed method, which can be applied in various industrial contexts. These patterns are crucial for creating modular and reusable ontological descriptions. [THF] focuses on the creation of a methodology called "Ontologies for Reasoning and Communication (OReCo) in Multi-Agent Systems", which is designed to facilitate the development and utilization of ontologies in multi-agent systems. It mentions role of TBox in the structure and functionality of ontologies within multi-agent systems, facilitating effective communication and reasoning by defining the necessary concepts and relationships. The integration and modularity of TBoxes are essential for developing robust ontologies that can adapt to various applications.

Ontology-Driven Frameworks for Modeling and Simulation :

[MB] discusses the development of the Discrete-Event Modeling Ontology (DeMO) which emphasizes balancing expressivity and complexity in ontology design and highlights semantic web technologies (RDF, OWL, SWRL) for enhancing discoverability and usability of simulation models. DeMO facilitates modular and interoperable simulation architectures, supporting dynamic adaptation to specific requirements. Whereas [HBH⁺] introduces the Ontology-based Simulation Integration Framework (OSIF) which addresses semantic interoperability challenges in distributed simulations through semantic validation, mismatch analysis (process, object, abstraction, and data), and composability improvements. [BPM] discusses the benefits of ontology-based approaches in simulation modeling and suggests that further development in this area could lead to more robust and flexible co-simulation architectures. It proposes an Ontology-driven Simulation Modeling Framework (OSMF) solution which includes model libraries and reference libraries that contain reusable components, which can be integrated into simulations. [TS08] presents a novel framework COMposable Discrete-Event scalable Simulation (CODES) aimed at enhancing component-based simulation through the use of

ontologies. CODES introduces the COmponent-oriented Simulation and Modeling Ontology (COSMO), which plays a crucial role in classifying simulation components and facilitating their discovery and reuse. This ontology helps in ensuring that components are semantically valid and can be integrated seamlessly.

[BA09] highlights two fundamental problems in simulation application integration: Semantic Inaccessibility and Logical Disconnectedness and sets the stage for addressing them through ontology-driven approaches. It explores the role of ontologies in achieving dynamic composability of disparate simulation models and includes an automation approach for creating ontologies from simulation system description data, which can streamline the integration process and enhance efficiency. [MBSF04] highlights the potential benefits of using ontologies in the simulation and modeling community, including improved semantic search, integration of heterogeneous information sources, and enhanced analytics and knowledge discovery capabilities. It describes the structure and components of the DeMO ontology, including its main classes: DeModel, ModelConcepts, ModelComponents, and ModelMechanisms and discusses the challenges and considerations in developing such an ontology. [RJMS⁺20] outlines a methodology for developing ontology-based simulation models, which includes problem formulation, model creation, data preparation, and verification. They emphasize that data preparation is often the most time-consuming step, and using ontologies can significantly reduce the time and effort required to create new models. It also discusses the potential for expanding the simulation scope to include various levels of operations, from manufacturing to supply chain management. This expansion is facilitated by the reusability of ontologies, which can be adapted for different simulation models.

[SLDIMG16] introduces an ontology network that conceptualizes different domains relevant to supply chain(SC) and federation. This network facilitates a modular and incremental approach to developing simulation models, enhancing the ability to manage complex systems effectively. [LFW23] presents a knowledge graph-based framework for managing discrete-event simulation models in the context of digital twins and validates the framework against real production data, focusing on automatic model generation. [GCMS12] proposes a layered architecture for ontology representation, which includes an upper ontology for common concepts, a layer for simulator-specific ontologies, and a layer for ontology-based simulation models. And through the employment of ontology querying languages and reasoners, the proposed representation allows for advanced inquiries and inferences, enhancing the usability of simulation models.

[N1] introduces a structured approach using three ontologies: a plant ontology representing the structure of real plants, a simulation ontology detailing available simulation blocks, and a signal ontology for specifying the signals between blocks. This formalization aids in reducing errors and improving the efficiency of simulation model design. Their proposed semantic engine automates the assembly of simulation models by utilizing the formalized knowledge in ontologies to match real plant devices with appropriate simulation blocks, thereby streamlining the design process. [YLZ⁺23] paper proposes an ontology-based approach to configure Shop-floor Digital Twins(SDTs), aiming to facilitate the rapid generation, reconstruction,

and integration of SDTs within the broader context of Cognitive Digital Twins. The approach leverages ontology technology for resource virtualization and semantic integration of multi-domain models, extending the semantic information of Digital Twins(DTs) to support intelligent development. [DYW06] introduces a multi-layered architecture that integrates ontology with grid technologies, providing a semantic-heterogeneous solution for describing simulation resources. This integration allows for a more intuitive understanding of complex simulation environments, as users can interact with simulation resources through a user-friendly ontology-based GUI.

Ontologies in Co-simulation context :

[SSL19] proposes an information model for high-level modeling of co-simulation scenarios. This model which is based on an ontology, is designed to facilitate strategic simulations of future scenarios, allowing for the modeling of data flows, dependencies, and required results. The approach incorporates a component catalog implemented in a Semantic Media Wiki, leveraging Semantic Web technologies. This enables the recommendation of suitable simulation components and the validation of co-simulation scenarios, which is vital for ensuring that the components interact correctly. [LWT20] focuses on the design of a scenario-based ontology aimed to integrate technical resources, system development, and artifacts using a service-oriented approach within a model-based systems engineering (MBSE) toolchain. The ontology serves as a specification for realizing simulation automation in co-simulation platforms which allows stakeholders to manipulate tools for automatic co-simulations, thereby reducing manual operations. [TSP⁺20] introduces Tools Control Center (TOOCC), a multi-agent tool designed to create, simulate, and analyze scenarios to facilitate better integration and communication between heterogeneous systems, addressing interoperability issues in existing co-simulation tools.[SK22] highlights the difficulties faced in integrating diverse modeling approaches, which can hinder the development of comprehensive simulations. So the authors propose the use of domain ontologies to facilitate the parameterization and integration of models. By mapping variables to an ontology, the paper outlines how scenario files can be developed to ensure that all necessary parameters are included for effective simulation that will further support interoperability among different simulation models.

Ontologies in the Capability context :

[JSHL18] presents the Product Model ontology, which is specifically tailored for capability matchmaking, focusing on product-related input information necessary for this process. This ontology is crucial for automating the matchmaking of product requirements with resource capabilities, thereby enhancing efficiency in production systems. Moreover, it emphasizes the importance of formal ontological descriptions for product requirements and resource capabilities, which are essential for effective co-simulation in manufacturing environments. [KHVdSF20] presents a capability modeling approach that is not tied to specific manufacturing scenarios, enhancing reusability across different contexts. This is achieved by utilizing a modular

approach to ontology development, using independent ODPs instead of a large monolithic ontology. This design promotes extensibility and reusability, allowing for easier updates and adaptations to the capability model. [KHC⁺20] proposes a method that significantly reduces the effort required for machine developers to create ontological descriptions of machine capabilities and skills. It leverages existing engineering artifacts to streamline the process and highlights the importance of TBox and ODPs in creating a structured and efficient ontology for machine capabilities and skills, which is vital for automating the development process and enhancing the integration of machines into production environments.[RJF23] presents a concept for automating the selection and generation of simulation sequences based on required process information. The authors emphasize the use of a capability model to describe the functions of simulations which allows for a better understanding of how simulations can generate desired outputs, facilitating a more effective selection process. It also discusses the importance of incorporating quality criteria into the simulation descriptions. This ensures that the simulations not only meet functional requirements but also adhere to quality standards, which is crucial for reliable outcomes. [JSHL19] focuses on creating a formal ontology, known as the Manufacturing Resource Capability Ontology (MaRCO) which addresses limitations found in existing resource description models by enabling the representation and automatic inference of combined capabilities from simpler capabilities of cooperating resources. The paper outlines a systematic development process for MaRCO, which follows an ontology engineering methodology consisting of five phases: feasibility study, kickoff, refinement, evaluation, and usage and evolution. [PGL⁺19] propose an ontology-based approach to integrate capability paradigms into cognitive manufacturing systems. This involves creating semantic descriptions of manufacturing processes and resources, which can facilitate the mapping of process steps to available resources. It highlights the need for standardized sets of skills to enhance capability-based systems engineering which can help bridge the gap between resource providers and manufacturing execution systems. There is another research work that one of the Fraunhofer colleagues is working on, which represents an information model which is designed to support the automated generation of simulation sequences and their capabilities, crucial for efficient planning and execution in manufacturing environments. It utilizes ODPs based on established industrial standards to enhance interoperability and reusability of simulation artifacts.

2.2.3 Research Gaps

Our systematic mapping study identified several key research gaps:

- **Standardization Needs:** The literature consistently highlights a significant gap in the standardization of simulation ontologies. Several studies [LFS⁺22, HBH⁺, MBSF04] reveal that existing approaches lack a unified framework that can seamlessly integrate heterogeneous simulation models, capabilities, and quality metrics. The absence of alignment with widely accepted foundational ontologies and industrial standards limits interoperability and reusability.

- **Integration Challenges:** Existing frameworks struggle with integrating heterogeneous simulation models and ensuring semantic interoperability. As highlighted in [HBH⁺], semantic interoperability is often hindered by ad hoc modeling practices and the lack of common representations for domain-specific elements like communication protocols. Prior work by [JSHL19] and [SK22] also reveals that most current systems are limited in their ability to incorporate different types of simulation assets or metadata consistently. This results in poor support for collaborative and distributed simulation environments.
- **Scalability and Dynamic Adaptation:** Handling large-scale simulation models and adapting them to real-time constraints remain challenging. Several studies [Mou20, TS08] highlight the scalability issues in existing solutions, particularly as simulation models grow in size and complexity.
- **Validation and Verification:** Despite growing interest in ontology-driven simulation, there remains a significant lack of methods for validating and verifying both the simulation models and the underlying ontologies. Existing approaches rarely incorporate quality metrics or formal evaluation techniques. Prior research by [RJF23], [TS08] has emphasized that manual planning of simulation sequences is both time-consuming and error-prone, with quality metrics often not sufficiently integrated into simulation model selection.

Therefore, this thesis proposes the idea of developing an ontology architecture to address the research gaps identified through the literature review. The rationale for the chosen ontology architecture is outlined below:

Addressing Standardization and Interoperability Challenges:

To unify the representation of simulation models, capabilities, and quality attributes under a standardized schema, the proposed architecture is structured into four layers: upper-level, mid-level, domain-level, and use-case-level — to ensure that the representation is modular, extensible, and aligned with industry standards.

- **Upper-Level Ontology:** By adopting the BFO (ISO 21838), the proposed architecture establishes a domain-independent foundation that guarantees consistency and interoperability. This aligns with the need for a shared semantic framework that constrains the interpretation of concepts across different domains.
- **Mid-Level Ontology:** The incorporation of established standards such as VDI 3633 for simulation processes, CSS for capabilities, resources, and processes, ISO25010 for quality metrics and IDTA SMT "Provision of Simulation Models" ensures that the ontology leverages widely accepted industrial benchmarks. This integration bridges the gap between disparate simulation models and enhances interoperability.

- **Domain-Level Ontology:** Focusing on communication protocols (specifically CAN, Ethernet, FlexRay, and LIN), the architecture addresses the need for standardized representations of protocol-specific features. This supports dynamic capability matching and semantic interoperability, as identified in the literature.
- **Use-Case-Level Ontology:** By tailoring the domain-level ontologies to specific simulation scenarios, the framework can provide practical, application-oriented configurations that reduce manual configuration efforts and improve validation processes.

Enabling Automated Configuration and Quality Evaluation:

One of the core research gaps is the lack of mechanisms for automated configuration and quality-based selection of simulation models. So the proposed approach takes into consideration:

- **Capability Modeling:** By formalizing simulation model capabilities using the CSS pattern, the ontological framework can facilitate the automatic matching of simulation models to specific process requirements. This not only improves the selection process but also minimizes the manual intervention required during configuration.
- **Quality Attributes Integration:** The architecture incorporates ISO 25010-based quality metrics into the ontology, ensuring that each simulation model is evaluated against standardized criteria such as Functional Suitability, Reliability, Performance Efficiency, Flexibility and so on. This enables automated reasoning using valid reasoners and SPARQL queries to select simulation models that meet the desired quality thresholds.

Promoting Modularity, Extensibility, and Reusability:

The development of ontologies in complex environments often suffers from scalability issues and high maintenance costs when a monolithic approach is taken. Several studies [HKK⁺20, GP09] advocate for a modular, pattern-based approach to ontology development to enhance reusability and reduce engineering effort. The proposed architecture leverages ODPs to break down the ontology into independent, reusable modules. By aligning each module with industry standards (e.g., CSS [KBH⁺23], VDI 3633 [VDI14], IDTA SMT "Provision of Simulation Models" [Ind22], ISO25010), this design ensures that the TBox is built on a robust, standardized foundation. This modular approach not only simplifies future extensions but also facilitates the integration of additional ODPs as new challenges emerge. Furthermore, the clear separation between the TBox (the schema) and the Assertional Box (ABox) (the instance data) allows the ontology to remain reusable across different use cases. The TBox captures the generic concepts and relationships, while the ABox can be instantiated with scenario-specific data while dealing with different use cases.

In summary, the chosen ontology architecture is designed to systematically address the deficiencies identified in the literature. By leveraging a layered structure that integrates upper-level, mid-level, domain-level, and use-case-level ontologies, the architecture ensures standardization, interoperability, and scalability. The incorporation of quality metrics and capability

modelling enables automated configuration, while the modular, ODP-driven design facilitates reusability and future expansion.

3 Ontological Framework for Capability-based Co-Simulation

The layered ontological architecture adopted for this work is based on the framework described in the chapter “*An Ontology Architecture for Integration of Ontologies*” from the book “*Theory and Applications of Ontology: Computer Applications*”, where ontological architecture is presented into upper, mid-level and domain ontology levels [PHK10]. As illustrated in Figure 2 of that model, the ontology ecosystem is segmented into three key layers: upper, mid-level, and domain levels; at each level, multiple ontologies can be present. Each layer serves different purposes:

1. **Upper-level ontologies** are the most abstract and general, dealing with fundamental concepts like identity, parts and wholes, substance, constitution, space, time, various dynamic properties and other basic philosophical categories. These apply universally across all mid and domain level ontologies.
2. **Mid-level ontologies** bridge the gap between abstract upper-level concepts and specific domain knowledge. They handle more concrete but still general concepts that span multiple domains.
3. **Domain ontologies** model context-specific constructs, dealing with particular fields or areas of knowledge.

The authors emphasize that instead of a monolithic approach, this layered approach allows for better organization, reusability, and interoperability of ontological knowledge across different applications and domains. Therefore, the adoption of this multi-layered architecture offers several benefits for our simulation modeling in the context of capability based co-simulation. First, it enables semantic modularity, allowing the domain-specific ontologies to reuse core concepts from the mid-level ontology without redundancy. Second, this structure enhances interoperability and extensibility — for instance, future simulation domains can be incorporated by simply extending the domain layer without altering the foundational semantics. Third, it fosters logical coherence, aligning well with the engineering-centric need for well-scoped yet interoperable semantic models. This layered design also supports reasoning and query federation: mid-level and upper-level concepts provide anchors for inferencing and abstraction, while domain-level ontologies capture precise implementation semantics relevant for validation and co-simulation tasks.

Therefore, utilizing this foundational structure to build the concept diagram and its subsequent ontologies ensures semantic clarity, reusability, and alignment with standardized knowledge representation principles. This architecture is further extended by incorporating use-case-level ontologies at the lowest level, following the approach outlined by [LFS⁺22], to represent the most specific ontologies for different usecases. These ontologies are derived from the domain level ontologies and contain additional information that are specific to the use case and are typically not present in the upper or mid-level ontologies. This extension leads to

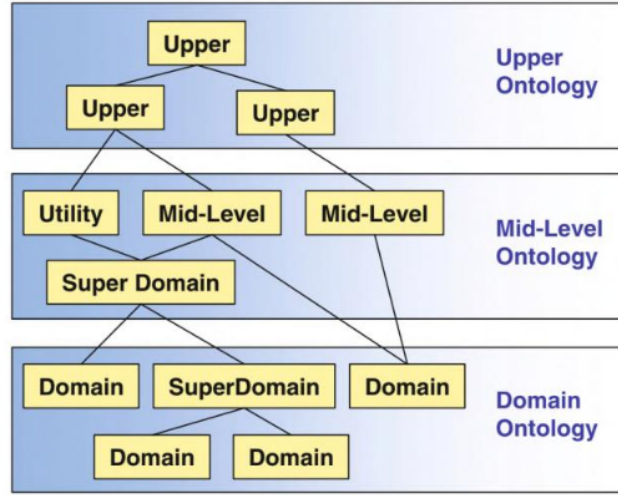
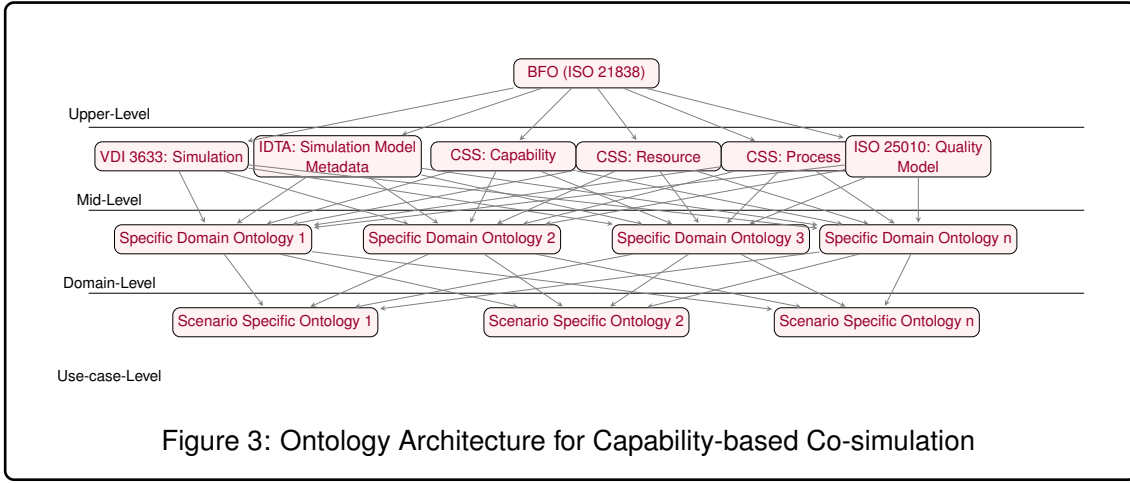


Figure 2: Ontology Architecture: the Big Picture.
[PHK10]

structure the ontological architecture into four different levels: **upper-level ontologies**, **mid-level ontologies**, **domain-level ontologies**, and **use-case-level ontologies**, similar to what is depicted in Figure 2. The overview of our ontological architecture can be seen in Figure 3. While our ontological architecture was inspired by the layered approach presented in [LFS⁺22], there are several important distinctions that justify the need for further elaboration and adaptation in this thesis. [LFS⁺22]’s architecture integrates ISA-95 extensively, primarily to model production systems and processes. In our work, however, we align multiple industry standards like BFO (ISO 21838) for top-level conceptual grounding, CSS for Capability, Resource, and Process modeling, VDI 3633 for Simulation process, IDTA SMT "Provision of Simulation Models" for simulation model specific metadata structures, and ISO 25010 for modeling Quality Attributes. This broader and more modular combination of standards allows us to represent both structural and behavioral simulation semantics, as well as quality-driven configurations, which [LFS⁺22]’s model doesn’t fully address. Moreover, [LFS⁺22]’s ontology architecture is centered around material flow simulation within manufacturing systems, particularly focusing on integrating production and simulation data using ISA-95 and VDI 3633 standards. In contrast, our architecture focuses specifically on capability-based co-simulation for communication networks in automotive systems (e.g., CAN, LIN, Ethernet, FlexRay). Our domain-level ontology is tailored for safety-critical systems such as Automated Emergency Braking (AEB), which is not addressed in [LFS⁺22]’s manufacturing simulation context. Also, [LFS⁺22]’s work introduces "simulation scenarios" as part of the meta-simulation ontology, but stops short of formalizing simulation goals. Where in our architecture, we explicitly define simulation goals which further support semantic queries and reasoning for automated or semi-automated simulation configuration, a step beyond [LFS⁺22]’s setup. These reasons necessitated the development of our proposed ontology architecture, which is formalized and validated in this thesis.



3.1 Upper-level ontology

The **Upper-level ontology** of our proposed framework is based on **BFO**, standardized as *ISO 21838*, which serves as a domain-independent foundation that ensures consistency, formalism, and interoperability across all levels of the ontological architecture. Importantly, this upper-level ontology remains the same regardless of the specific domain or use case being addressed. BFO (ISO 21838) provides fundamental ontological categories such as Function, Object, Process, and Quality [ISO21], which serve as the basis for structuring concepts in the mid-level ontology.

By aligning multiple standards (CSS, VDI 3633, IDTA SMT "Provision of Simulation Models", ISO 25010) under BFO (ISO 21838), we achieve a cohesive integration of simulation capabilities, simulation processes, metadata structures, and quality attributes, ensuring a shared semantic structure and improved interoperability in capability-based co-simulation. The classification of mid-level ontology concepts (e.g., Capability, Resource, Process, Quality attributes) under the foundational categories of BFO (ISO 21838) is discussed in the following subchapter.

3.2 Mid-level ontology

The **mid-level ontology** introduces domain-agnostic but simulation-relevant concepts derived from widely accepted standards. Like the upper-level ontology, this mid-level ontology remains the same regardless of the specific domain or use case being addressed. It incorporates essential concepts such as:

- **Simulation** (VDI 3633) [Bet18]: Governs how simulation models are created, validated, and evaluated. As per the VDI3633 definition, "It is the representation of a system with its dynamic processes in an experimentable model with the aim of reaching findings which are transferable to reality. In a broader sense, simulation is understood to be the preparation, execution and evaluation of targeted experiments using a simulation

model. Simulation allows to examine the running behaviour of complex systems over time [Bet18]."

- **IDTA SMT "Provision of Simulation Models"** [Ind22]: Provides a standard for representing the metadata of simulation models in a structured and machine-readable way which can enhance the architecture by offering clear standardization, interoperability, and alignment in Industry 4.0 ecosystems.
- **Capability** (CSS) [DBB⁺22]: Defines the concept of capabilities required for modeling simulation model capabilities that are needed in the co-simulation processes.
- **Process** (CSS) [DBB⁺22]: Defines the concept of processes required for modeling the co-simulation processes that are involved in simulation execution.
- **Resource** (CSS) [DBB⁺22]: Defines the concept of resources for modeling the simulation models and its simulators that are necessary for simulation execution.
- **Quality Model** (ISO 25010) [Iso]: Addresses the quality aspects of simulation models.

Each of these mid-level ontology concepts is structured within the ontology by mapping them to BFO (ISO21838) foundational categories:

1. Capability (CSS) → BFO: Function

In CSS, the concept of Capability refers to The implementation-independent description of the function of a resource to achieve a certain effect in the physical or virtual world [DBB⁺22]. Whereas the function class in BFO (ISO21838) "is a disposition that exists in virtue of its bearer; physical make-up; this physical make-up is something the bearer possesses because it came into being either through evolution (in the case of natural biological entities) or through intentional design (in the case of artefacts) in order to realize processes of a certain sort" [ISO21]. Thus, by classifying Capability under the Function class of BFO, we ensure that capabilities in co-simulation are not merely abstract properties but well-defined functional components that depend on the intrinsic properties of a resource. This classification enables the formal representation of a system's ability to perform certain tasks within a simulation environment, aligning with BFO's notion of function as a disposition resulting from intentional design.

2. Resource (CSS) → BFO: Object

In CSS, Resources are entities capable of performing functions specified as the capabilities [DBB⁺22]. In BFO, an Object is a material entity that exhibits causal unity, meaning it maintains structural and functional coherence as an independent entity [ISO21]. By classifying Resources as Objects, we establish that any simulation resource, whether physical (e.g., a robot arm) or virtual (e.g., a software service), exists as an independent entity that can be assigned or allocated to a simulation task. This ensures that all simulation resources are distinctly represented, managed, and referenced within the ontology framework.

3. Process (CSS) And Simulation (VDI 3633) → BFO: Process

BFO (ISO21838) defines a Process as an occurrent that has temporal parts and includes material entities as participants over time [ISO21]. In CSS, a Process refers to the set of procedures that transform, transport, or store material, energy, or information [DBB⁺22]. Similarly, VDI 3633 defines "Simulation as the representation of a system with its dynamic processes in an experimentable model with the aim of reaching findings which are transferable to reality"[Bet18]. Since both definitions describe time-dependent transformations involving participants (e.g., physical systems, control logic, or data flows), they align well with BFO's Process class. This classification ensures that co-simulation workflows are accurately modeled as time-evolving processes that impact entities in the simulation environment.

4. Quality Model (ISO 25010) → BFO: Quality

ISO 25010 defines a Software Product Quality Model that provides a framework for evaluating system and software quality attributes, such as reliability, performance, and maintainability [Int23]. BFO (ISO21838) categorizes Quality as a specifically dependent continuant that does not require a process to be realized but is inherent in an entity [ISO21]. Thus, by classifying Quality Attribute under BFO's Quality class, we establish that simulation model quality metrics are not separate entities but inherent properties that can be assessed based on predefined evaluation criteria. This classification ensures that simulation outputs align with industry-defined quality standards and facilitates formalized quality assessment in co-simulation environments.

5. SMT Provision of Simulation Models (IDTA) → BFO: Object

IDTA SMT "Provision of Simulation Models" serve as the structured representations of simulation models for Asset Administration Shells that facilitate the search, discovery, and integration of suitable models into a simulation environment [Ind22]. Since simulation models are independent informational artifacts rather than dynamic processes, they are best classified under BFO's Object class. This classification ensures that simulation models can be instantiated as reusable, standalone objects, allowing for consistent metadata representation, standardization across different simulation environments, and alignment with Industry 4.0 digital twin ecosystems.

Therefore, this structured classification ensures that capability-based co-simulation is formally grounded, making the ontology framework interoperable, scalable, and industry-compliant. So, importing these mid-level ontologies into domain-level ontologies can facilitate the description of resources, processes, capabilities, and quality metrics consistently across different domains.

3.3 Domain-level ontology

The domain-level ontology serves as the third layer in the proposed ontology architecture. It specializes mid-level ontology constructs into domain-relevant semantics depending on the simulation focus of the implementing organization. As shown in Figure 3, this level can contain any number of domain-specific ontologies, which can be tailored to different domain specific

areas such as network protocols, robot control, or manufacturing logistics, depending on the organization's simulation models and domain specific regulations.

The role of the domain-level ontology is to bridge abstract concepts of capability, resource, process, and others with operational domain semantics. While this thesis models communication protocols, the concept is not restricted to this domain. Any company or research setting can create domain ontologies for simulation models relevant to their context (e.g., robotic arms, conveyor belts, energy grids). These ontologies encapsulate reusable structures, behaviors, and metadata critical for simulation modeling in a given domain, while still maintaining interoperability with the mid-level structure.

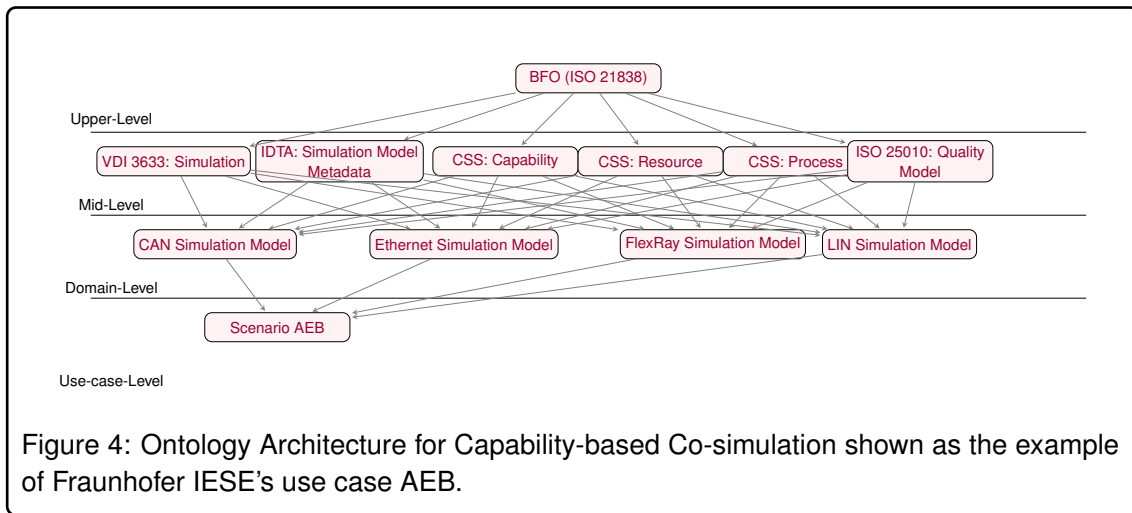
3.4 Use-case-level ontology

The final layer of the ontological framework consists of the use-case-level ontology, as shown in Figure 3, which contextualizes and extends the domain-level ontology to represent real-world simulation scenarios. These ontologies are instantiated from domain-level ontologies and incorporate scenario-specific details such involved simulation resources, capabilities and other scenario-specific configuration. As depicted in the diagram, multiple scenario-specific ontologies can be created — each tailored to a particular simulation use case. These could represent different use cases tailored to the organization's domain and use case specific requirements. There is no limitation on how many or what type of use-case ontologies can be created. This flexibility supports scalability and enables users to define simulations aligned with distinct objectives. Overall, this layer brings simulation metadata into the instance level, while supporting reusability, traceability, and domain-specific adaptation.

4 Implementation of the Ontological Architecture

4.1 Overview of the Implementation

This chapter describes the practical implementation of the ontology architecture for capability-based co-simulation configuration as conceptualized in the previous chapter. The focus lies on concretizing each level of the architecture — upper, mid, domain, and use-case — through reusable OWL-based ontologies, guided by industrial standards and ODPs. Ontologies were developed using Protégé with OWL 2 DL support and validated using SPARQL queries and reasoning. To illustrate this architecture concretely, Fraunhofer IESE's protocol simulation models were applied within the context of virtual testbeds, and a simulation use case of AEB was modeled to showcase the scenario-specific application. This implementation focuses specifically on communication protocols (CAN, LIN, Ethernet, FlexRay) at the domain level and a network-level emergency braking simulation at the use-case level.



As depicted in Figure 4, the ontology architecture was instantiated to reflect the implementation of four levels. At the upper-level, the BFO (ISO 21838) serves as the universal foundational ontology. At the mid-level, standards such as VDI 3633, IDTA SMT "Provision of Simulation Models", CSS, and ISO 25010 are incorporated to define abstract semantic constructs like SimulationModel, Capability, Resource, QualityAttribute and so on. These mid-level constructs are then specialized in the domain-level, where simulation models for communication protocols—CAN, LIN, Ethernet, and FlexRay — are formalized. Each protocol-specific ontology maps the mid-level semantics to domain-relevant simulation behaviors. Finally, in the use-case level, a scenario-specific ontology for AEB is created, instantiating the domain-level constructs and aligning them with simulation requirements..

Protégé was chosen as the primary ontology development tool due to its rich support for OWL 2, user-friendly interface, and extensibility via plugins. It allows visual ontology modeling, integration of reasoners (like Pellet, Hermit), and export of OWL files for deployment. Protégé's tight integration with reasoning engines, and support for ontology alignment make it highly suited for complex simulation ontology projects. Additionally, its broad community

support and extensive documentation significantly enhance development productivity and reproducibility, making it preferable over alternatives like TopBraid Composer or WebProtégé for detailed modeling tasks. And as a widely-adopted, mature reasoner Pellet was selected as the reasoning engine for this ontology stack due to its full support for OWL 2 DL and built-in capabilities for consistency checking, ontology classification, and instance realization. As a common choice in the ontology engineering community, Pellet has proven reliability and extensive documentation. As demonstrated in the inferencing case studies by [DuC13], Pellet's integration with Protégé allows it to infer implicit class memberships (e.g., domain and range implications), resolve subclass relationships, and support SPARQL-DL for hybrid querying and reasoning. Unlike lighter reasoners, Pellet supports reasoning with cardinality constraints, disjointness, datatype properties, and crucially, complex inferencing features such as `owl:inverseOf`, `owl:symmetricProperty`, and `owl:subClassOf`, which are crucial for ensuring the logical soundness of simulation configurations. Moreover, its open-source availability makes it accessible for both design-time and runtime ontology validation.

Ontology plays a crucial role in structuring in-depth knowledge by defining concepts and their relationships within a specific domain [HKK⁺20]. According to [Usc99], an ontology "...may take a variety of forms, but it will necessarily include a vocabulary of concepts and some specification of their meaning. This includes definitions and an indication of how concepts are interrelated, which collectively impose a structure on the domain and constrain the possible interpretations of concepts." These concepts and relationships — along with their attributes — constitute the TBox. The TBox specifies the schema or structure of the domain, while the ABox contains the specific instances that adhere to the TBox [Gru93]. For example, when defining the ontology of machine states, the TBox formalizes the semantics of possible machine states and transitions, whereas the ABox utilizes these definitions to represent concrete state machines. [HKK⁺20] further mentioned that the majority of the effort in ontology development lies in constructing the TBox — identifying and formalizing the appropriate concepts and relationships that model the domain effectively and this problem has been faced by many researchers and practitioners of the domain.

Therefore, in our work, we intended to leverage this principle by focusing on developing TBox that encapsulates all relevant concepts and relationships for capability-based co-simulation configuration. The TBox becomes reusable across various use cases, while the ABox represents instance-specific information of the concepts of TBox for specific use cases. We incorporate this through the inclusion of concepts such as simulation, simulation models, capabilities, resources, processes, and quality attributes, which form the foundational structure (TBox) reused across domain and use case levels (represented in the ABox). We also utilized some of the mechanisms as mentioned by [HKK⁺20] to facilitate the extension of the existing TBoxes. And they are:

1. "Subclassing" of existing concepts which involves creating new subclasses of existing classes.
2. "Equivalent-To" relationships for concept alignment which establishes equivalence be-

tween classes from different ontologies.

3. "Relation-To" for connecting different ontological components through the creation of properties (relations) between classes from different ontologies.

[HKK⁺20] also emphasize that if ontologies are to be combined — such that one ontology uses the TBox or ABox of another — then the ontologies must be matched to ensure logical consistency. Matching can occur through merging (creating a new ontology from two source ontologies while leaving them unaltered) or through reconciliation (harmonizing the content of two or more ontologies without a full merge, leading to coevolution). In both cases, the resulting ontology has a consistent TBox that integrates the TBoxes of the input ontologies. Therefore, we also went with the ontology reconciliation approach where multiple ontologies are harmonized to work together without being fully merged into a single new ontology. The approach enables different ontologies to coevolve while maintaining logical consistency between their concepts. So, we imported some existing ontologies without creating an entirely new merged ontology and tried to harmonize these ontologies by defining relationships between their concepts.

Moreover, to promote reuse and standardization, it's better to rely on established and proven concepts and relations that are widely accepted in the manufacturing and simulation communities [HKK⁺20]. Reusing ontologies that are not designed modularly leads to increased complexity and effort [Ham17]. As defined by [GP09], ODPs provide reusable solutions for recurring ontology design challenges and are maintained and used separately [KHC⁺20], enabling seamless integration with future standards and patterns. So, a modular approach to ontology development can enhance extensibility, reusability, and reduce overall engineering effort [HKK⁺20]. Therefore, rather than creating a large, monolithic ontology and for ensuring modularity and standardized approach, our mid-level ontology is composed of some existing independent, industry standard based ODPs like BFO (ISO21838), CSS, VDI3633, IDTA SMT "Provision of Simulation Models" and each of these patterns is aligned with a specific component of our ontology.

To validate the developed ontology, we employed Competency Questions (CQs) as a means of defining its intended requirements. CQs extract requirements in a structured question-answer format, ensuring that the ontology can effectively answer domain-specific queries [SF11],[HKK⁺20]. Each CQ was designed to test the ontology's ability to provide meaningful insights, with corresponding SPARQL queries used for validation which are discussed in detail in the Evaluation Chapter 5.

So, all in all the implementation of the ontologies was guided by a structured methodology that integrates modular ODPs and industry standards to ensure standardization and generalizability with a strong focus on TBox utilization, allowing it to serve as a reusable framework for different scenarios. Additionally, we validated the ontological architecture through CQs and corresponding SPARQL queries, ensuring its correctness and applicability.

4.2 Implementation of the Upper-level Ontology

This research implements a comprehensive upper-level ontology by integrating the BFO - (ISO21838) as the foundational semantic framework of the ontology architecture. Rather than developing a new upper-level ontology from scratch — which would be methodologically unsound and inferior to established standards — this work strategically leverages BFO's proven formal foundations to ensure semantic consistency and logical rigor across mid-level and domain-level ontologies based on the discussion in Chapter 3.1. The key BFO classes utilized in this work include: Function, Object, Process, and Quality. Figure 5 shows an example of one of these classes in the ontology.

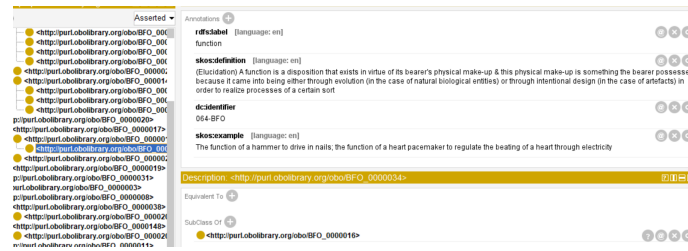


Figure 5: Upper-level ontology.

BFO serves here as a referential backbone, with mid-level ontology elements mapped to BFO classes to preserve ontological rigor and align with broader semantic web practices. The following semantic alignments establish formal relationships between mid-level concepts and BFO categories based on the mapping discussion in Chapter 3.2, where the subsumption symbol ($\sqsubseteq$) denotes subclass-of relationships:

$\text{idta:SimulationModel} \sqsubseteq \text{bfo:Object}$
 $\text{vdi3633:Simulation} \sqsubseteq \text{bfo:Process}$
 $\text{css:Capability} \sqsubseteq \text{bfo:Function}$
 $\text{css:Resource} \sqsubseteq \text{bfo:Object}$
 $\text{css:Process} \sqsubseteq \text{bfo:Process}$
 $\text{iso25010:QualityAttribute} \sqsubseteq \text{bfo:Quality}$

This alignment enables coherent layering of simulation-specific semantics on top of a universally applicable ontological structure, supporting robust reasoning and interoperability throughout the capability-based co-simulation configuration framework.

4.3 Implementation of the Mid-level Ontology

The mid-level ontology implemented in this thesis represents the core conceptual structure that bridges the domain-specific and upper-level abstractions in the proposed four-layer architecture for capability-based co-simulation configuration. This ontology was developed using OWL 2 DL in RDF/XML with Protégé ontology editor as you can see from Figure 6 and there

we used the reasoner Pellet to check the consistency of the ontology. Positioned between the foundational BFO upper-level ontology and domain-specific ontologies, it semantically models essential simulation constructs including Simulation Models, Simulation, Capabilities, Resources, Processes, and Quality Attributes, leveraging several standardized ODPs. Therefore, as part of that it imports BFO (ISO 21838) as foundational upper-level ontology for categorizing simulation entities as Object, Process, Function, and Quality; VDI 3633 (Simulation) for defining Simulation as a formal Process, CSS (Capability, Resource, Process) to provide reusable patterns for Capability, Resource, and Process, IDTA SMT "Provision of Simulation Models" to capture the meta information relevant to our simulation models, and ISO 25010 (Quality Model) to represent essential quality metrics associated with the simulation models. The ODPs used for the ontology include the CSS model, the VDI 3633, IDTA SMT "Provision of Simulation Models". So, importing these ODPs and standards allow the ontology to leverage standardized patterns and modularity across industrial simulation and system modeling contexts. It also promotes extensibility, maintainability, and semantic coherence across domains.

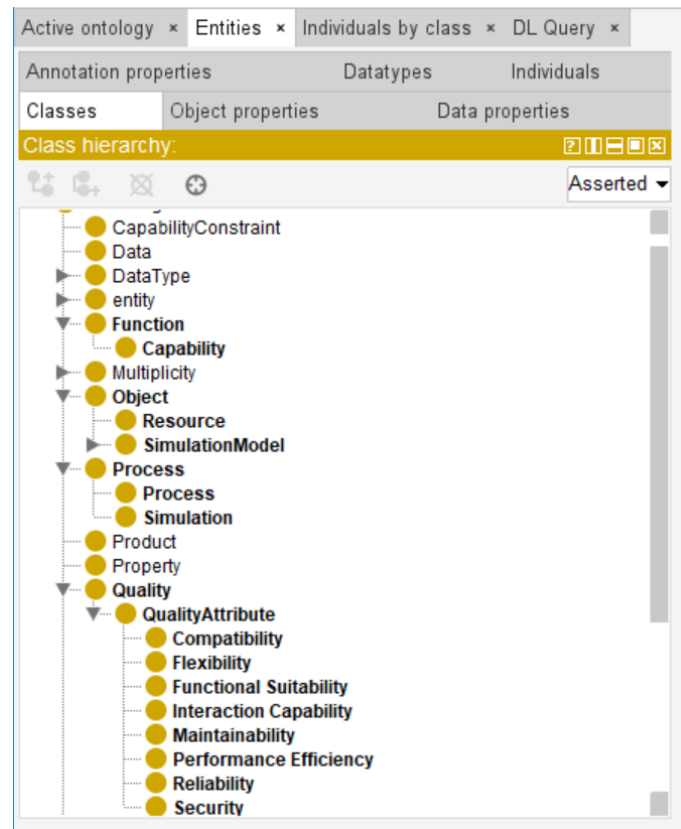


Figure 6: Mid-level ontology.

The namespace of the mid-level ontology is defined at "http://example.org/ontology", and all core classes are aligned with standardized concepts using owl:subClassOf or owl:equivalentClass when relevant, to ensure semantic consistency and cross-domain reasoning and integration. Namespace and Internationalized Resource Identifier (IRI) conventions follow a consistent structure, ensuring easy extension, integration and better maintainability. Each property is clearly mapped to a standardized concept and includes domain and range restrictions.

tions as well as comments explaining their semantic use. One such property example is shown in Figure 7. This enables automated validation, reasoning, and inferencing in OWL reasoners. This ontology acts as the foundational schema (TBox) providing classes and properties which serves as a flexible base to populate and extend concepts for respective domain-level ontologies (e.g., communication protocols) and scenario instances for use-case-level ontologies, fulfilling the architectural goal of capability-based co-simulation configuration. So, as part of that the classes we defined and used are the ones you see in Table 3. For example, class 'SimulationModel' of IDTA SMT "Provision of Simulation Models" standard is defined as a subclass of 'bfo:Object', indicating it's a persistent, structured entity in the ontology. The explanation notes that simulation models must be reusable and instantiated independently, thus justifying their object-oriented representation. The mapping of object properties with their associated semantics can be seen in Table 4, facilitating intuitive understanding and extendability. For example, consider the property 'usesResource'. This property connects a SimulationModel (Domain) to a Resource (Range). This semantic relationship captures the notion that a simulation model depends on resources for simulation execution.

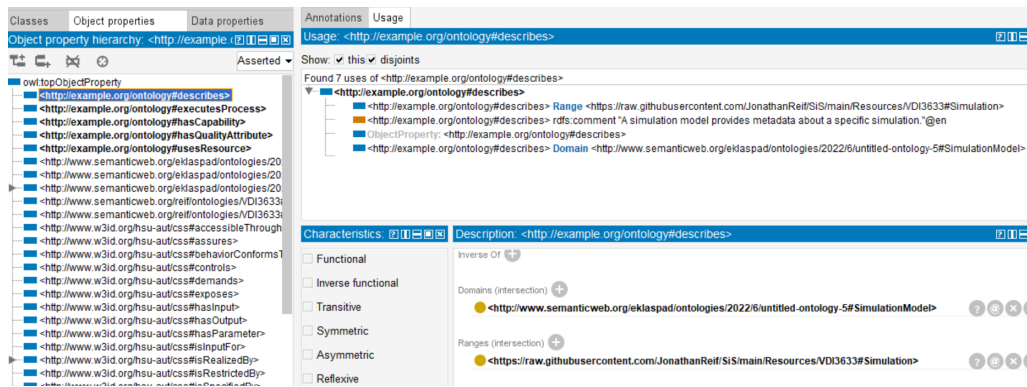


Figure 7: Object property mapping in Protégé showing domain and range restrictions with semantic comments for automated reasoning and validation

Class Name	Subclass Of	Source ODP / Standard	Explanation and Reasoning
SimulationModel	bfo:Object	IDTA Simulation Model Metadata	A simulation model represents a self-contained digital artifact that can be instantiated, reused, and managed. Classified under <i>Object</i> since it is a persistent entity capable of having properties, participating in processes, and serving as a referential unit.
Simulation	bfo:Process	VDI 3633	Describes the temporal progression of system behavior modeled through simulation. It aligns with BFO's definition of <i>Process</i> as an occurrent with temporal parts and participants. Essential to represent simulation as an executable dynamic activity.
Capability	bfo:Function	CSS Ontology	Describes the functional potential a resource or system offers (e.g., fault injection). Mapped to <i>Function</i> in BFO as it denotes a designed or evolved disposition to perform specific activities, crucial for matching simulation goals with available model behavior.
Resource	bfo:Object	CSS Ontology	Refers to entities (hardware/software) that offer or consume capabilities. In the CSS-concept, only a <i>Resource</i> may have a capability. The <i>Resource</i> is the simulator and its simulation models. So, simulation models are considered artifacts or parts of <i>Resources</i> . This alignment supports the CSS design, where <i>Resources</i> represent functionally meaningful actors in the system.
Process	bfo:Process	CSS Ontology	Represents domain-relevant procedures (e.g., braking, sensing, message transmission) executed by simulation models. It inherits from <i>Process</i> as it unfolds in time and interacts with material entities (resources).
QualityAttribute	bfo:Quality	ISO 25010	Represents measurable criteria used to evaluate simulation models (e.g., reliability, maintainability). Fits <i>Quality</i> in BFO as it describes inherent, specifically dependent characteristics of entities that don't require additional processes to manifest.
Functional Suitability	iso25010:QualityAttribute	ISO 25010	Captures how well the simulation fulfills its functional requirements. Critical for capability-based evaluation in simulation.
Reliability	iso25010:QualityAttribute	ISO 25010	Expresses the simulation's ability to operate without failure over time. Relevant for robust modeling of fault-prone scenarios.
Performance Efficiency	iso25010:QualityAttribute	ISO 25010	Reflects how efficiently simulation models use resources (e.g., CPU, memory) during execution. Important for optimizing simulation platforms.
Maintainability	iso25010:QualityAttribute	ISO 25010	Defines how easily a simulation model can be modified, extended, or updated. Critical for lifecycle management.
Usability (Interaction Capability)	iso25010:QualityAttribute	ISO 25010	Describes user interaction quality with the model/system interface—important for simulation platform accessibility.
Security	iso25010:QualityAttribute	ISO 25010	Ensures data and model protection during simulation execution, vital in distributed or cloud-based simulation environments.
Portability (Flexibility)	iso25010:QualityAttribute	ISO 25010	Denotes the ability of models to be reused across environments or configurations. Important for co-simulation reuse.
Compatibility	iso25010:QualityAttribute	ISO 25010	Indicates how well a model interacts with others in a shared simulation setting. Crucial for interoperability in co-simulation.

Table 3: Ontology class alignment with BFO and related standards.

So, this mid-level ontology plays a pivotal role in bridging abstract simulation concepts with domain-specific modeling. By integrating core elements from standards such as VDI 3633,

Property	Domain	Range	Description	Justification
hasCapability	IDTA:SimulationModel	css:Capability	Denotes what capabilities a simulation model offers	Enables semantic matching between simulation goals and functional offerings. Reflects BFO's notion of dispositions (Function).
usesResource	IDTA:SimulationModel	css:Resource	Resources utilized by a simulation model	Captures dependencies for execution. Important for validating resource allocation during co-simulation.
executesProcess	IDTA:SimulationModel	css:Process	Links models to the processes they simulate	Formalizes the temporal behavior simulated by the model. Binds static model entities to dynamic simulations.
hasQualityAttribute	IDTA:SimulationModel	iso25010:QualityAttribute	Declares the quality attributes associated with a simulation model.	Supports model selection and benchmarking based on quality measures (reliability, efficiency, etc.).
describes	IDTA:SimulationModel	VDI3633:Simulation	Links the simulation model to the actual simulation instance it represents.	Anchors metadata to execution logic, enabling traceability and reasoning over model instances.
providesCapability	css:Resource	css:Capability	What capabilities a resource can offer	Enables capability-resource traceability. Supports capability-based model orchestration.
requiresCapability	css:Process	css:Capability	Capabilities necessary for process execution	Crucial for matching models with simulation goals, supports process configuration logic.

Table 4: Ontology property alignment with domains and ranges.

ISO 25010, CSS, and IDTA SMT "Provision of Simulation Models", this ontology provides a modular and reusable scaffold upon which domain-level ontologies (described in the following Chapter 4.4) are constructed. Its alignment with upper-level ontology patterns ensures philosophical consistency, while its semantic grounding in simulation metadata allows for practical application in co-simulation configuration. The use of Protégé and OWL DL enables reasoning and validation across this layer, ensuring semantic integrity before domain instantiation.

4.4 Implementation of the Domain-level Ontology

In alignment with the four-layer architecture, the domain-level ontology specializes the mid-level structure for modeling the protocol simulation models, as this is the domain of Fraunhofer IESE. Therefore, the proposed domain-level ontology serves as a structured representation of communication networks as shown in Figure 4, focusing on the simulation models of four network protocols: CAN, Ethernet, FlexRay and LIN, and their key attributes. We first discuss some key concepts of each communication protocol, their role on simulation fidelity and performance evaluation. Moreover, we capture details of the simulation models of these four protocols from the documentation of Fraunhofer FERAL framework, which provides an event-driven simulation environment to evaluate protocol behavior, timing constraints, and data exchange mechanisms, ensuring accurate representation for network validation and performance analysis. Based on these details we build the main foundation of our domain-level ontology:

- **CAN** : CAN is a robust, message-based communication protocol designed for real-time

applications in automobile and industrial domains [WFX⁺18]. It operates as a multi-master bus system, where multiple devices (nodes) communicate over a shared bus [BMP07]. CAN protocol simulations have primarily focused on improving reliability and robustness in cyber-physical systems. [WFX⁺18] developed an Optimized Network Engineering Tools (OPNET) model of the CAN bus tailored for power system control applications which includes detailed structures for CAN bus nodes, processes, packets, and links, essential for simulating network behavior in power systems. Moreover, their proposed design of a cyber-physical power system co-simulation platform through the integration of OPNET and Power Systems Computer Aided Design (PSCAD) allows for the simultaneous simulation of both the power system and the communication network, addressing the need for a more holistic approach to system analysis. Whereas [BMP07] created simulation models for CAN bus communication using TrueTime and Colored Petri Nets (CPN) Tools. And the simulation results were validated against a laboratory CAN network modeled after a Vehicle Control Unit (VCU) to demonstrate the accuracy of the simulation models, confirming their reliability for future applications.

FERAL provides a discrete-event CAN simulation model that captures key functionalities such as message frame transmission, error detection, and network dynamics, as per the CAN standard [ISO24]. This model is built on the COMNET networking module, which provides an event-driven infrastructure for modeling network topology, device interactions, and message exchange. The simulation model comprises the following core components:

- CAN Network Architecture:

1. High-Level Structure simulates a CAN network, comprising components such as Network Topology, Application Interfaces, Network Devices, and the physical medium. External applications (FERAL Workers) interact with the network via Application Interfaces.
2. Low-Level Structure consists of Network Coder to convert Logic Frames (application-level data) into CAN-specific frames, a Queue to buffer frames before transmission and Controller to manage message transmission and arbitration.

- Supported CAN Protocol Variants: FERAL supports multiple CAN protocol types.

1. CAN Standard (Classical CAN) uses 11-bit or 29-bit message identifiers.
2. CAN FD (Flexible Data Rate) allows larger payloads (up to 64 bytes) and faster transmission speeds.
3. CAN XL (Extended Length) is designed for future automotive applications with higher data rates and extended frame structure.

- Message Transmission & Arbitration:

1. Frame Handling enables network Coder to process Logic Frames and convert them into CAN Frames. Frames are categorized as data frames, remote frames, error frames, and overload frames.

2. Arbitration Mechanism of the CAN Bus implements Carrier Sense Multiple Access with Collision Resolution (CSMA/CR) where messages with lower CAN IDs have higher priority and win arbitration.
 3. Queue Management process buffers the CAN frames into the Queue before sending them through the Medium. It ensures sequential processing and controlled network congestion.
- Error Handling & Data Integrity: FERAL's CAN simulation captures error detection and fault tolerance mechanisms that are aligned with standard CAN protocol behavior.
 1. Bit Error Detection identifies bit mismatches during transmission.
 2. Acknowledgment Errors ensure frames are received successfully.
 3. Cyclic Redundancy Check (CRC) verifies data integrity of message payloads.
 4. Lost Arbitration Logging tracks arbitration failures when multiple nodes attempt to transmit simultaneously.
 - Bus Simulation Characteristics:
 1. Medium Simulation models the physical layer behavior of the CAN bus, capturing aspects such as bit stuffing, synchronization, and timing constraints.
 2. Port and Link Management enables transmission (Tx) and reception (Rx) Ports to handle frame encapsulation and logical message transmission.
 - Quality Metrics in FERAL CAN Simulation: Although FERAL does not define explicit quality criteria, it supports performance evaluation through the following metrics:
 1. Arbitration Loss & Collision Handling monitors frame loss due to simultaneous transmissions and identifies patterns of contention.
 2. Frame Dropping Error Rate simulates conditions where frames are dropped due to congestion or errors. It Helps in evaluating network robustness and reliability.
 3. Message Latency determines network efficiency under varying load conditions. It can be influenced by frame size, bit stuffing, and bus congestion.
- **Ethernet** : Ethernet is a packet-switched, high-speed communication protocol widely used in automotive, industrial automation [FFRG08],[DFF⁺07], and networking environments [DFF⁺07]. Industrial Ethernet has emerged as a crucial area of study, particularly in the context of smart manufacturing and Industry 4.0. [IZ08]'s Ethernet case study in heterogeneous environments presents a comprehensive approach to modeling and analyzing network performance using queuing theory. It enables reliable, scalable, and high-bandwidth communication between devices, making it essential for real-time data exchange, control systems, and infotainment applications [WOLL17].

Timing characteristics and synchronization accuracy have been key focus areas in Ethernet simulation research. [DFF⁺07] evaluated timing characteristics of industrial Ether-

net networks synchronized via IEEE 1588 by introducing a new simulation model for the IEEE 1588 Precision Time Protocol (PTP) using the OPNET™ simulation environment, while [FFRG08] extended this work by examining clock synchronization accuracy across coexistent Real-Time Ethernet protocols by mentioning potential enhancements to the PTP tracking algorithms. The literature shows particular attention to specific Real-Time Ethernet protocols, as evidenced by [SZ08]’s simulation approach to EtherCAT. Their work demonstrated the development of a simulation model of EtherCAT networks using the OPNET simulation tool which can be used to evaluate the behavior of different EtherCAT configurations in industrial communication scenarios. [WOLL17]’s research on integrated Industrial Ethernet solutions further highlighted the protocol’s role in smart factory implementation, focusing on the integration of physical entities and information systems.

FERAL provides an Ethernet simulation model that captures key aspects of event-driven communication in switched and unswitched networks. It aligns with the IEEE 802.3 Ethernet standard [IEE22] and enables the simulation of communication patterns for protocol evaluation and performance analysis.

- Ethernet Network Architecture: The model supports both:
 1. Switched Ethernet implements store-and-forward switching using internal buffering to forward frames based on MAC addresses.
 2. Unswitched Ethernet emulates hub-like multi-cast behavior without switching logic.

Both models are instantiated as event-triggered FERAL Workers that interact through well-defined Tx and Rx message ports. Internally, the network architecture includes components such as the Network Topology, Medium, and Application Interfaces that define how simulation participants (nodes) are connected.

- Ethernet Communication and Addressing Mechanisms:
 1. Store-and-Forward Switching: In switched networks, Ethernet frames are buffered before forwarding to ensure correct transmission.
 2. Packet-Based Data Transfer: Unlike time-triggered networks like FlexRay, Ethernet uses variable-length frames with configurable payload sizes for efficient data exchange.
 3. MAC Address-Based Communication: Each Ethernet node communicates using MAC addresses.
 4. Multicast Behavior: In the unswitched model, multicast-style delivery is simulated. Frames are sent to all nodes, with each node responsible for handling relevant messages.
 5. Full-Duplex Communication enables concurrent transmission and reception between endpoints.
- Quality Metrics in FERAL Ethernet Simulation: Although FERAL does not enforce formal quality constraints, the following metrics can be assessed:

1. Transmission Speed and Throughput are influenced by bit rate and payload size. Higher configured bit rates reduce latency and increase throughput.
 2. Packet Delay Network Latency is calculated based on the interval between transmission and reception events which is useful for analyzing potential bottlenecks.
- **FlexRay:** FlexRay is a deterministic, time-triggered communication protocol designed for safety-critical applications in automotive systems, particularly in domains such as brake-by-wire, steer-by-wire, and powertrain control [KCA⁺10].

FlexRay has received significant attention in simulation research due to its importance in safety-critical automotive applications. [KCA⁺10] and [KKK⁺10] present the development of FlexRayXpert platforms. While both papers discuss FlexRayXpert.Sim for simulation, [KCA⁺10] additionally introduces FlexRayXpert.Lab for hardware integration. This heterogeneous co-simulation approach enables effective validation of simulation models against real-world hardware, enhancing result trustworthiness. [KCA⁺10] particularly emphasizes signal integrity analysis, examining factors such as cable length, termination, and environmental conditions, while [KKK⁺10] focuses on architectural decisions and efficient simulation methods for FlexRay-based automotive distributed embedded systems. Their other work [KASW10] further focused on the development of a holistic simulation approach using run-time model switching within the heterogeneous co-simulation model of FlexRay network. This approach allows for dynamic adjustments in the simulation model based on the current requirements, leading to improved performance and efficiency in simulations. [CB10] employs the Real-Time Calculus (RTC) framework to model the FlexRay bus and emphasizes the need for hard performance guarantees on critical properties such as worst-case response times, buffer requirements, and end-to-end latency for messages transmitted over the FlexRay bus.

Several researchers have focused on system-level verification of FlexRay. [KKAM08] presented a SystemC-based development and verification approach for the simulation of complex timing behaviors and various requirements of the FlexRay communication controller, while [MV10] conducted behavioral simulations for system verification specifically designed for Flexray communication networks to analyze various parameters that influence communication, such as network topology, cable lengths, and termination. This body of work demonstrates the importance of comprehensive verification methodologies in protocol simulation.

The FlexRay module in FERAL enables the simulation of FlexRay network protocol, adhering to the FlexRay specification [Fle10]. This simulation model captures key aspects of FlexRay protocols such as its dual channel architecture, high-speed data exchange, deterministic communication with flexible bandwidth allocation using both static and dynamic segments. Some of its key features are:

- Dual-Channel Communication for Fault Tolerance: FlexRay supports dual independent communication channels (Channel A and Channel B), so messages transmit-

ted on one channel do not interfere with messages on the other. This feature enables fault-tolerant redundancy, improving reliability in safety critical applications. Moreover, it allows parallel transmission on both channels simultaneously (for higher bandwidth).

– Time-Triggered and Event-Triggered Communication:

1. Static slots (time-triggered) enable Predefined, periodic message transmission ensuring deterministic communication.
2. Dynamic slots (event-triggered) allow flexible, on-demand message transmission for better bandwidth utilization.
3. Message addressing follows the <segment>:<slot> format, consistent with the FlexRay protocol.

– Message Properties for Enhanced Diagnostics: Each FlexRay frame in FERAL is enriched with metadata for traceability and debugging:

1. Startup Frame Indicator identifies frames used for network initialization.
2. Sync Frame Indicator helps in maintaining clock synchronization across nodes.
3. Null Frame Indicator detects unused slots or empty transmissions.
4. Payload Preamble Frame Indicator marks the presence of payload data.
5. Cycle Counter tracks the network cycle in which a frame was transmitted.

These properties enhance network diagnostics, fault analysis, and message tracking.

– FlexRay Network Configuration: FERAL allows network parameters to be configured to model realistic FlexRay protocol behavior.

1. Static Slot Duration determines the time allocated for static transmissions.
2. Number of Static Slots per Cycle specifies the number of preallocated time slots.
3. Symbol Window Duration configures optional symbol window length for special signaling.
4. Number of Macroticks per Cycle controls the synchronization and granularity of the time cycle.
5. Cycle Start (0–63 cycles) defines the starting point of a network cycle.

– Quality Metrics in FERAL FlexRay Simulation: Although FERAL does not explicitly define any quality criteria in specific, it implies several key attributes for the performance evaluation of FlexRay protocol.

1. Fault Tolerance: Its dual redundant channels improve reliability. As all network nodes can receive on both channels thus it provides fault tolerance.
2. Flexibility: Its dynamic segment enables event-driven communication with adaptable bandwidth allocation through configurable minislots. It allows independent channel selection (A, B, or BOTH) for traffic distribution and can utilize both channels simultaneously to optimize bandwidth usage.

- **LIN** : LIN is a low-cost, single-wire serial communication protocol designed for non-time-critical applications in automotive subsystems, such as window regulators, seat positioning, and climate control systems [ACD14]. Unlike CAN or FlexRay, LIN follows a simplified master-slave architecture, making it ideal for low-bandwidth, deterministic communication. LIN protocol simulations are typically explored in the context of lightweight and cost-effective communication. [ECN17] analyzed the reliability of LIN under electrostatic discharge conditions, revealing the protocol's vulnerabilities and strategies for mitigation. [ACD14] implemented a low-cost communication network, showcasing LIN's practical applications in automotive and industrial systems. [GK11] developed an Field Programmable Gate Arrays(FPGA)-based simulation for an adaptive front light system, underscoring LIN's potential in real-time control applications.

FERAL provides a LIN simulation model that captures some of the LIN's key features like Master-Slave based architecture, single-wire communication and schedule-based frame transmission, as per the LIN standard [LIN10].

- LIN Modules in FERAL: FERAL offers two distinct LIN simulation modules, each catering to different use cases.
 1. LIN Module: This module in FERAL implements an integrated LIN Master, which controls the entire network schedule. It supports schedule tables, provided either via a LIN Description File (LDF) or manually during simulation setup.
 2. LINPhys Module: This module delegates scheduling control to an external component. FERAL only simulates the network behavior, without controlling message timing.
- Master-Slave Architecture: LIN's master-controlled scheduling controls the network's communication through a scheduling mechanism. The Master Node controls the entire network by dictating message timing and order using a schedule table and all slave nodes respond only upon receiving a request from the master. This time-deterministic communication ensures consistent data transmission across networked nodes and simplifies the entire simulation setup.
- Supported LIN Message Types: FERAL supports multiple LIN frame types, each serving different communication needs.
 1. Unconditional Frames are regularly scheduled for master-slave communication.
 2. Sporadic Frames are sent only under specific conditions when required.
 3. Event-Triggered Frames allow multiple slave nodes to transmit without causing bus collisions.
- Quality Metrics in FERAL LIN Simulation: Although FERAL does not explicitly define any quality criteria in specific, it implies several key attributes for the performance evaluation of LIN networks.

1. **Scheduling Accuracy:** It's schedule-based communication and time-triggered approach assesses how well slave nodes align with the master using the sync break signal which is important for avoiding timing mismatches in scheduled communication.

So, based on each protocols' standards and features supported by the Fraunhofer FERAL documentation discussed above, we develop specific ontologies for the simulation model of each of these protocols. The following subchapters discuss the implementation of the four domain-level ontologies for respective communication protocols: CAN, LIN, Ethernet and FlexRay. The concepts within the domain-level ontologies were modeled based on two criteria: (1) semantic alignment with the standardized IDTA SMT "Provision of Simulation Models" and (2) structural support from mid-level ontology patterns (CSS, ISO 25010). The following subsections discuss the implementation of each ontology.

4.4.1 CAN Simulation Model

This subchapter details the implementation of the ontology module for the CAN — a robust, message-based protocol widely adopted in the automotive domain. Some key protocol semantics of CAN bus such as frame arbitration, bit rate configuration, error handling, and communication scheduling were derived from both the literature and the FERAL framework documentation. These details served as the conceptual foundation for formalizing a semantically consistent and simulation-aware ontology. To ensure interoperability and traceability, we adopted the IDTA SMT "Provision of Simulation Models" to structure the conceptual characteristics of the CAN simulation setup. This mapping was critical to translate domain-level simulation requirements into reusable semantic artifacts. The following section presents this mapping in detail.

Mapping of CAN Simulation Model characteristics to the attributes of IDTA SMT "Provision of Simulation Models":

The data below provides a comprehensive representation of the CAN simulation model as defined in the FERAL framework, mapped against the IDTA SMT "Provision of Simulation Models". This mapping identifies all relevant simulation model related configurations, parameters, capabilities, and environment details, which are later used in formalizing the ontology for CAN simulation model.

1. **Summary:** This simulation model simulates CAN protocol communication using COMNET. It supports CAN Standard, CAN FD, and CAN XL, with functionalities such as message arbitration, transmission timing, and error handling.
2. **SimPurpose:**
 - **PosSimPurpose:**

- Message frame transmission: multi-node setups with message broadcasting and arbitration.
 - Fault injection: Dropping duplicate frames, error detection through cyclic redundancy check (CRC).
 - Protocol behavior testing: Bit rate, payload size, and arbitration-based message prioritization.
 - NegSimPurpose: Not suitable for analog or physical hardware simulations beyond communication protocols.
3. TypeOfModel: Discrete-event network simulation using COMNET.
4. ScopeOfModel: Supports CAN Standard, CAN FD, and CAN XL, with the following physical characteristics:
- Physical Layer: Differential signaling (CAN_H and CAN_L) and bit-wise arbitration.
 - Logical Layer: Message arbitration, error detection, and transmission timing.
 - Bit Rates: Configurable arbitration and data bit rates.
 - Frame Types: Standard 11-bit, extended FD (up to 64 bytes), and CAN XL frames.
 - Standards Compliance: Based on ISO 11898 standards.
5. LicenseModel: FERAL has so far been available as a subscription license (annual payment).
6. Environment:
- OperatingSystem: Java, Python-supported operating systems, such as Windows, Linux.
 - ToolEnvironment: Java Runtime Environment (JRE) or JDK, FERAL module (COMNET).
 - DependencyEnvironment:
 - Software: Java and FERAL dependencies.
 - No specific hardware dependencies.
 - VisualizationInformation: No explicit visualization; supports transmission logging for analysis.
 - SimToolName: FERAL Framework (3rd Generation).
 - DependencySimTool: Requires Java runtime.
 - Compiler: Java version ≥ 11 .
 - SolverAndTolerances:
 - Solver settings: Discrete-event scheduler.
 - Tolerances: Configurable bit (data, arbitration) rates and payload sizes for timing control.

7. ModelFile:

- ModelFileType: Source code written in Java and Python for CAN simulation setup.
- ModelFileVersion: FERAL version associated with the CAN module.

8. ParamMethod: Parameterization via source code.

9. ParamFile: No parameter files; parameterization is handled programmatically.

10. InitStateMethod: State variables are initialized programmatically during simulation setup.

11. InitStateFile: No external files; initial states are defined programmatically.

12. DefaultSimTime: Simulation time is user-defined during setup; no default.

13. Ports:

- PortConnector:
 - PortConnectorName: "app1" (Port names uniquely identify Tx and Rx ports within the application interface).
 - PortConDescription:
 - * Tx Port: Encapsulates and transmits logic frames to the network.
 - * Rx Port: Processes received logic frames and forwards them to applications.
 - Variable:
 - * VariableName: Network Address (CAN ID), Payload Size (in bytes).
 - * VariableType: Integer for both variables.
 - * Range:
 - Payload Size: [0, 8] bytes (CAN Standard), [0, 64] bytes (CAN FD).
 - CAN ID: [0, 2047] (standard 11-bit IDs).
 - * VariableDescription:
 - Payload Size: Size of the payload data in the CAN frame.
 - CAN ID: Unique identifier for the CAN frame.
 - * UnitList: "bytes" for payload size, CAN ID: "No unit".
 - * VariableCausality: "Input" for Rx ports, "Output" for Tx ports.
 - * VariablePrefix: Not applicable.

Building upon the above IDTA SMT "Provision of Simulation Models"-aligned conceptual model, the ontology implementation encapsulates all simulation-relevant concepts as OWL classes, object properties, and data properties. This CAN domain ontology directly reuses and specializes classes from the mid-level ontology defined in the thesis to create a detailed, protocol-specific simulation model for CAN. It reuses core concepts from the ODPs of CSS (for Capability, Resource, Process), ISO 25010 (for Quality Attributes), and IDTA SMT "Provision

of Simulation Models", aligning them with specific CAN-based simulation requirements. The decision to model MessageFrameTransmission and FaultInjection as capabilities aligns with the simulation goals of evaluating data flow and protocol resilience. CANBus's arbitration and fault-tolerance mechanisms require an explicit representation of frame-level behavior and error conditions, justifying their elevation as first-class ontological constructs. Additionally, modeling CAN_Port as a Resource (equivalent to portsConnector) ensures interoperability with hardware-interface semantics used in IDTA SMT "Provision of Simulation Models". Refer to Table 5 which captures all the class definitions and semantic justifications of the ontology in detail. For example, CAN_SimulationModel is defined as a subclass of idta:SimulationModel (IDTA SMT "Provision of Simulation Models" specification), reused from the mid-level ontology where SimulationModel refers to a digital artifact intended for simulation execution and reuse. By subclassing from the mid-level concept of class idta:SimulationModel, it gains the necessary semantic contracts for modeling the complete simulation of CAN-based communication and serves as the root of all CAN-specific simulation assertions and encapsulates metadata and behaviors under a reusable entity.

Class	Subclass Of	Description	Justification
CAN_SimulationModel	idta:SimulationModel	Represents the full CAN simulation model for multi-node communication, including fault injection and message transfer.	Establishes this model as a subtype of simulation model aligned with the IDTA mid-level definition.
CAN_Node	css:Resource	Denotes ECUs or individual devices that participate in CAN communication .	Each node acts as a resource that can both transmit and receive messages on the bus, consistent with the CSS Resource definition.
CAN_Port	css:Resource, equivalentClass to idta:portsConnector	Defines interfaces through which CAN nodes communicate.	Ports are specialized connection resources, reused here to match hardware design patterns.
FaultInjection	css:Capability, equivalentClass to idta:simPurpose	Enables injecting network faults into the CAN simulation model protocol for testing resilience.	This capability enables testing of systems' resilience and response to various failures and errors through the simulation of a wide range of fault conditions.
MessageFrameTransmission	css:Capability, equivalentClass to idta:simPurpose	Represents the communication capability to send/receive messages across the CAN bus.	As part of this capability, messages are broadcast to all nodes on the bus.
Automated-EmergencyBrake	vdI3633:Simulation	Represents the simulation scenario specific to Automated Emergency Braking over the CAN bus.	This subclass specifies a simulation scenario aligned with VDI 3633. It provides semantic anchoring of scenario-specific validation and query capabilities.
SimulationSettings	—	Includes parameters for simulation execution (speed, distance, duration, etc.).	Separates configuration metadata from execution logic for clear parameterization.
Message-Transmission	css:Process	Captures the step-by-step behavior of sending a message frame.	A defined simulation process, consistent with CSS and BFO "process" modeling.
Reliability	Equivalentclass to iso25010:-Reliability	Encodes robustness of CAN bus communication under varying conditions.	Reuses ISO definition directly to ensure semantic fidelity.
Transmission-Efficiency	iso25010:Performance-Efficiency	Indicates the data throughput and timing accuracy of simulation.	Adds performance-oriented evaluation of CAN messaging.

Table 5: Class Definitions and Semantic Justifications for CAN simulation model Ontology.

Relationships between the CAN simulation model and its nodes, capabilities, and configuration parameters are defined using object properties inherited from mid-level patterns and extended as needed as they establish semantic relationships between classes to effectively model system architecture, communication patterns, and role-based capabilities. They are foundational for expressing structural and behavioral dependencies in the system and those details are captured in Table 6. For example, the object property 'hasCapability' links the CAN_SimulationModel (Domain) to the capabilities FaultInjection and MessageFrameTrans-

mission (Range). These align with the simulation objectives of CAN Bus for testing its functionalities.

Property	Domain	Range	Description	Justification
hasPort	CAN_Node	CAN_Port	Links CAN node to physical/virtual ports for communication.	Captures hardware-level connectivity.
hasSettings	CAN_SimulationModel	SimulationSettings	Associates the CANBus with its operational parameters.	Enables parameterization and reuse.
communicatesWith	MessageFrame Transmission	MessageFrame Transmission	Models TX-RX pairs for two-way communication.	Supports automated discovery of protocol-level communication chains.
usesResource	CAN_Simulation Model	CAN_Node	Establishes the CAN simulation model's dependency on specific ECUs/devices.	Inherited relationship from the mid-level ontology to anchor simulation model to its participants.
hasCapability	CAN_Simulation Model	FaultInjection, MessageFrame Transmission	Declares supported capabilities of the CAN simulation model.	Inherited relationship from the mid-level ontology.
providesCapability	CAN_Node	FaultInjection, MessageFrame Transmission	Specifies the functionalities offered by each CAN node.	Inherited relationship from the mid-level ontology. Connects ECUs with functional roles.

Table 6: Object Properties and Their Roles in CAN Simulation model Ontology.

Datatype properties in ontologies are used to link classes to data values such as numbers, strings, or booleans. They help capture specific, often quantitative or descriptive attributes of instances that are not conceptually complex enough to warrant full semantic modeling as classes. In the context of our CAN domain-level simulation model ontology, datatype properties serve to define configuration parameters and operational characteristics of the simulation environment and those details are captured in Table 7. For example, the datatype property 'hasBitRate' links the CAN_SimulationModel to a literal value of type xsd:integer. This models the configured data rate for message transmission over the CAN bus (e.g., 500000 bits per second).

This structure ensures the ontology is formally robust, semantically consistent, and practically useful for driving simulation and validation tasks in a co-simulation environment involving CAN communication protocols.

Property	Domain	Range	Purpose
hasBitRate	CAN_SimulationModel	xsd:integer	Defines the data rate of CAN protocol transmission i.e. the speed at which the actual payload data is transmitted on the CAN bus.
hasBitStuffingMode	CAN_SimulationModel	xsd:string	Indicates how bits are configured within the Data field of a CAN frame for error resilience during data transmission.
hasLogTransmissions	CAN_SimulationModel	xsd:boolean	Toggles message logging for traceability.
hasName	CAN_Node	xsd:string	Names each Electronic Control Unit (ECU) for identification.
hasDropDuplicatedFrames	CAN_Node	xsd:boolean	Determines whether duplicate CAN frames with the same network address should be dropped or allowed through the network.
hasStartTimeMillis, hasEndTimeMillis	FaultInjection	xsd:integer	Sets the window for fault injection.
hasErrorModel	FaultInjection	xsd:string	Represents the type of failure to be simulated.
hasErrorProcessor	FaultInjection	xsd:string	Determines how the error is applied to network messages.
hasAddress	MessageFrame-Transmission	xsd:string	defines the CAN ID of a frame used during message transmission.
hasPortName	MessageFrame-Transmission	xsd:string	Name of the port of respective nodes during message transmission for port identification.
hasSimulatedPayloadSize	MessageFrame-Transmission	xsd:integer	Enables the transmission of high-level payload objects.
hasType	MessageFrame-Transmission	xsd:string	Encodes Defines transmit (Tx) and receive (Rx) ports during message transmission.
hasSpeedMetersPerSecond	SimulationSettings	xsd:float	Represents the initial speed of the vehicle in meters per second when the emergency braking simulation begins.
hasStartingDistance-Meters	SimulationSettings	xsd:float	Indicates the initial distance between the vehicle and the obstacle/stopping point in meters at the start of the simulation.
hasSensorTimeStepNs	SimulationSettings	xsd:integer	Defines the time step for sensor sampling in nanoseconds during simulation.
hasMinimumBraking-Distance	SimulationSettings	xsd:float	Represents the theoretical minimum distance required to bring the vehicle to a complete stop under ideal conditions, measured in meters.
hasSimulationDuration	SimulationSettings	xsd:float	Defines the total duration of the simulation in seconds.
hasBrakingDuration	SimulationSettings	xsd:float	Specifies the time required for the complete braking process in seconds.

Table 7: Datatype Properties for Simulation Configuration in CAN Simulation model Ontology.

4.4.2 LIN Simulation Model

Building on the conceptual foundation, the LIN simulation model represents a lightweight, master-slave based communication protocol tailored for deterministic and cost-sensitive applications in automotive systems. This implementation aims to semantically model the structure, behavior, and capabilities of LIN communication protocol, aligning it with mid-level standards defined in the ontology architecture.

Mapping of LIN Simulation Model Characteristics to the attributes of IDTA SMT "Provision of Simulation Models":

To ensure semantic interoperability, we adopted the IDTA SMT "Provision of Simulation Models" as the reference structure for mapping key simulation properties derived from the FERAL LIN simulation modules. These mappings translate simulation-relevant aspects — such as schedule-based frame transmission, communication timing, and network configuration — into well-defined semantic descriptors that can be reused, queried, or reasoned over within the co-simulation framework.

1. Summary: Simulation of LIN protocol communication with master-slave configuration,

supporting unconditional, sporadic, and event-triggered frames, with compatibility for LIN 2.0+.

2. SimPurpose:

- PosSimPurpose:
 - Enable LIN master-slave scheduling and communication.
 - Simulate transmission of frames like unconditional, sporadic, and event-triggered.
- NegSimPurpose:
 - Not suitable for LIN versions prior to 2.0.
 - Cannot simulate diagnostic or reserved frame types.

3. TypeOfModel:

- Event-Driven Communication.

4. ScopeOfModel: Simulation of physical characteristics such as bit rate, timing intervals (inter-byte space, response space, time base), synchronization parameters (sync break and delimiter), and network configuration (number of slaves).

5. LicenseModel: FERAL has so far been available as a subscription license (annual payment).

6. Environment:

- OperatingSystem: Java, Python-supported operating systems, such as Windows, Linux.
- ToolEnvironment: Java Runtime Environment (JRE) or JDK, FERAL Framework (including LIN and LINPhys modules).
- DependencyEnvironment:
 - Hardware: Master-slave node configuration (1 master, up to 16 slaves).
 - Software: LDF files or manually defined schedules.
- VisualizationInformation: Text-based logging of message transmissions and scheduling.
- SimToolName: FERAL Framework (3rd Generation).
- DependencySimTool: Java Runtime Environment (JRE) or JDK, FERAL LIN modules.
- Compiler: Java version ≥ 11 .
- SolverAndTolerances:
 - Solver settings: Discrete-event scheduler for time-triggered and event-driven communication.
 - Tolerances: Configurable time base, jitter range, and inter-byte space.

7. ModelFile:

- ModelFileType: LDF File (LIN-specific exchange format), Source Code (Java) [manual configuration].
- ModelFileVersion:
 - LDF File Versions: LIN 2.0 or 2.2A compliant LDF files.
 - Source Code Versions: FERAL library version determines simulation model updates.

8. Ports:

- PortConnector:
 - PortConnectorName: "linID5_out" (Rx port for message ID 5) and "linID5_in" (Tx port for message ID 5).
 - PortConDescription:
 - * Rx port: "Receives message ID 5 from master interface."
 - * Tx port: "Transmits message ID 5 to slave interface."
 - Variable:
 - * VariableName: "Message ID" and "Payload Size".
 - * VariableType: "Integer" for Message ID, "Byte Array" for Payload.
 - * Range: [0, 61] for Message ID, [0, 8] for Payload Size.
 - * VariableDescription:
 - Message ID: Unique identifier for each LIN frame.
 - Payload Size: Defines the size of the transmitted LIN frame in bytes.
 - * UnitList: "ms" for timing variables, "bytes" for payload size.
 - * VariableCausality: "Input" for Rx ports, "Output" for Tx ports.
 - * VariablePrefix: Not applicable (FERAL LIN uses causal connections only).
- BinaryConnector: FERAL documentation does not explicitly mention binary interfaces for LIN.

9. ParamMethod: Parameterization required via LDF files or manual configuration using source code.

10. ParamFile: LDF (LIN Description File) for model parameterization.

11. initStateMethod: Initialization of state variables via LDF files or manual configuration of timing and network properties.

12. initStateFile: LDF (LIN Description File) for initializing network and timing parameters.

13. DefaultSimTime: No predefined simulation time; user specifies duration in seconds during simulation setup.

Building upon the above IDTA SMT "Provision of Simulation Models"-aligned conceptual model, the ontology implementation encapsulates all simulation-relevant concepts as OWL classes, object properties, and data properties. The LIN simulation model ontology specializes the mid-level concepts from the ODPs of CSS, ISO 25010, and IDTA SMT "Provision of Simulation Models" for modeling LIN simulation model specific semantics. The ontology encapsulates domain knowledge using OWL 2 DL constructs and aligns simulation behavior with IDTA SMT "Provision of Simulation Models" primitives, supporting reuse in configuration and validation tasks. The core components of the ontology are structured as follows. Table 8 captures the class definitions and semantic justifications for the ontology. For example, LIN_SimulationModel is defined as a subclass of idta:SimulationModel (IDTA SMT "Provision of Simulation Models" specification), modeling the complete simulation of LIN-based communication. By subclassing from the mid-level concept of class idta:SimulationModel, it gains the necessary semantic contracts and serves as the root of all LIN-specific simulation assertions and encapsulates an entire communication setup involving a master-slave architecture. Relationships between the LIN simulation model and its nodes, capabilities, and configuration parameters are defined using object properties inherited from mid-level patterns and extended as needed as they establish semantic relationships between classes to effectively model system architecture, communication patterns, and role-based capabilities. We captured those details in Table 9. For example, the object property 'hasCapability' links the LIN_SimulationModel (Domain) to one or more capabilities — e.g., MasterSlaveScheduling and FrameTransmission (Range). These capabilities represent key functional behaviors expected from the LIN protocol. On the other hand, Table 10 capture the datatype properties of the ontology which is needed to define configuration parameters and operational characteristics of the simulation environment for LIN. For example, the datatype property 'hasBitRate' connects the LIN_SimulationModel to a value (of type xsd:integer) representing the data rate of LIN transmission.

Class	Subclass Of	Description	Justification
LIN_SimulationModel	idta:SimulationModel	Represents the LIN simulation Model as a subclass of supporting master-slave frame exchanges.	Establishes this model as a sub type of simulation model aligned with the IDTA mid-level definition.
LIN_MasterNode	css:Resource	Represents the Master device that controls scheduling and initiating all communication.	Encodes the role of a LIN master as a unique resource with scheduling responsibility.
LIN_SlaveNode	css:Resource	Represents devices that respond to master device's frames.	Captures limited operational scope of LIN slaves consistent with resource abstraction.
LIN_Port	css:Resource, equivalentClass to idta:portsConnector	Interfaces that transmit and receive LIN frames.	Represents LIN ports as communication interfaces, consistent with port modeling across protocols.
MasterSlave-Scheduling	css:Capability, equivalentClass to idta:simPurpose	Validation of LIN master-slave scheduling and communication where only the master can initiate communication and slaves respond when polled by master.	Reflects core LIN mechanism of deterministic master-driven scheduling.
FrameTransmission	css:Capability, equivalentClass to idta:simPurpose	Simulates different type of frame communication.	Encodes LIN protocol's core message behaviors essential for simulation.
MasterScheduled-Communication	css:Process	Represents scheduling governed by the LIN master.	Describes temporal control over network traffic as defined in LDF.
SchedulingAccuracy	iso25010:Performance Efficiency	Represents timing precision of LIN message flow.	Uses ISO-aligned metric to assess fidelity of time-triggered LIN simulation.

Table 8: Class Definitions and Semantic Justification in LIN Simulation Model Ontology.

Property	Domain	Range	Description	Justification
hasCapability	LIN_SimulationModel	MasterSlave-Scheduling, FrameTransmission	Connects model with LIN-specific capabilities like scheduling or frame transmission.	Defines purpose-specific simulation capability to capture simulation model features based on protocol functionality.
usesResource	LIN_SimulationModel	LIN_MasterNode, LIN_SlaveNode	Links the simulation model to components like master/slave nodes to employ simulation.	Inherited from mid-level to capture the dependency of simulation model with its resources to enable the communication.

Table 9: Object Properties and Their Roles in LIN Simulation Model Ontology.

Property	Domain	Range	Purpose
hasBitRate	LIN_SimulationModel	xsd:integer	Defines data rate for frame transmission.
hasTimeBase	LIN_SimulationModel	xsd:integer	Represents the basic timing unit for scheduling.
hasScheduleStartDelay	LIN_SimulationModel	xsd:integer	Time offset for initiating schedule.
hasPayloadSize	LIN_Port	xsd:integer	Size of data in the LIN message frame.
hasFrameID	LIN_Port	xsd:integer	Unique identifier for a specific LIN frame.

Table 10: Datatype Properties for Simulation Configuration in LIN Simulation Model Ontology.

This implementation formalizes the LIN simulation model using OWL semantics, linking it directly to the mid-level ontology. By integrating frame structure, port behavior, scheduling semantics, and quality attributes, the LIN simulation model ontology module facilitates automated reasoning and simulation configuration in cost-sensitive automotive scenarios.

4.4.3 Ethernet Simulation Model

The Ethernet domain-level ontology focuses on semantically capturing the communication behavior of Ethernet-based systems in co-simulation settings. The ontology design aims to capture information for both switched and unswitched Ethernet networks, full-duplex communication, and packet-based data flow. This implementation extends mid-level semantic abstractions for capabilities, processes, resources, and quality metrics, and grounds them in the event-driven modeling framework of the FERAL simulation environment.

Mapping of the Ethernet Simulation Model Characteristics to the attributes of IDTA SMT "Provision of Simulation Models":

To ensure semantic interoperability, we adopted the IDTA SMT "Provision of Simulation Models" as the reference structure for mapping key simulation properties derived from the FERAL Ethernet simulation modules. The following mapping captures how the Ethernet simulation model, as implemented in FERAL, corresponds to the IDTA SMT "Provision of Simulation Models" attributes.

1. Summary: The Ethernet module simulates full-duplex switched and unswitched (hub) Ethernet networks. It supports event-driven modeling for switched Ethernet (store-and-forward) and unswitched Ethernet (multicast).
2. SimPurpose:
 - PosSimPurpose:
 - Simulation of switched Ethernet networks for store-and-forward communication.

- Simulation of unswitched Ethernet networks for multicast communication.
 - Event-driven Ethernet network simulation.
 - Full-duplex communication testing.
 - Ethernet topology analysis and performance testing.
 - Fast Ethernet.
 - NegSimPurpose:
 - Resource contention or collision modeling.
 - Transmission jitter simulation.
 - Physical-layer effects (e.g., cable properties).
 - Real-time or hardware-in-the-loop simulations.
3. TypeOfModel: Discrete Event Simulation.
4. ScopeOfModel:
- Models full-duplex communication.
 - Simulates switched and unswitched Ethernet topologies.
 - Represents Ethernet packet transmission, including MAC addresses and payload size.
 - Models timing properties, such as bit rate.
5. LicenseModel: FERAL has so far been available as a subscription license (annual payment).
6. Environment:
- OperatingSystem: Java, Python-supported operating systems, such as Windows, Linux.
 - ToolEnvironment: Java Runtime Environment (JRE 11), FERAL Framework with Ethernet modules.
 - DependencyEnvironment: No specific hardware dependencies. Software dependencies include event-triggered simulations using the FERAL Ethernet framework.
 - VisualizationInformation: Visualization limited to logging of network messages and configurations.
 - SimToolName: FERAL Framework (3rd Generation).
 - DependencySimTool: Java Runtime Environment (JRE) or JDK, FERAL Ethernet modules.
 - Compiler: Java version ≥ 11 .
 - SolverAndTolerances:
 - Solver settings: Discrete-event scheduler (MOCC).

- Tolerances: Bit rate and payload size impact transmission and reception timing.

7. ModelFile:

- ModelFileType: Source Code (Java).
- ModelFileVersion: FERAL Ethernet module version depends on the FERAL library release.

8. ParamMethod: Parameterization via source code.

9. ParamFile: No predefined parameter file; parameterization is done programmatically in Java.

10. InitStateMethod: Initialization of state variables (e.g., bit rate, payload size) via source code during setup.

11. InitStateFile: No predefined initialization file; initial states are programmatically set during simulation setup.

12. DefaultSimTime: No predefined simulation period; users set it explicitly.

13. Ports:

- PortConnector:
 - PortConnectorName: "myInterfacePortName" (for transmission and reception of data frames).
 - PortConDescription:
 - * Tx: Sends Ethernet frames to the specified network address.
 - * Rx: Receives Ethernet frames from the specified network address.
 - Variable:
 - * VariableName: Network Address (MAC address), Payload Size (in bytes).
 - * VariableType: Network Address is String, Payload Size is Integer.
 - * Range:
 - Network Address: No defined range, but must follow MAC address format.
 - Payload Size: [0, MAX_PAYLOAD].
 - * VariableDescription:
 - Network Address: Specifies the MAC address of the Ethernet node.
 - Payload Size: Defines the size of the transmitted or received Ethernet frame.
 - * UnitList: "bytes" for payload size.
 - * VariableCausality: "Input" for Rx ports, "Output" for Tx ports.
 - * VariablePrefix: Not applicable.

Building upon the above IDTA SMT "Provision of Simulation Models"-aligned conceptual model, the ontology implementation encapsulates all simulation-relevant concepts as OWL classes, object properties, and data properties. We developed the Ontology using OWL 2 DL which semantically represents all relevant Ethernet communication behaviors and their simulation setup. This ontology specializes mid-level concepts from the ODPs of CSS, IDTA SMT "Provision of Simulation Models", and ISO25010, and connects them to concrete simulation behaviors. Table 11 captures the Class Definitions and Semantic Justifications for the Ontology. For example, `Ethernet_SimulationModel` is defined as a subclass of `idta:SimulationModel`, inheriting the conceptual structure for digital artifacts designed for simulation. In this case, it models all relevant Ethernet simulation behaviors — switched (store-and-forward) and unswitched (multicast). Relationships between the Ethernet simulation model and its nodes, capabilities, and configuration parameters are defined using object properties inherited from mid-level patterns and extended as needed as they establish semantic relationships between classes to effectively model system architecture, communication patterns, and role-based capabilities. We captured those details in Table 12. For example, the 'hasCapability' object property connects the `Ethernet_SimulationModel` (Domain) to capabilities such as `StoreAndForwardCommunication` and `MulticastCommunication` (Range). These capture fundamental behaviors modeled in Ethernet networks. On the other hand, Table 13 capture the datatype properties of the ontology which is needed to define configuration parameters and operational characteristics of the simulation environment for Ethernet. For example, The datatype property 'hasMACAddress' connects the `Ethernet_Port` to a value (of type `xsd:string`) representing the MAC address of Ethernet Port.

Class	Subclass Of	Description	Justification
<code>Ethernet_SimulationModel</code>	<code>idta:SimulationModel</code>	Represents the simulation model for Ethernet-based communication, supporting various transmission modes.	Anchors Ethernet-specific configurations into the simulation model template.
<code>Ethernet_Node</code>	<code>css:Resource</code>	Denotes individual devices communicating via Ethernet.	Captures communication participants as reusable resources which is consistent with the CSS ontology definition of resources.
<code>Ethernet_Port</code>	<code>css:Resource</code> , equivalentClass <code>idta:portsConnector</code>	Communication interface connecting applications to the network.	Represents connection points in layered simulation to facilitate packet exchange between components with Ethernet-specific properties.
<code>StoreAndForwardCommunication</code>	<code>css:Capability</code> , equivalentClass <code>idta:simPurpose</code>	Models switched Ethernet behavior for packet transmission.	Encodes Ethernet protocol's core behavior for evaluation of buffering and switching logic.
<code>MulticastCommunication</code>	<code>css:Capability</code> , equivalentClass <code>idta:simPurpose</code>	Models unswitched, hub-based message forwarding of Ethernet.	Reflects the broadcasting features used in simple Ethernet setups.
<code>Ethernet_PacketTransmission</code>	<code>css:Process</code>	Simulates the process of transmitting Ethernet packets across the network.	Formalizes discrete-event behavior of Ethernet at the simulation layer.
<code>Efficiency</code>	<code>iso25010:PerformanceEfficiency</code>	Evaluates throughput and latency performance in Ethernet simulation.	Enables quality-driven assessment of data transmission using the ISO standard.
<code>Reliability</code>	Equivalent to <code>iso25010:Reliability</code>	Describes robustness and packet delivery integrity.	Uses ISO-aligned terminology for fault-tolerance validation.

Table 11: Class Definitions and Semantic Justification in Ethernet Simulation model Ontology.

So, this ontology-driven implementation of the Ethernet simulation model demonstrates how protocol-specific semantics can be embedded into a reusable and interoperable semantic model. By mapping key simulation concerns to the IDTA SMT "Provision of Simulation Models" and expressing them through OWL constructs, the Ethernet domain level ontology serves as

Property	Domain	Range	Description	Justification
hasCapability	Ethernet_SimulationModel	StoreAndForward Communication, Multicast Communication	Declares simulation model's functionality like multicast or switched communication.	Inherited relationship from the mid-level ontology to add semantic clarity to simulation goals.
usesResource	Ethernet_SimulationModel	Ethernet_Node	Connects Ethernet Simulation model to participating network nodes.	Inherited relationship from the mid-level ontology to anchor simulation model to its participants that establishes resource dependencies in simulation.

Table 12: Object Properties and Their Roles in Ethernet Simulation model Ontology

Property	Domain	Range	Purpose
hasPayloadSize	Ethernet_Port	xsd:integer	Indicates the size of the data packet transmitted.
hasMACAddress	Ethernet_Port	xsd:string	Specifies the MAC address of a given Ethernet port.
hasBitRate	Ethernet_SimulationModel	xsd:integer	Defines data rate for packet transmission.
hasLogTransmissions	Ethernet_SimulationModel	xsd:boolean	Toggles logging of transmission events for traceability.

Table 13: Datatype Properties for Simulation Configuration in Ethernet Simulation model Ontology

a foundational asset for simulation configuration, performance analysis, and co-simulation orchestration in complex automotive and industrial systems.

4.4.4 FlexRay Simulation Model

This subchapter details the domain-level ontology implementation for the FlexRay protocol within the FERAL co-simulation environment. The ontology captures the protocol's deterministic timing, fault-tolerant dual-channel communication, and hybrid scheduling mechanisms (static and dynamic slots), while semantically aligning with the IDTA SMT "Provision of Simulation Models", the CSS mid-level ontology, and ISO 25010 quality attributes.

Mapping of the FlexRay Simulation Model Characteristics to the attributes of IDTA SMT "Provision of Simulation Models":

To ensure semantic interoperability, we adopted the IDTA SMT "Provision of Simulation Models" as the reference structure for mapping key simulation properties derived from the FERAL FlexRay simulation modules. The following mapping captures how the FlexRay simulation model, as implemented in FERAL, corresponds to the IDTA SMT "Provision of Simulation Models" attributes.

1. Summary: Simulates FlexRay networks with deterministic time-triggered communication with event-driven flexible bandwidth allocation, supporting independent channels (A and B), static and dynamic slots, and fully compliant with FlexRay Protocol Specification Version 3.0.1.
2. SimPurpose:
 - PosSimPurpose:
 - Event-Driven simulation for the allocation of bandwidth only when needed (efficient use of resources).

- Dual-channel communication to provide redundancy for safety-critical applications and allow parallel transmission on both channels simultaneously.
 - Deterministic communication testing through time-triggered behavior.
 - NegSimPurpose: Not suitable for Physical hardware-level simulations beyond protocol-level behavior.
3. TypeOfModel: Event-driven network simulation.
4. ScopeOfModel:
- Dual-channel support (A and B) for fault tolerance.
 - Time-triggered (static slots) and event-triggered (dynamic slots) scheduling.
 - Compliant with FlexRay v3.0.1 specifications.
5. LicenseModel: FERAL has so far been available as a subscription license (annual payment).
6. Environment:
- OperatingSystem: Cross-platform: Windows, Linux.
 - ToolEnvironment: Java Runtime Environment (JRE) or JDK, FERAL Framework with FlexRay modules.
 - DependencyEnvironment: No specific hardware dependencies; software depends on Java runtime and FERAL libraries.
 - VisualizationInformation: Visualization limited to logging transmission behavior.
 - SimToolName: FERAL Framework (3rd Generation).
 - DependencySimTool: Java Runtime Environment (JRE) or JDK, FERAL FlexRay modules.
 - Compiler: Java version ≥ 11 .
7. ModelFile:
- ModelFileType: Source Code (Java).
 - ModelFileVersion: Determined by the FERAL FlexRay library version.
8. ParamMethod: Parameters involved in the simulation are configured programmatically.
9. ParamFile: No parameterization files are required.
10. InitStateMethod: Initialization of state variables happen (e.g., starting cycle) via source code during setup.
11. InitStateFile: No external files; initial states are defined programmatically for simulation startup.
12. DefaultSimTime: No predefined simulation period; simulation time is user-specified.

13. Ports:

- PortConnectorName: "app1" (used for both transmission and reception).
- PortConDescription:
 - Tx Port: Transmits FlexRay frames according to defined slots and cycles.
 - Rx Port: Receives FlexRay frames tagged with slot and cycle metadata.
- Variable:
 - VariableName: "Payload Size", "Slot Type", "Cycle Count".
 - VariableType: "Payload Size": Integer, "SlotType": String (s:, d:), "Cycle Count": Integer.
 - Range: "Payload Size": [0, 254] (maximum FlexRay payload), "Cycle Count": [0, 63].
 - VariableDescription: "Payload Size": Size of the payload data in the frame. "Slot Type": Type of slot (static or dynamic) associated with the port. "Cycle Count": FlexRay cycle counter value for the message.
 - UnitList: "bytes" for payload size..
 - VariableCausality: "Input" for Rx ports, "Output" for Tx ports.
 - VariablePrefix: Not applicable .
- BinaryConnector: No direct mapping; no mention of binary visualization interfaces in the FlexRay documentation.

Building upon the above IDTA SMT "Provision of Simulation Models"-aligned conceptual model, the FlexRay domain ontology was implemented using OWL 2 and extends the mid-level concepts from the ODPs of CSS (Capability, Resource, Process), and ISO 25010 (Quality Attributes) through the definition of relevant OWL classes, object properties, and data properties, while being compliant with the IDTA SMT "Provision of Simulation Models". The ontology enables semantic representation of FlexRay-specific properties such as dual-channel communication, hybrid scheduling, deterministic transmission, and simulation configuration. Below you will find the breakdown of the ontology's components. Table 14 captures the Class Definitions and Semantic Justifications for the Ontology. For example, FlexRay_SimulationModel is defined as a subclass of the idta:SimulationModel class and models time-triggered and event-driven simulation setups typical of FlexRay networks. Relationships between the FlexRay simulation model and its nodes, capabilities, and configuration parameters are defined using object properties inherited from mid-level patterns and extended as needed as they establish semantic relationships between classes to effectively model system architecture, communication patterns, and role-based capabilities. We captured those details in Table 15. For example, the property 'hasCapability' links the FlexRay_SimulationModel (Domain) to its capabilities such as TimeTriggeredCommunication, EventDrivenCommunication, and DualChannelCommunication (Range). This makes it possible to semantically describe functional behavior of FlexRay. On the other hand, Table 16 captures the datatype properties of the ontology which

is needed to define configuration parameters and operational characteristics of the simulation environment for FlexRay. For example, the property 'hasSegmentType' is used to capture the type of segment (static or dynamic) in a given FlexRay_Port. The string value indicates how the message is scheduled, which is central to hybrid scheduling strategies in FlexRay.

Class	Subclass Of	Description	Justification
FlexRay_SimulationModel	idta:SimulationModel	Represents the full FlexRay simulation model for with time-triggered deterministic communication and Event-Driven flexible bandwidth allocation.	Establishes this model as a sub type of simulation model aligned with the IDTA mid-level definition.
FlexRay_Node	css:Resource	ECU or device node that participates in FlexRay communication.	Represents simulation resources that transmit/receive messages using FlexRay channels.
FlexRay_Port	css:Resource, equivalentClass to idta:portsConnector	Ports that interface application logic with network slots by combining segment type and slot number.	Represents connection points in layered simulation to facilitate message exchange between components with FlexRay-specific properties.
TimeTriggered-Communication	css:Capability, equivalentClass to idta:simPurpose	Enables periodic message transmission using static slots.	Captures time-critical behavior of the FlexRay protocol essential for safety-critical applications.
EventDrivenCommunication	css:Capability, equivalentClass to idta:simPurpose	Supports event-triggered messaging using dynamic slot window.	Represents flexibility in bandwidth usage which provides efficient resource utilization for less critical data.
DualChannelCommunication	css:Capability, equivalentClass to idta:simPurpose	Allows redundant, simultaneous transmission across channels A and B.	Captures fault-tolerant and high-availability aspects of FlexRay.
HybridCommunication	css:Process	Execution process combining static and dynamic slot scheduling.	Encodes execution logic for complete FlexRay cycle behavior.
Flexibility	iso25010:Portability	Describes model's adaptability and configurability with dynamic segment for adaptable bandwidth allocation.	Reuses ISO-defined portability characteristics to assess model modularity.
Reliability	EquivalentClass to iso25010:Reliability	Evaluates robustness and fault-tolerance of transmission.	ISO-defined metric reused for semantic fidelity to highlight simulation robustness under dual-channel redundancy.

Table 14: Class Definitions and Semantic Justification in FlexRay Simulation Model Ontology

Property	Domain	Range	Description	Justification
hasCapability	FlexRay_SimulationModel	TimeTriggeredCommunication, EventDrivenCommunication, DualChannelCommunication	Links FlexRay simulation model to protocol-level features.	Defines purpose-specific simulation capability.
usesResource	FlexRay_SimulationModel	FlexRay_Port	Associates the simulation model with respective ports to enable frame transmission.	Inherited from mid-level to capture the dependency of simulation model with its resources to enable the communication.

Table 15: Object Properties and Their Roles in FlexRay Simulation Model Ontology

Property	Domain	Range	Purpose
hasPayloadSize	FlexRay_Port	xsd:integer	Sets data size in bytes per port transaction for transmitted frames.
hasSlotValue	FlexRay_Port	xsd:integer	Indicates slot identifier for scheduling. (e.g., "s:3" for static slot 3).
hasSegmentType	FlexRay_Port	xsd:string	Distinguishes static (s) and dynamic (d) segments.
hasBitRate	FlexRay_SimulationModel	xsd:integer	Defines data rate for frame transmission.
hasGdStaticSlot	FlexRay_SimulationModel	xsd:integer	Specifies the number of static slots for time-triggered frames.

Table 16: Datatype Properties for Simulation Configuration in FlexRay Simulation Model Ontology

This ontology-based implementation formalizes the domain-specific semantics of FlexRay simulation model by structuring capabilities, processes, and resources, quality attributes within an interoperable OWL representation. It supports rigorous validation and configuration of hybrid communication patterns, dual-channel redundancy, and deterministic scheduling strategies within the FERAL co-simulation environment. This aligns FlexRay's protocol behavior

with the IDTA SMT "Provision of Simulation Models", thereby enabling semantic reuse, scalability, and integration in distributed simulation workflows.

In summary, the domain-level ontologies for CAN, LIN, Ethernet, and FlexRay Simulation models were developed to semantically represent the key simulation aspects of each protocol. These implementations reflect not only the structural and functional diversity across communication protocols but also highlight the reusability and adaptability of the mid-level ontology constructs. Common patterns such as Resource, Capability, and Process from the CSS ontology, alongside quality attributes from ISO 25010, allowed consistent modeling of simulation concepts while supporting protocol-specific customizations such as time-triggered communication in FlexRay or master-slave scheduling in LIN. These ontologies are designed to bridge simulation model configurations with semantic reasoning, providing the foundation for capability-based simulation orchestration in co-simulation environments. The resulting formal models ensure that both structural fidelity and simulation behavior are captured in a machine-interpretable form.

4.5 Implementation of the Use-case-level Ontology

The use-case-level ontology builds on communication protocol-specific details defined at the domain level (e.g., CAN) as this is the domain of Fraunhofer IESE and in context of virtual testbeds we are simulating communication networks which you can see in Figure 4, enabling the configuration of simulation models for concrete validation scenarios. By maintaining hierarchical consistency across all layers — from abstract concept definitions to scenario-specific instances — this ontology serves as the application-focused ABox complementing the domain-level TBox.

The selected simulation scenario is AEB, a safety-critical system that detects obstacles and initiates automatic braking to prevent collisions, as illustrated in the Figure 8. In modern vehicles, such braking decisions are supported by a network of ECUs that exchange sensor and control data over a communication bus — commonly CAN. Thus, this scenario aligns naturally with the thesis's domain-level focus on communication protocols.



Figure 8: Automated Emergency Braking Scenario. image source: <https://ackodrive.com/car-guide/autonomous-emergency-braking/>

The conceptualization of this use-case-level ontology is further driven by defined simulation goals as part of the capability-based simulation configuration (described in detail in the follow-

ing Chapter 4.6). These simulation goals outline required capabilities and quality attributes for simulation models and inform the formal specification of behavior, network layout, and simulation parameters. Two simulation goals were defined to demonstrate the effectiveness of the ontology:

Simulation Goal 1: "Find the effect of network fault injection on emergency braking." This simulation goal focuses on the capability of Fault Injection, which is crucial for testing robustness in safety-critical scenarios.

Fault injection is a deliberate technique used in system testing to introduce controlled faults — enabling the validation of failure response mechanisms. Its relevance in the automotive domain stems from the following:

1. **Safety-Critical System Validation:** Since emergency braking directly affects vehicle and passenger safety, fault injection helps ensure the system remains functional even when components fail.
2. **Robustness Testing:** By introducing controlled faults, developers can verify if the braking system maintains acceptable performance under adverse conditions.
3. **Failure Mode Analysis:** Fault injection helps identify and understand potential failure modes and their impacts on overall system behavior.

In the AEB context, we focus on Network Fault Injection, which simulates disruptions in ECU communication. This aligns with our domain-level focus on communication protocols and enables modeling of disruption scenarios using the CAN bus. So with this simulation goal we try to identify the ECUs and communication protocols that provide the fault injection capabilities and its related configuration. While investigating the effects of network fault injection on emergency braking, understanding which ECUs support fault injection is critical because different ECUs play different roles in the emergency braking system and some can be more critical than others. Therefore, knowing which ECUs in our simulation support fault injection will help us determine if there are ECUs that should support fault injection but currently don't or to focus on the most safety-critical ECUs first. So, the use-case-level ontology proposed here can serve as a structured knowledge base for querying and evaluating such fault-tolerant behaviors.

Simulation Goal 2: "Measure the message frame transmission of simulation models during emergency braking." This simulation goal examines the Message Frame Transmission capability, another key performance indicator in communication systems. This capability is crucial for analyzing the integrity and timing of data exchanges between ECUs during AEB activation. The ontology enables the tracing of participating ECUs, message types exchanged during braking, key performance attributes like latency, transmission rate, and potential frame loss. By semantically representing these data flows, the use-case-level ontology supports queries that establish pre-fault performance baselines and enable comparative fault scenario analysis.

Semantic Role of the Use-case-level Ontology:

The ontology developed for this use case demonstrates how domain-specific models can be instantiated to meet simulation requirements. Serving as an ABox, it instantiates simulation-specific resources (e.g., ECUs), capabilities (e.g., FaultInjection, MessageFrameTransmission), and configuration parameters (e.g., bit rate, payload size) to represent a concrete simulation scenario aligned with specific goals such as "Find the effect of network fault injection on emergency braking" and "Measure the message frame transmission of simulation models during emergency braking". This layer of ontology bridges abstract protocol-level definitions and concrete configuration requirements, enabling capability-driven queries, semantic validation and reasoning to determine which simulation models support critical functionalities like fault injection or message frame transmission for braking system analysis. OWL 2 DL logic was used to ensure computational decidability while interoperability with the mid-level ontology was maintained using consistent object and datatype properties (usesResource, hasCapability, hasBitRate, and so on). Figure 9 and 10 show some of the most important elements (concepts, individuals and relations) of the instances of the CAN simulation model from can use-case-level ontology, generated for the AEB scenario.

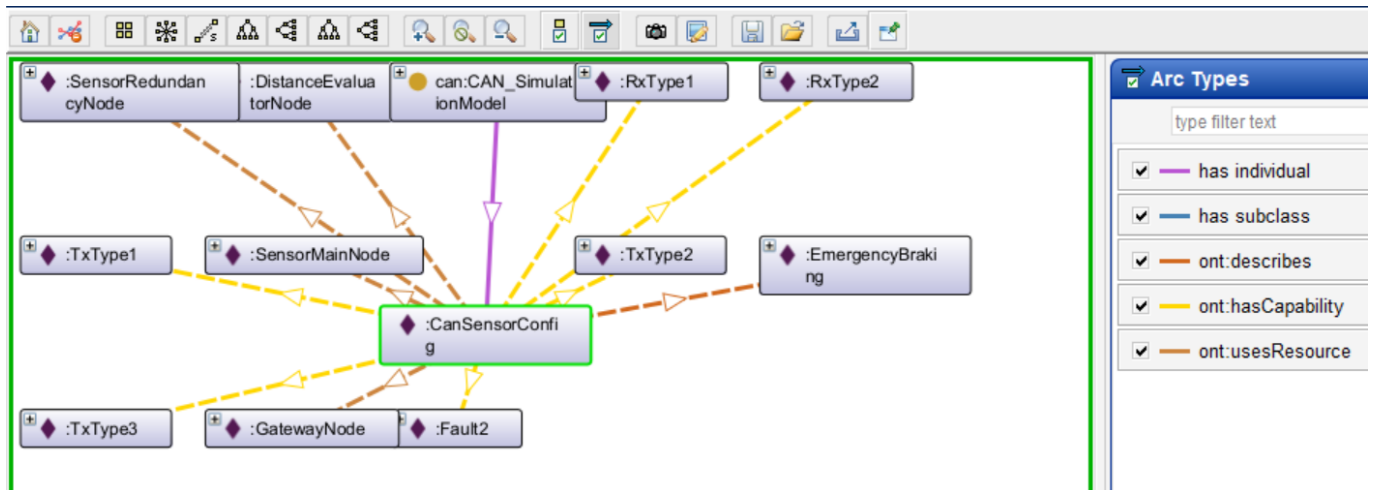


Figure 9: Instance of CAN simulation model for sensoric system in its use-case-level ontology.

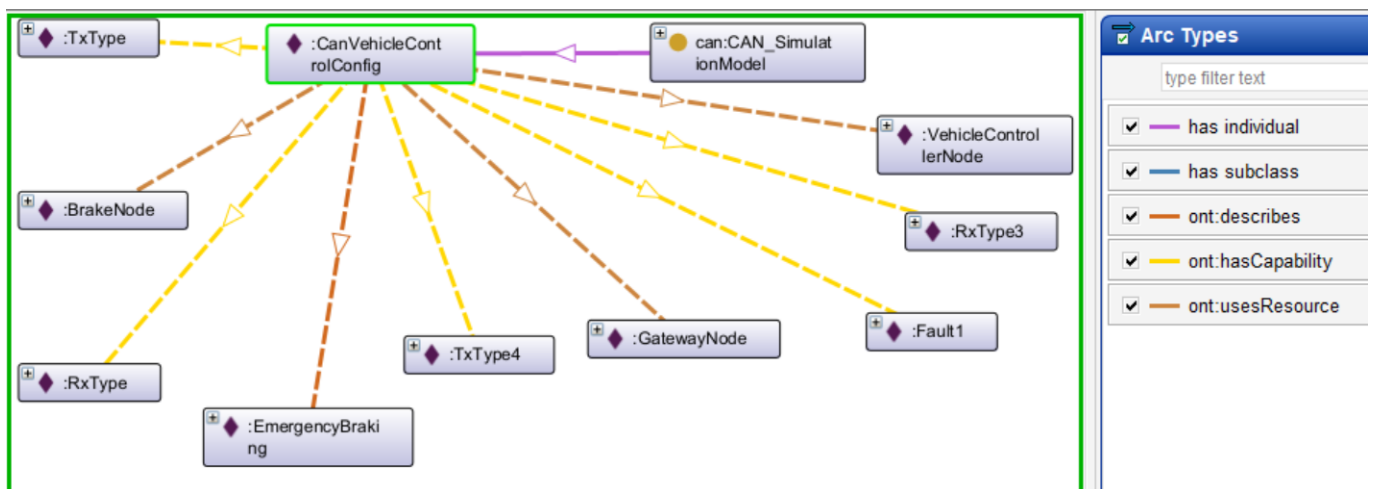


Figure 10: Instance of CAN simulation model for vehicle control system in its use-case-level ontology.

The targeted use case is "AEB", wherein various ECUs communicate over a CAN network to initiate safe braking when an obstacle is detected. This safety-critical application involves a set of simulation models, resources (ECUs), capabilities (message transmission and fault injection), and configuration parameters that together describe the entire scenario in a structured ontology.

CanVehicleControlConfig and CanSensorConfig are both instances of CAN_SimulationModel, extended from the mid-level ontology's SimulationModel class. These configurations contain specific bitrates, bit stuffing modes, logging preferences, and instances of linked resources (ECUs), capabilities, and required simulation settings for emergency braking. They encapsulate the behavioral configuration of the emergency braking logic and sensor interaction within the simulated environment. CanVehicleControlConfig models the configuration of the vehicle's control subsystem, responsible for processing distance evaluation and initiating braking. It aggregates relevant ECUs like VehicleControllerNode, GatewayNode, and BrakeNode along with associated TX/RX message types (TxType, RxType3, TxType4, RxType) and fault conditions (Fault1). CanSensorConfig represents the configuration of the sensor subsystem in the vehicle. It includes sensors for object detection and redundancy, such as SensorRedundancyNode, SensorMainNode, and DistanceEvaluatorNode, handling message types (TxType1, TxType2, TxType3, RxType1, RxType2) and related fault simulation (Fault2).

Multiple CAN_Nodes individuals represent various ECUs in the network such as SensorRedundancyNode, SensorMainNode, VehicleControllerNode, GatewayNode, Brake Node, and DistanceEvaluatorNode. Each node provides specific capabilities and configurations relevant to its role in the network, such as TxType1, RxType2, or Fault1. These resources define which ECUs participate in the communication network and the roles they take. SensorRedundancyNode and SensorMainNode simulate redundant and primary sensors that send environmental data. VehicleControllerNode handles message processing and braking decisions. GatewayNode facilitates message relay and bus coordination. DistanceEvaluatorNode processes object proximity and generates control signals. BrakeNode simulates the actuation of emergency braking. These instances concretely represent how message frames and diagnostics flow between critical vehicle subsystems.

Message frame transmission instances (e.g., TxType1, RxType2) encapsulate communication links, describing properties like address, payload size, portname and directionality (TX/RX). Their communicatesWith relations establish logical pathways between nodes(TX/RX pairs), modeling the direction and connectivity of messages between nodes. Fault injection instances (Fault1, Fault2) simulate anomalies such as message drops or value corruption to evaluate system resilience under failure conditions through the configuration of error model and processor parameters (SenderLinkFailure, DropFrame), timing attributes (StartTimeMillis, EndTimeMillis). This supports testing robustness under simulated fault conditions in the CAN bus.

The individual EmergencyBraking is instantiated from the class AutomatedEmergencyBrake, which itself is defined as a subclass of vdi3633:Simulation, formally linking the simulation

model configuration to the test scenario. It enables querying results by scenario and facilitates scenario-centric validation.

EmergencyBrakingSettings captures environmental and vehicle-specific parameters using the SimulationSettings class by defining dynamic properties such as starting speed, braking distance, and simulation duration, which directly influence the dynamics and evaluation of simulation goals.

So, this use-case-level ontology provides an executable and queryable instance-based model of a real-world scenario over CAN, effectively enabling simulation tools to reason about configuration, capabilities, and communication patterns critical for network simulation specific tasks.

4.6 Implementation of the Capability based engineering approach

This chapter demonstrates the application of a capability-based engineering workflow in alignment with our defined simulation goals. Based on [BDG⁺23], we decided to follow the steps depicted in Figure 11. As the first step of this cycle, we already defined two simulation goals which will drive the workflow of this semantic simulation configuration. The aim of the simulation goal is to define the required simulation capabilities and quality attributes for a specific scenario, which then enables the identification and selection of appropriate simulation models. The remaining sequence is as follows — Capability Check, Feasibility Check, and Simulation Execution. As part of this thesis, we implement the first two stages while deferring the execution phase for future work.

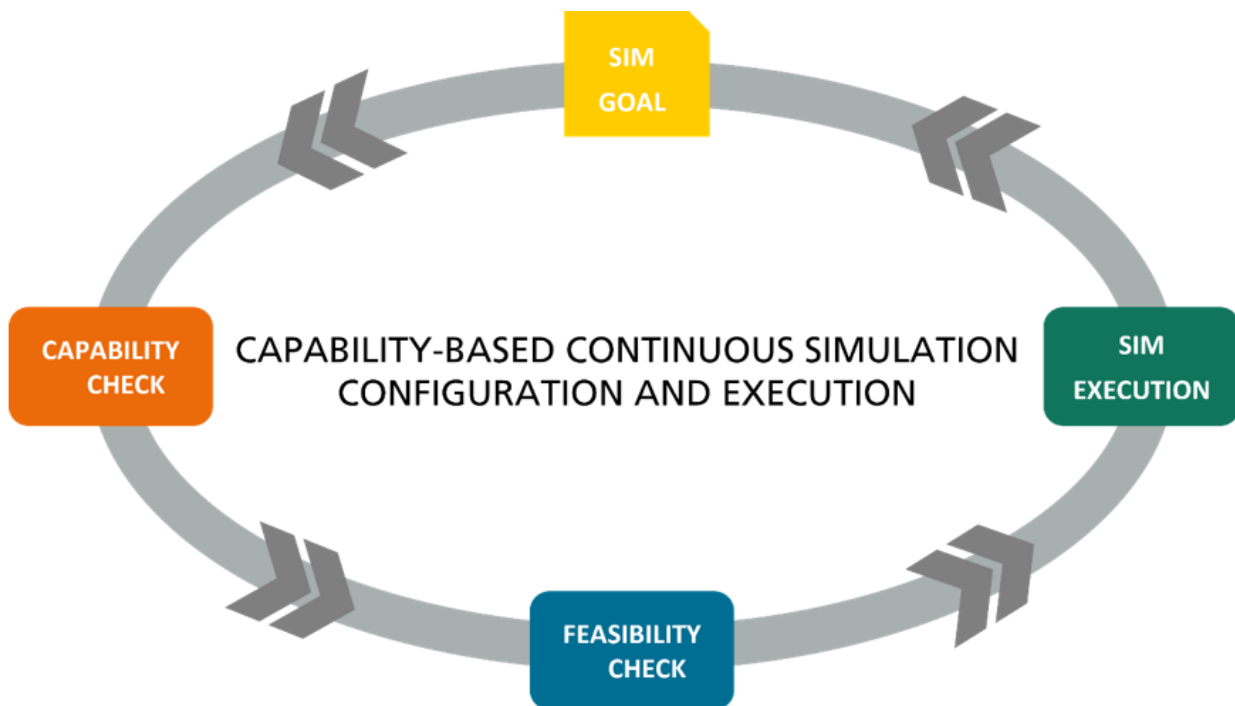


Figure 11: Process followed for the capability-based simulation configuration. Based on [BDG⁺23]

- **CAPABILITY CHECK:**

This step involves identifying simulation models that provide the necessary capabilities to fulfill the simulation goals and the result of the CAPABILITY CHECK can be none, one or more. As emphasized in Chapter 4.5, two simulation goals were defined:

1. **Simulation Goal1:** Find the effect of network fault injection on emergency braking.
2. **Simulation Goal2:** Measure the message frame transmission of simulation models during emergency braking.

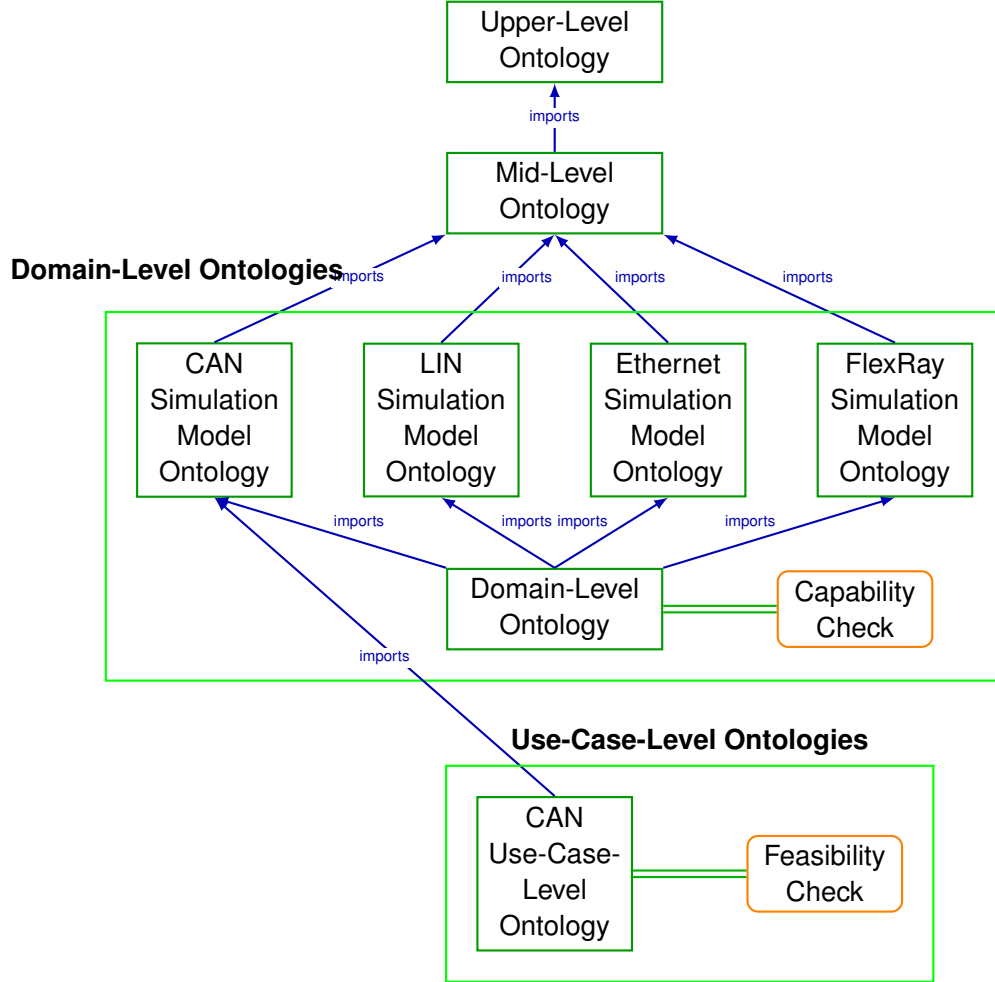


Figure 12: Conceptual Flow for Capability and Feasibility Evaluation

In the context of our thesis we performed capability check in the domain level as you see from Figure 12. Therefore, to enable semantic matching, we constructed a unified domain-level ontology by importing all four protocol-specific ontologies (CAN, LIN, Ethernet, and FlexRay). This setup ensured that simulation models across different protocols could be queried together. The next step was to conduct the Capability Check and in order to perform that we formulated CQs for each simulation goal. CQs play a crucial role in guiding the formulation of SPARQL queries [HKK⁺20]. They help ensure that ontology content aligns with simulation objectives, and allow systematic translation of stakeholder needs into machine-readable query logic. By grounding SPARQL queries in CQs, semantic traceability from requirements to simulation execution is achieved.

Therefore, we used this approach to frame our SPARQL queries to retrieve models offering the required capabilities.

1. Simulation Goal 1: CQs & their corresponding SPARQL Queries:

- CQ1: Which simulation models have capability **network fault injection** for **emergency braking** simulation scenario? And its associated SPARQL query is:

```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX ont: <http://example.org/ontology#>
PREFIX idta: <http://www.semanticweb.org/eklaspad/ontologies/2022/6/untitled-ontology-5#>
PREFIX can: <http://example.org/CANSimulationModelOntology#>

SELECT ?simulationModel ?c4FI ?scenario
WHERE {
    ?simulationModel
        rdfs:subClassOf idta:SimulationModel ;
        ont:hasCapability ?c4FI ;
        ont:describes ?scenario .
    FILTER (?c4FI = can:FaultInjection)
    FILTER (?scenario = can:AutomatedEmergencyBrake)
}
```

- CQ2: Which resources (e.g., CAN nodes) in these models are responsible for performing this **fault injection** capability for the **emergency braking** scenario? And its associated SPARQL query is:

```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX ont: <http://example.org/ontology#>
PREFIX idta: <http://www.semanticweb.org/eklaspad/ontologies/2022/6/untitled-ontology-5#>
PREFIX can: <http://example.org/CANSimulationModelOntology#>

SELECT ?simulationModel ?node ?c4FI ?scenario WHERE {

    ?simulationModel
        rdfs:subClassOf idta:SimulationModel ;
        ont:hasCapability ?c4FI ;
        ont:describes ?scenario ;
        ont:usesResource ?node .
    FILTER (?c4FI = can:FaultInjection)
    FILTER (?scenario = can:AutomatedEmergencyBrake)
}
```

2. Simulation Goal 2: CQs & their corresponding SPARQL Queries:

- CQ1: Which simulation models have capability **message frame transmission** for **emergency braking** simulation scenario? And its associated SPARQL query is:

```

PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX ont: <http://example.org/ontology#>
PREFIX idta: <http://www.semanticweb.org/eklaspad/ontologies/2022/6/untitled-ontology-5#>
PREFIX can: <http://example.org/CANSimulationModelOntology#>

SELECT ?simulationModel ?c4FT ?scenario WHERE {
  ?simulationModel
    rdfs:subClassOf idta:SimulationModel ;
    ont:hasCapability ?c4FT ;
    ont:describes ?scenario .
  FILTER (?c4FT = can:MessageFrameTransmission)
  FILTER (?scenario = can:AutomatedEmergencyBrake)
}

```

- CQ2: Which resources (e.g., CAN nodes) in these models are responsible for performing this **message frame transmission** capability for the **emergency braking** scenario? And its associated SPARQL query is:

```

PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX ont: <http://example.org/ontology#>
PREFIX idta: <http://www.semanticweb.org/eklaspad/ontologies/2022/6/untitled-ontology-5#>
PREFIX can: <http://example.org/CANSimulationModelOntology#>

SELECT ?simulationModel ?node ?c4FT ?scenario WHERE {
  ?simulationModel
    rdfs:subClassOf idta:SimulationModel ;
    ont:hasCapability ?c4FT ;
    ont:describes ?scenario ;
    ont:usesResource ?node .
  FILTER (?c4FT = can:MessageFrameTransmission)
  FILTER (?scenario = can:AutomatedEmergencyBrake)
}

```

Through the above queries as you see the property `ont:hasCapability` is used to retrieve simulation models offering a given functionality, as defined in the unified domain-level ontology. While the property `ont:describes` ensures that only simulation models relevant to the emergency braking scenario are considered, aligning directly with respective simulation goal. **Note:** While we mention the structure of CQs and SPARQL queries here, detailed query results and their interpretations will be presented in the Evaluation Chapter 5.

- **FEASIBILITY CHECK:**

Once candidate models were retrieved through the capability check, the feasibility check verified whether these models contain all necessary preconditions and configuration pa-

parameters to realize the simulation goals and the result of the FEASIBILITY CHECK can also be none, one or more parameterized simulation configurations that can meet the simulation goal. This step was performed within the use-case-level ontology (e.g., CAN AEB scenario), as depicted in Figure 12, where individual resources (e.g., VehicleControllerNode, SensorNode) and configurations (bitrate, slot values, etc.) are instantiated. Thereafter, in order to perform feasibility validation, the following kinds of CQs were posed. These CQs were again mapped into SPARQL queries executed against the use-case-level ontology. Only if all relevant parameters are found and valid can the configuration be deemed feasible for simulation.

1. Simulation Goal 1: CQs & their corresponding SPARQL Queries:

- CQ1: Which CAN nodes and their configuration parameters are involved in **fault injection** for **emergency braking** scenario for the Can Simulation model. And its associated SPARQL query is:

```
PREFIX can: <http://example.org/CANSimulationModelOntology#>
PREFIX use: <http://example.org/CANUseCase#>
PREFIX ont: <http://example.org/ontology#>

SELECT ?CANBus ?CANNode ?scenario ?c4FI ?faultModel ?
      faultProcessor ?startTime ?endTime
WHERE {
  ?CANBus a can:CAN_SimulationModel ;
          ont:usesResource ?CANNode ;
          ont:describes ?scenario .
  ?scenario a can:AutomatedEmergencyBrake .
  ?CANNode a can:CAN_Node ;
          ont:providesCapability ?c4FI .
  ?c4FI a can:FaultInjection ;
        can:hasErrorModel ?faultModel ;
        can:hasErrorProcessor ?faultProcessor ;
        can:hasStartTimeMillis ?startTime ;
        can:hasEndTimeMillis ?endTime .
}
```

2. Simulation Goal 2: CQs & their corresponding SPARQL Queries:

- CQ1: Which CAN nodes and their configuration parameters are involved in **message frame transmission** for **emergency braking** scenario for the Can Simulation model. And its associated SPARQL query is:

```
PREFIX can: <http://example.org/CANSimulationModelOntology#>
PREFIX use: <http://example.org/CANUseCase#>
PREFIX ont: <http://example.org/ontology#>

SELECT ?CANBus ?CANNode ?scenario ?c4FT ?type ?address ?
      payloadSize ?portName
WHERE {
  ?CANBus a can:CAN_SimulationModel ;
          ont:usesResource ?CANNode ;
```

```
    ont:describes ?scenario .
?scenario a can:AutomatedEmergencyBrake .
    ?CANNode a can:CAN_Node ;
        ont:providesCapability ?c4FT .
    ?c4FT a can:MessageFrameTransmission ;
        can:hasType ?type ;
        can:hasAddress ?address ;
        can:hasSimulatedPayloadSize ?payloadSize ;
        can:hasPortName ?portName .
}
```

While the final phase "simulation execution" is an integral part of the full capability-based configuration cycle, it lies beyond the scope of this thesis. Nonetheless, the successful execution of Capability and semi-automated Feasibility checks semantically sets the stage for automated or semi-automated simulation execution.

These implementation steps ensure semantic traceability between simulation goals, capability definitions, and model resources. This alignment between ontological modeling and executable queries lays the foundation for transparent and reusable co-simulation configurations.

5 Evaluation

This chapter evaluates the semantic configuration workflow described in Chapter 4.6, focusing on the ability of the ontology-based system to identify and validate protocol simulation models that fulfill the defined goals, as per the focus area of Fraunhofer IESE, particularly in the context of virtual testbeds for communication network simulation.¹ To assess the effectiveness and expressiveness of the ontology architecture in modeling complex simulation behaviors, two complementary simulation goals were defined. Simulation Goal 1 (Fault Injection in Emergency Braking) focuses on testing system resilience and fault tolerance by checking if network fault injection mechanisms are correctly modeled and configurable in the ontology. Simulation Goal 2 (Message Frame Transmission during Emergency Braking) focuses on analyzing data integrity and communication behavior, i.e., whether the ontology can trace and validate message flows between ECUs during the AEB scenario. These two goals cover both robustness (Goal 1) and communication correctness (Goal 2) — two critical quality perspectives in safety-critical automotive systems.

To validate the semantic configuration, we compare the automated configuration done through our ontologies with the expert-developed manual FERAL configuration for the AEB use case. Using ontology-based queries, we verify if all necessary capabilities and parameters could be derived from semantic descriptions and match the configuration requirements. The evaluation emphasizes two key stages: Capability Check and Feasibility Check, executed using SPARQL queries via Apache Jena Fuseki over domain- and use-case-level ontologies. Furthermore, the reasoning support provided by the OWLFBRuleReasoner is briefly assessed for its role in enabling inferencing during query execution.

• Simulation Goal 1: Fault Injection in Emergency Braking

1. Capability Check Evaluation:

- Query 1 Model Identification: The first query retrieved all simulation models offering the FaultInjection capability and describing the AutomatedEmergency-Brake scenario. This was done using the `ont:hasCapability` and `ont:describes` properties in the domain-level ontology. As a result, CAN Simulation Model was identified as supporting fault injection for emergency braking as you see from Figure 13. This result verifies that the domain-level ontology correctly associates simulation models with scenario-specific capabilities. The ability to match models to simulation intents through formal queries confirms the expressiveness and correctness of our capability modeling.

¹In practice, other organizations or companies would develop domain-specific ontologies tailored to their particular areas of expertise, incorporating relevant simulation models and domain-specific regulations appropriate to their respective fields.

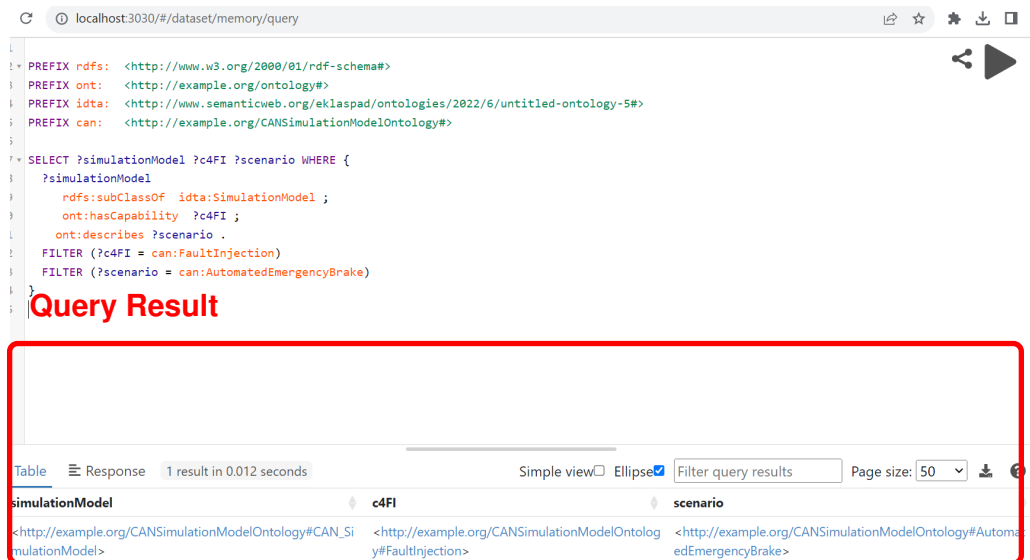


Figure 13: SPARQL query and its result of First Capability Check for Simulation Goal 1

- Query 2 Resource Identification: Based on the result of the first query (Query 1), we framed the second query with the purpose of finding out the existence of CAN Node for the identified CAN simulation model that can perform the FaultInjection capability, as you see from Figure 14. This mapping between capabilities and resources is critical to later find out instances of CAN nodes which are responsible for performing fault injection and then verify whether configuration parameters (e.g., error model, start time) exist for these nodes.



Figure 14: SPARQL query and its result of Second Capability Check for Simulation Goal 1

2. Feasibility Check Evaluation:

- Query Configuration Verification: The feasibility query inspected whether the CAN node instances involved in fault injection contain all required configuration attributes to simulate emergency braking scenario. Attributes such as Error Models, Error processors, and start and end times of Fault Injection were queried. All required parameters were found for the responsible nodes of re-

spective CAN Buses as you see from Figure 15. This outcome confirms that the selected simulation model is not only semantically matched at the capability level but is also feasible to execute based on available configurations.

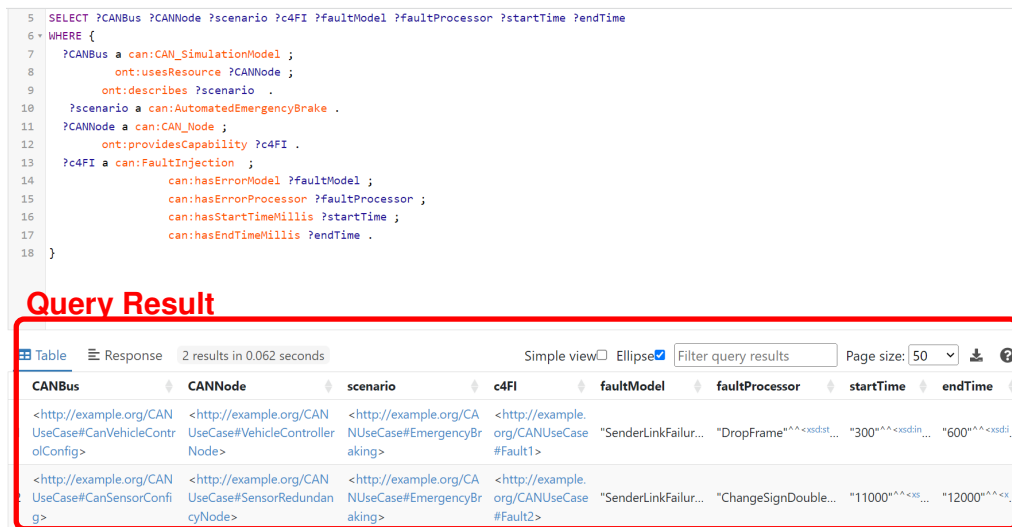


Figure 15: SPARQL query and its result of Feasibility Check for Simulation Goal 1

• Simulation Goal 2: Message Frame Transmission in Emergency Braking

1. Capability Check Evaluation:

- Query 1 Model Identification: The first query retrieved all simulation models offering the MessageFrameTransmission capability and describing the AutomatedEmergencyBrake scenario. This was done using the ont:hasCapability and ont:describes properties in the domain-level ontology. As a result we again received CAN Simulation Model which is supporting this functionality as you see from Figure 16.



Figure 16: SPARQL query and its result of first Capability Check for Simulation Goal 2

- Query 2 Resource Identification: The second query identified the existence

of CAN node which is responsible for the transmission behavior in the context of the selected scenario, as you see from Figure 17. This query result guides us to find out instances of CAN nodes that are responsible in conducting message transmission and further verify whether required configuration parameters exist for these nodes.

```

1 PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
2 PREFIX ont: <http://example.org/ontology#>
3 PREFIX idta: <http://www.semanticweb.org/eklaspad/ontologies/2022/6/untitled-ontology-5#>
4 PREFIX can: <http://example.org/CANSimulationModelOntology#>
5
6 SELECT ?simulationModel ?node ?c4FT ?scenario WHERE {
7
8   ?simulationModel
9     rdfs:subClassOf idta:SimulationModel ;
10    ont:hasCapability ?c4FT ;
11    ont:describes ?scenario ;
12    ont:usesResource ?node .
13   FILTER (?c4FT = can:MessageFrameTransmission)
14   FILTER (?scenario = can:AutomatedEmergencyBrake)
15 }

```

Query Result

simulationModel	node	c4FT	scenario
<http://example.org/CANSimulationModelOntology#CAN_SimulationModel>	<http://example.org/CANSimulationModelOntology#CAN_Node>	<http://example.org/CANSimulationModelOntology#MessageFrameTransmission>	<http://example.org/CANSimulationModelOntology#AutomatedEmergencyBrake>

Figure 17: SPARQL query and its result of Second Capability Check for Simulation Goal 2

2. Feasibility Check Evaluation:

- Query Configuration Verification: Afterwards, we executed the query for feasibility check, as you see in Figure 18 to find out if the responsible CAN node instances involved in message transmission contain all required configuration attributes such as Payload Size, Ports, and Address, to simulate emergency braking scenario. All relevant parameters were retrieved for the responsible nodes of respective CAN Buses. The outcome as noticed from Figure 19 confirms that the selected simulation model contains sufficient detail to support message transmission analysis for the emergency braking scenario.

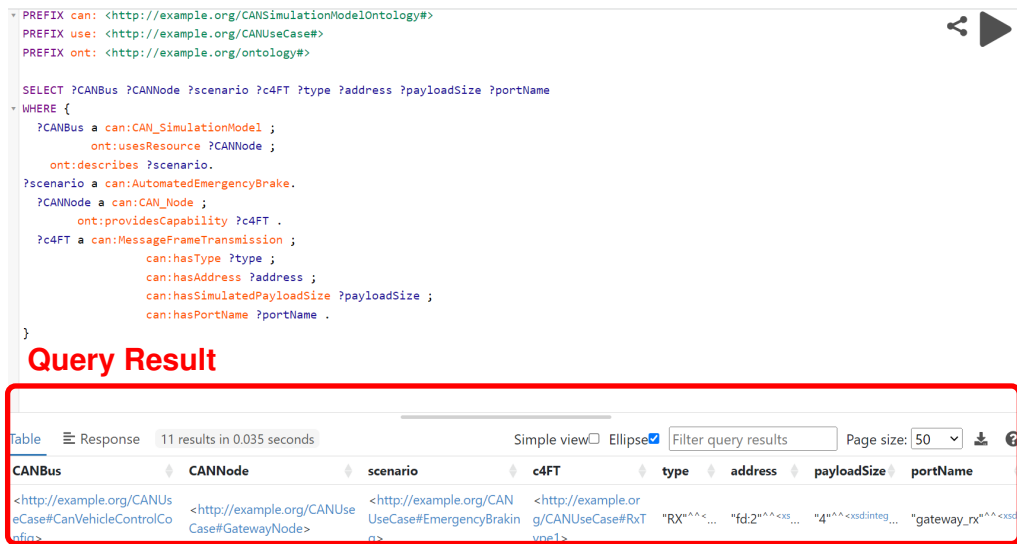


Figure 18: SPARQL query of Feasibility Check for Simulation Goal 2

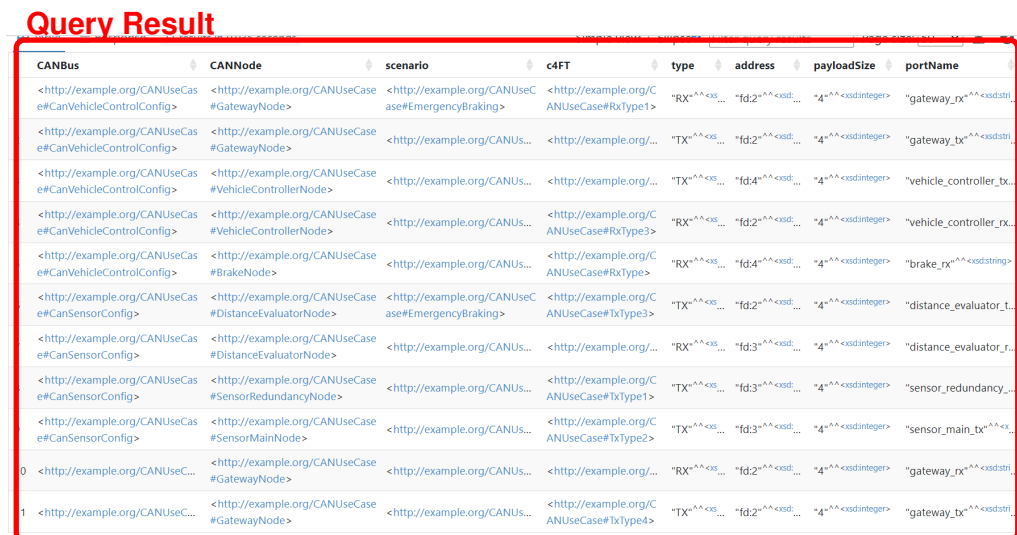


Figure 19: Result of the SPARQL query of Feasibility Check for Simulation Goal 2

We used Apache Jena Fuseki server as the SPARQL endpoint for executing queries. Fuseki enables efficient querying and management of ontologies during development and debugging. Its SPARQL 1.1 support made it a suitable tool for managing and querying our domain-level and use-case-level ontologies. To support inferencing during SPARQL query execution and ontology exploration, we utilized the OWLFBRuleReasoner provided by the Jena API. It supports forward-chaining over OWL and RDFS semantics and enabled basic rule-based inference, such as inferring subClass relationships and property transitivity. This was particularly useful in evaluating whether a simulation model implicitly satisfies a capability even when it is not explicitly asserted.

The following points have been observed through this evaluation:

- The ontology correctly identified models and associated capabilities that fulfill the simulation goals.

- The ontology allowed scenario-specific queries by decomposing capabilities and resources, showing high adaptability.
- Required simulation parameters were present and correctly mapped to resources, indicating feasibility.

To validate the effectiveness of the automated semantic configuration approach, we compared the ontology-generated configuration with the manual FERAL configuration done by simulation experts for the AEB use case. The manual FERAL configuration represents the ground truth, having been crafted through expert knowledge and extensive domain experience in automotive simulation modeling. The automated semantic approach followed in this work selected the same CAN Simulation Model that simulation experts manually identified as optimal for the emergency braking scenario. All relevant configuration parameters automatically derived through SPARQL queries matched the parameters manually specified in the expert-developed FERAL configuration. So, this validation demonstrates that the ontology-based semantic configuration can achieve expert-level configuration quality while significantly reducing manual effort and potential human errors in the simulation setup process.

6 Discussion

The current implementation and design of the multi-layered ontology architecture provides clear insights into how the RQs outlined in Chapter 2.1.1 are answered. This chapter systematically reflects on each RQ in light of the realized ontology stack and domain-specific simulation mappings.

- **RQ1: How can a structured ontology architecture be developed to support capability-based co-simulation configuration?**

This question is thoroughly addressed through the layered ontology architecture developed and implemented in this work. The thesis introduces a clear four-layer structure:

1. Upper-Level Ontology using BFO for grounding all concepts in a well-established top-level framework.
2. Mid-Level Ontology incorporating modular, reusable ontological patterns and concepts from CSS, VDI 3633, ISO25010 and IDTA SMT "Provision of Simulation Models", aligning simulation abstractions like SimulationModel, Capability, Resource, and so on.
3. Domain-Level Ontology, which specializes in formalizing simulation semantics for four key automotive protocols — CAN, LIN, Ethernet, and FlexRay — based on FERAL documentation and IDTA SMT "Provision of Simulation Models".
4. Use-Case-Level Ontology, which binds protocol-level representations with a use-case specific scenario, supporting targeted co-simulation setups.

The structured design is motivated by the need for semantic reusability, cross-domain interoperability, and extensibility. This layered framework enables clear separation of concerns, promotes abstraction over simulation semantics, and supports the integration of future protocols or tools without disrupting the entire architecture.

- **RQ2: How can simulation models be represented into the ontology architecture to guide capability-based co-simulation configuration?**

This question is addressed through the formal representation of simulation models using OWL-based ontologies, derived from IDTA SMT "Provision of Simulation Models" and populated with protocol-specific information.

Each domain-level ontology (for CAN, LIN, Ethernet, FlexRay) encapsulates the semantic structure of the corresponding simulation model, including capabilities offered (e.g., fault injection, frame transmission, scheduling control) and their classification using CSS Capability classes, resources needed by those simulation models and various critical simulation parameters such as bit rate, payload size, arbitration mechanisms, slot configurations, and so on. Figure 20 demonstrates this ontological modeling approach through the domain-level ontology of the CAN simulation model. The semantic

relationships illustrate some key properties of the CAN simulation model in relation to its capabilities, resources, and other simulation-relevant parameters.

All models are defined under the `idta:SimulationModel` superclass, ensuring semantic alignment across all protocol ontologies. Moreover, the use of various property axioms ensures that both the static configuration and dynamic behavior of simulation models are formally captured. This enables simulation models to be queried, compared, and selected based on their functionality — an essential requirement for automated or semi-automated simulation orchestration.

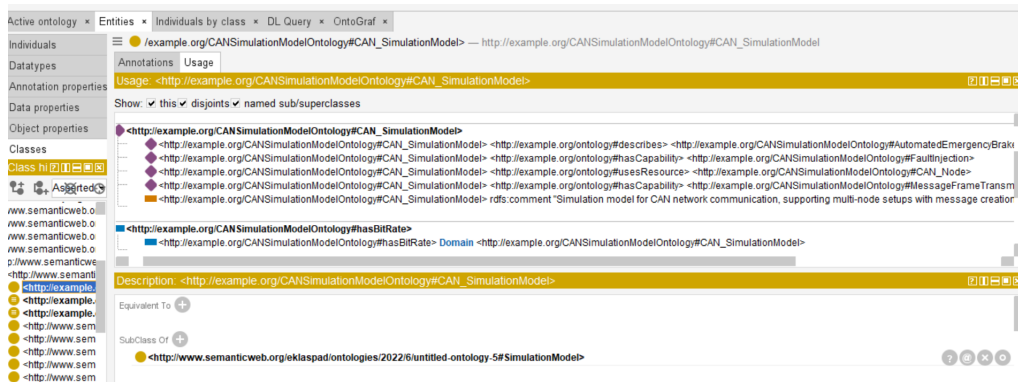


Figure 20: Domain-Level Ontology of CAN Simulation Model.

- **RQ3: How can capabilities and quality attributes be formally incorporated to enable automated configuration?**

The work integrates capability modeling using the `css:Capability` class and quality attribute modeling using the ISO 25010-based `QualityAttribute` class hierarchy. These concepts are further instantiated and linked with simulation models to express what a model can do and how well it does it.

Capabilities such as `FaultInjection`, `MessageFrameTransmission`, `MasterSlaveScheduling`, `StoreAndForwardCommunication`, and `TimeTriggeredDeterministicCommunication` are described in relevance to different communication protocol models. Quality attributes such as `Reliability`, `PerformanceEfficiency`, and `Portability` are modeled using ISO 25010 standards and got linked to simulation models using their object properties. These formalizations allow querying the ontology to retrieve necessary data for capabilities and quality attributes. Additionally, the use of such standardized capability and quality definitions supports semantic matchmaking between simulation models and configuration goals. This paves the way for dynamic co-simulation environments where users will be able to select models based on desired outcomes. Figure 21 illustrates this modeling approach using the `FaultInjection` capability as a concrete example, demonstrating how capability definitions are semantically linked to its associated CAN simulation model.

7 Conclusion and Future Work

This thesis presents the concept of a comprehensive and modular ontology architecture designed to support the semantic configuration of co-simulation environments which has been evaluated on the use case Automated Emergency Braking (AEB). By formalizing simulation model metadata with a focus on their capabilities and quality attributes, and protocol-specific knowledge using OWL ontologies and industry-standard design patterns, we addressed major challenges in simulation model reusability, interoperability, and configurability. The developed domain-level ontologies for Controller Area Network (CAN), Local Interconnect Network (LIN), Ethernet, and FlexRay demonstrate how protocol-specific semantics can be abstracted and reused within a unified framework. The implementation of a use-case-level ontology, instantiated for an AEB scenario, further validates the adaptability and extensibility of the architecture in modeling real-world, safety-critical applications. Through Competency Questions (CQs), SPARQL querying, and logical validation, the ontologies have been shown to be semantically coherent and practically applicable. The layered approach ensures modularity and provides a scalable basis for future extensions, for e.g. the inclusion of other communication protocols or other relevant domain specific aspects.

While the overall thesis aims to enable semantic automation of simulation model configuration, the current implementation focuses on a manual realization of the capability and feasibility checks. Although these checks were conceptually grounded in the defined simulation goals, the process of deriving capabilities and scenarios and executing SPARQL queries was conducted manually due to time and tooling constraints. Nonetheless, the architecture has been structured to support automation in future work. So, future work will involve developing a query-driven or rule-based mechanism to automatically derive capabilities, scenarios and other simulation configuration related aspects from the simulation goal itself and finally integrating with simulation execution platforms to complete the capability-based configuration cycle through to automated execution. Further research can also explore enhancing the representation and integration of quality attributes of simulation models in a more fine-grained and reusable manner. Ultimately, this work lays the groundwork for semantic automation of co-simulation configuration, enabling scalable and intelligent simulation engineering for complex systems.

Bibliography

References

- [ACD14] J. S. Artal, J. Caraballo, and R. Dufo. Can/lin-bus protocol. implementation of a low-cost serial communication network. In *2014 XI Tecnologias Aplicadas a la Ensenanza de la Electronica (Technologies Applied to Electronics Teaching) (TAE)*, pages 1–8, 2014.
- [BA09] Perakath Benjamin and Kumar Akella. Towards ontology-driven interoperability for simulation-based applications. In *Proceedings of the 2009 Winter Simulation Conference (WSC)*, pages 1375–1386, 2009.
- [BDG⁺23] Jürgen Bock, Christian Diedrich, Stephan Grimm, Simon Jungbluth, Tobias Klausmann, Tobias Kleinert, Aljosh Köcher, Kristof Meixner, Thomas Müller, Jörn Peschke, Miriam Schleipen, Siwara Schmitt, Boris Schnebel, Marco Simon, Melanie Stolze, Chris Urban, Luis Miguel Vieira da Silva, and Michael Winter. Capabilities, skills and services – css model extensions and engineering methodology. Technical report, Plattform Industrie 4.0, 2023. Available at: <https://www.plattform-i40.de/>.
- [Bet18] Fachbereich Fabrikplanung Und Betrieb. Verein deutscher ingenieure simulation von logistik-, materialfluss-und produktionssystemen begriffe simulation of systems in materials handling, logistics and production terms and definitions vdi 3633 vdi-gesellschaft produktion und logistik (gpl) vdi-handbuch produktionstechnik und fertigungsverfahren, band 1: Grundlagen und planung vdi-handbuch fabrikplanung und-betrieb, band 2: Modellierung und simulation vdi-handbuch technische logistik, band 8: Materialfluss ii (organisation/steuerung), 2018.
- [BMP07] Marko Bago, Sinisa Marijan, and Nedjeljko Peric. Modeling controller area network communication. In *2007 5th IEEE International Conference on Industrial Informatics*, volume 1, pages 485–490, 2007.
- [BPM] Perakath Benjamin, Mukul Patki, and Richard Mayer. Using ontologies for simulation modeling.
- [CB10] Devesh B. Chokshi and Purandar Bhaduri. Performance analysis of flexray-based systems using real-time calculus, revisited. In *Proceedings of the 2010 ACM Symposium on Applied Computing, SAC '10*, page 351–356, New York, NY, USA, 2010. Association for Computing Machinery.
- [DBB⁺22] Christian Diedrich, Alexander Belyaev, Rafi Blumenfeld, Patrick Zimmermann, et al. Information model for capabilities, skills & services. Technical report, Plattform Industrie 4.0, 11 2022. Christian Diedrich's Lab, CDL-SQI Module 4.

- [DFF⁺07] A. Depari, P. Ferrari, A. Flammini, D. Marioli, and A. Taroni. Evaluation of timing characteristics of industrial ethernet networks synchronized by means of ieee 1588. In *2007 IEEE Instrumentation Measurement Technology Conference IMTC 2007*, pages 1–5, 2007.
- [DuC13] Bob DuCharme. Rdf schema, owl, and inferencing. In *Learning SPARQL*, chapter 9. O'Reilly Media, Inc., 2nd edition, July 2013.
- [DYW06] Yiyou Dong, Jianhua Yang, and Zhaohui Wu. Odsg: An architecture of ontology-based distributed simulation on grid. In *First International Multi-Symposiums on Computer and Computational Sciences (IMSCCS'06)*, volume 1, pages 759–765, 2006.
- [ECN17] Fabien Escudié, Fabrice Caignet, and Nicolas Nolhier. Lin communication behaviours against esd events. In *2017 International Symposium on Electromagnetic Compatibility - EMC EUROPE*, pages 1–6, 2017.
- [FFRG08] P. Ferrari, A. Flammini, S. Rinaldi, and G. Gaderer. Evaluation of clock synchronization accuracy of coexistent real-time ethernet protocols. In *2008 IEEE International Symposium on Precision Clock Synchronization for Measurement, Control and Communication*, pages 87–91, 2008.
- [Fle10] FlexRay Consortium. Flexray communications system protocol specification. Protocol Specification Version 3.0.1, 2010.
- [GCMS12] Katarina Grolinger, Miriam A M Capretz, José R Marti, and Krishan D Srivastava. Ontology-based representation of simulation models, 2012.
- [GK11] Monal Giradkar and Milind Khanapurkar. Design and implementation of adaptive front light system of vehicle using fpga based lin controller. In *2011 Fourth International Conference on Emerging Trends in Engineering Technology*, pages 258–261, 2011.
- [GP09] Aldo Gangemi and Valentina Presutti. *Ontology Design Patterns*, pages 221–243. Springer Berlin Heidelberg, Berlin, Heidelberg, 2009.
- [Gru93] Thomas R. Gruber. A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2):199–220, 1993.
- [Ham17] Karl Hammar. Content ontology design patterns: Qualities, methods, and tools. 2017.
- [HBH⁺] S G Henderson, B Biller, M.-H Hsieh, J Shortle, J D Tew, R R Barton, Perakath Benjamin, Kumar Akella, and Ajay Verma. Proceedings of the 2007 winter simulation conference using ontologies for simulation integration.
- [HKK⁺20] Constantin Hildebrandt, Aljosha Kocher, Christof Kustner, Carlos Manuel Lopez-Enriquez, Andreas W. Muller, Birte Caesar, Claas Steffen Gundlach, and

- Alexander Fay. Ontology building for cyber-physical systems: Application in the manufacturing domain. *IEEE Transactions on Automation Science and Engineering*, 17:1266–1282, 7 2020.
- [HTB⁺18] Constantin Hildebrandt, Sebastian Törsleff, Torsten Bandyszak, Birte Caesar, Alexander Ludewig, and A. Fay. Ontology engineering for collaborative embedded systems - requirements and initial approach. In *Modellierung*, 2018.
- [IEE22] IEEE Computer Society. Ieee standard for ethernet. IEEE Standard IEEE Std 802.3™-2022, Institute of Electrical and Electronics Engineers, 2022. Revision of IEEE Std 802.3-2018. Accessed: March 30, 2025.
- [Ind22] Industrial Digital Twin Association. Idta-02005-1-0 submodel template provision of simulation models. Specification IDTA-02005-1-0, Industrial Digital Twin Association, Lyoner Strasse 18, 60528 Frankfurt am Main, Germany, 12 2022. Release of the official Submodel template published by IDTA.
- [Int23] International Organization for Standardization. Systems and software engineering — systems and software quality requirements and evaluation (square) — product quality model. International Standard ISO/IEC 25010, ISO/IEC, 11 2023. Reference number: ISO/IEC 25010:2023(E).
- [Iso] Iso. Systems and software engineering-systems and software quality requirements and evaluation (square)-product quality model.
- [ISO21] Information technology — top-level ontologies (tlo) — part 2: Basic formal ontology (bfo). Standard, International Organization for Standardization, Geneva, Switzerland, 2 2021.
- [ISO24] ISO. Road vehicles – controller area network (can) – part 1: Data link layer and physical coding sublayer. International Standard ISO 11898-1:2024(en), International Organization for Standardization, Geneva, Switzerland, May 2024. Accessed: 2024-12-09.
- [IZ08] Mohd Nazri Ismail and Abdullah Mohd Zin. Development of simulation model: An ethernet case study in heterogeneous environment. In *2008 International Symposium on Information Technology*, volume 3, pages 1–8, 2008.
- [JSHL18] Eeva Järvenpää, Niko Siltala, Otto Hylli, and Minna Lanz. Product model ontology and its use in capability-based matchmaking. *Procedia CIRP*, 72:1094–1099, 2018. 51st CIRP Conference on Manufacturing Systems.
- [JSHL19] Eeva Järvenpää, Niko Siltala, Otto Hylli, and Minna Lanz. The development of an ontology for describing the capabilities of manufacturing resources. *Journal of Intelligent Manufacturing*, 30:959–978, 2 2019.

- [KASW10] Michael Karner, Eric Armengaud, Christian Steger, and Reinhold Weiss. Holistic simulation of flexray networks by using run-time model switching. In *2010 Design, Automation Test in Europe Conference Exhibition (DATE 2010)*, pages 544–549, 2010.
- [KBH⁺23] Aljosha Köcher, Alexander Belyaev, Jesko Hermann, Christian Diedrich, et al. A reference model for common understanding of capabilities and skills in manufacturing. *at - Automatisierungstechnik*, 71(2):94–104, February 2023.
- [KCA⁺10] Martin Krammer, Federico Clazzer, Eric Armengaud, Michael Karner, Christian Steger, and Reinhold Weiss. Exploration of the flexray signal integrity using a combined prototyping and simulation approach. In *13th IEEE Symposium on Design and Diagnostics of Electronic Circuits and Systems*, pages 111–116, 2010.
- [KHC⁺20] Aljosha Köcher, Constantin Hildebrandt, Birte Caesar, Jupiter Bakakeu, Jörn Peschke, André Scholz, and Alexander Fay. Automating the development of machine skills and their semantic description. In *2020 25th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, volume 1, pages 1013–1018, 2020.
- [KHVdSF20] Aljosha Köcher, Constantin Hildebrandt, Luis Miguel Vieira da Silva, and Alexander Fay. A formal capability and skill model for use in plug and produce scenarios. In *2020 25th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, volume 1, pages 1663–1670, 2020.
- [KKAM08] Woo Sik Kim, Hyun Ah Kim, Jin-Ho Ahn, and Byungin Moon. System-level development and verification of the flexray communication controller model based on systemc. In *2008 Second International Conference on Future Generation Communication and Networking*, volume 2, pages 124–127, 2008.
- [KKK⁺10] Michael Karner, Martin Krammer, Stefan Krug, Eric Armengaud, Christian Steger, and Reinhold Weiss. Heterogeneous co-simulation platform for the efficient analysis of flexray-based automotive distributed embedded systems. In *2010 IEEE International Workshop on Factory Communication Systems Proceedings*, pages 231–240, 2010.
- [LFS⁺22] Franz Georg Listl, Jan Fischer, Annelie Sohr, Stephan Grimm, and Michael Weyrich. Ontological architecture for knowledge graphs in manufacturing and simulation. In *IEEE International Conference on Emerging Technologies and Factory Automation, ETFA*, volume 2022-September. Institute of Electrical and Electronics Engineers Inc., 2022.
- [LFW23] Franz Georg Listl, Jan Fischer, and Michael Weyrich. An architecture for knowledge graph based simulation support. In *2023 IEEE 28th International Confer-*

- ence on Emerging Technologies and Factory Automation (ETFA), pages 1–8, 2023.
- [LIN10] LIN Consortium. Lin specification package. Specification Revision 2.2A, December 2010.
- [LWT20] Jinzhi Lu, Guoxin Wang, and Martin Törngren. Design ontology in a case study for cosimulation in a model-based systems engineering tool-chain. *IEEE Systems Journal*, 14(1):1297–1308, 2020.
- [MB] John A Miller and Gregory Baramidze. 2005: Simulation and the semantic web.
- [MBSF04] J.A. Miller, G.T. Baramidze, A.P. Sheth, and P.A. Fishwick. Investigating ontologies for simulation modeling. In *37th Annual Simulation Symposium, 2004. Proceedings.*, pages 55–63, 2004.
- [Mou20] Dimitris Mourtzis. Simulation in the design and operation of manufacturing systems: state of the art and new trends. *International Journal of Production Research*, 58(7):1927–1949, 2020.
- [MV10] Candice Muller and Maurizio Valle. System verification of flexray communication networks through behavioral simulations. In *2010 IEEE International Behavioral Modeling and Simulation Workshop*, pages 1–6, 2010.
- [NSM5] Petr Novák, Estefanía Serral, Richard Mordinyi, and Radek Šindelář. Integrating heterogeneous engineering knowledge and tools for efficient industrial simulation model support. *Advanced Engineering Informatics*, 29(3):575–590, 2015.
- [N1] Petr Novák and Radek Šindelář. Applications of ontologies for assembling simulation models of industrial systems. In *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, volume 7046 LNCS, pages 148–157, 2011.
- [PFMM08] Kai Petersen, Robert Feldt, Shahid Mujtaba, and Michael Mattsson. Systematic mapping studies in software engineering. In *Proceedings of the 12th International Conference on Evaluation and Assessment in Software Engineering, EASE’08*, page 68–77, Swindon, GBR, 2008. BCS Learning & Development Ltd.
- [PGL⁺19] Alexander Perzylo, Julian Grothoff, Levi Lucio, Michael Weser, Somayeh Malakuti, Pierre Venet, Vincent Aravantinos, and Torben Deppe. Capability-based semantic interoperability of manufacturing resources: A basys 4.0 perspective. *IFAC-PapersOnLine*, 52(13):1590–1596, 2019. 9th IFAC Conference on Manufacturing Modelling, Management and Control MIM 2019.
- [PHK10] Roberto Poli, M.J. Healy, and Achilles Kameas. *Theory and Applications of Ontology: Computer Applications*. Springer, 2010.

- [RJF23] Jonathan Reif, Tom Jeleniewski, and Alexander Fay. An approach to automating the generation of process simulation sequences. In *IEEE International Conference on Emerging Technologies and Factory Automation, ETFA*, volume 2023-September. Institute of Electrical and Electronics Engineers Inc., 2023.
- [RJMS⁺20] Nour Ramzy, Christian James Martens, Shreya Singh, Thomas Ponsignon, and Hans Ehm. First steps towards bridging simulation and ontology to ease the model creation on the example of semiconductor industry. In *2020 Winter Simulation Conference (WSC)*, pages 1789–1800, 2020.
- [RP18] Thiago Regal and Carlos Eduardo Pereira. Ontology for conceptual modelling of intelligent maintenance systems and spare parts supply chain integration. volume 51, pages 1511–1516. Elsevier B.V., 1 2018.
- [SF11] Mari Carmen Suárez-Figueroa. Neon methodology for building ontology networks: specification, scheduling and reuse. In *DISKI*, 2011.
- [SK22] Leonard Stepien and Frank Köster. Flexible model exchange in modelling smart mobility by using domain ontologies. In *2022 IEEE International Systems Conference (SysCon)*, pages 1–7, 2022.
- [SLDIMG16] Juan L. Sarli, Horacio P. Leone, and Ma. De los Milagros Gutiérrez. Ontology-based semantic model of supply chains for modeling and simulation in distributed environment. In *2016 Winter Simulation Conference (WSC)*, pages 1182–1193, 2016.
- [SP14] Thiago Regal Da Silva and Carlos Eduardo Pereira. Building an ontology for intelligent maintenance systems and spare parts supply chain integration. In *IFAC Proceedings Volumes (IFAC-PapersOnline)*, volume 19, pages 7843–7848. IFAC Secretariat, 2014.
- [SSL19] Jan Sören Schwarz, Cornelius Steinbrink, and Sebastian Lehnhoff. Towards an assisted simulation planning for co-simulation of cyber-physical energy systems. In *2019 7th Workshop on Modeling and Simulation of Cyber-Physical Energy Systems (MSCPES)*, pages 1–6, 2019.
- [SZ08] Lucia Seno and Claudio Zunino. A simulation approach to a real-time ethernet protocol: Ethercat. In *2008 IEEE International Conference on Emerging Technologies and Factory Automation*, pages 440–443, 2008.
- [THF] Sebastian Törsleff, Constantin Hildebrandt, and Alexander Fay. Development of ontologies for reasoning and communication in multi-agent systems.
- [TS08] Yong Meng Teo and Claudia Szabo. Codes: An integrated approach to composable modeling and simulation. In *41st Annual Simulation Symposium (anss-41 2008)*, pages 103–110, 2008.

- [TSP⁺20] Brígida Teixeira, Gabriel Santos, Tiago Pinto, Zita Vale, and Juan M. Corchado. Application ontology for multi-agent and web-services' co-simulation in power and energy systems. *IEEE Access*, 8:81129–81141, 2020.
- [Usc99] Michael Uschold. A framework for understanding and classifying ontology applications. 1999.
- [VDI14] VDI. Simulation of systems in materials handling, logistics and production fundamentals. Standard VDI 3633:1, Verein Deutscher Ingenieure, Düsseldorf, Germany, December 2014. VDI-Richtlinie.
- [WBS⁺20] Michael Weser, Jürgen Bock, Siwara Schmitt, Alexander Perzylo, and Kathrin Evers. An ontology-based metamodel for capability descriptions. In *2020 25th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, volume 1, pages 1679–1686, 2020.
- [WFX⁺18] You Wu, Lijun Fu, Yinan Xu, Fan Ma, and Yao Lu. Controller area network modeling and its application in cyber-physical power system co-simulation. In *2018 37th Chinese Control Conference (CCC)*, pages 6178–6183, 2018.
- [WOLL17] Shiyong Wang, Jian Ouyang, Di Li, and Chengliang Liu. An integrated industrial ethernet solution for the implementation of smart factory. *IEEE Access*, 5:25455–25462, 2017.
- [YLZ⁺23] Xiaolang Yang, Xuemei Liu, Heng Zhang, Ling Fu, and Yanbin Yu. An ontology-based shop-floor digital twin configuration approach. In *Procedia CIRP*, volume 120, pages 326–331. Elsevier B.V., 2023.

A Appendix

A.1 Mid-level Ontology

```
<?xml version="1.0"?>
<rdf:RDF xmlns="http://example.org/ontology#"
  xml:base="http://example.org/ontology"
  xmlns:dc="http://purl.org/dc/elements/1.1/"
  xmlns:bfo="http://purl.obolibrary.org/obo/bfo#"
  xmlns:css="https://raw.githubusercontent.com/CaSkade-Automation/CSS/
    main/CSS-Ontology#"
  xmlns:obo="http://purl.obolibrary.org/obo/"
  xmlns:owl="http://www.w3.org/2002/07/owl#"
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:xml="http://www.w3.org/XML/1998/namespace"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema#"
  xmlns:foaf="http://xmlns.com/foaf/0.1/"
  xmlns:ida="http://www.semanticweb.org/eklaspad/ontologies/2022/6/
    untitled-ontology-5#"
  xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
  xmlns:terms="http://purl.org/dc/terms/"
  xmlns:vd13633="https://raw.githubusercontent.com/JonathanReif/SiS/main/
    Resources/VDI3633#"
  xmlns:iso25010="http://www.example.org/iso25010#">
  <owl:Ontology rdf:about="http://example.org/ontology">
    <owl:imports rdf:resource="http://purl.obolibrary.org/obo/bfo.owl"/>
    <owl:imports rdf:resource="http://www.semanticweb.org/eklaspad/
      ontologies/2022/6/untitled-ontology-5"/>
    <owl:imports rdf:resource="https://raw.githubusercontent.com/
      CaSkade-Automation/CSS/main/CSS-Ontology.owl"/>
    <owl:imports rdf:resource="https://raw.githubusercontent.com/
      JonathanReif/SiS/main/Resources/VDI3633.owl"/>
  </owl:Ontology>

  <!-- Object Properties -->

  <!-- http://example.org/ontology#hasQualityAttribute -->

  <owl:ObjectProperty rdf:about="http://example.org/ontology#
    hasQualityAttribute">
    <rdfs:domain rdf:resource="http://www.semanticweb.org/eklaspad/
      ontologies/2022/6/untitled-ontology-5#SimulationModel"/>
    <rdfs:range rdf:resource="http://www.example.org/iso25010#
      QualityAttribute"/>
    <rdfs:comment xml:lang="en">Relates a simulation model to its
      quality attributes, such as reliability or performance.</rdfs:
      comment>
  </owl:ObjectProperty>
```

```

<!-- http://example.org/ontology#describes -->

<owl:ObjectProperty rdf:about="http://example.org/ontology#describes">
<rdfs:domain rdf:resource="http://www.semanticweb.org/eklaspad/
  ontologies/2022/6/untitled-ontology-5#SimulationModel"/>
  <rdfs:range rdf:resource="https://raw.githubusercontent.com/
    JonathanReif/SiS/main/Resources/VDI3633#Simulation"/>
  <rdfs:comment xml:lang="en">A simulation model provides metadata
    about a specific simulation.</rdfs:comment>
</owl:ObjectProperty>

<!-- http://example.org/ontology#usesResource -->

<owl:ObjectProperty rdf:about="http://example.org/ontology#usesResource
">
  <rdfs:domain rdf:resource="http://www.semanticweb.org/eklaspad/
    ontologies/2022/6/untitled-ontology-5#SimulationModel"/>
  <rdfs:range rdf:resource="http://www.w3id.org/hsu-aut/css#Resource"
    />
  <rdfs:comment xml:lang="en">A simulation model requires a resource
    for execution.</rdfs:comment>

</owl:ObjectProperty>

<!-- http://example.org/ontology#hasCapability -->

<owl:ObjectProperty rdf:about="http://example.org/ontology#
  hasCapability">
  <rdfs:domain rdf:resource="http://www.semanticweb.org/eklaspad/
    ontologies/2022/6/untitled-ontology-5#SimulationModel"/>
  <rdfs:range rdf:resource="http://www.w3id.org/hsu-aut/css#
    Capability"/>
  <rdfs:comment xml:lang="en">A simulation model possesses specific
    capabilities.</rdfs:comment>

</owl:ObjectProperty>

<!-- http://example.org/ontology#executesProcess -->

<owl:ObjectProperty rdf:about="http://example.org/ontology#
  executesProcess">
  <rdfs:domain rdf:resource="http://www.semanticweb.org/eklaspad/
    ontologies/2022/6/untitled-ontology-5#SimulationModel"/>
  <rdfs:range rdf:resource="http://www.w3id.org/hsu-aut/css#Process"/
    >

```

```

        <rdfs:comment xml:lang="en">A simulation model executes a process.<
            /rdfs:comment>

</owl:ObjectProperty>

<!-- http://www.w3id.org/hsu-aut/css#providesCapability -->

<rdf:Description rdf:about="http://www.w3id.org/hsu-aut/css#
    providesCapability">
    <rdfs:domain rdf:resource="http://www.w3id.org/hsu-aut/css#Resource
        "/>
    <rdfs:range rdf:resource="http://www.w3id.org/hsu-aut/css#
        Capability"/>
</rdf:Description>

<!-- http://www.w3id.org/hsu-aut/css#requiresCapability -->

<rdf:Description rdf:about="http://www.w3id.org/hsu-aut/css#
    requiresCapability">
    <rdfs:domain rdf:resource="http://www.w3id.org/hsu-aut/css#Process"
        />
    <rdfs:range rdf:resource="http://www.w3id.org/hsu-aut/css#
        Capability"/>
</rdf:Description>

<!-- Classes -->

<!-- http://purl.obolibrary.org/obo/bfo#Object -->

<rdf:Description rdf:about="http://purl.obolibrary.org/obo/bfo#Object">
<rdfs:comment xml:lang="en">An object is a material entity which
    manifests causal unity & is of a type instances of which are
    maximal relative to the sort of causal unity manifested such as An
    organism; a fish tank; a planet; a laptop; a valve; a block of marble
    ; an ice cube [BF0]</rdfs:comment>
</rdf:Description>

<!-- http://purl.obolibrary.org/obo/bfo#Process -->

<rdf:Description rdf:about="http://purl.obolibrary.org/obo/bfo#Process"
    >
<rdfs:comment xml:lang="en">p is a process means p is an occurrent that
    has some temporal proper part and for some time t, p has some
    material entity as participant such as An act of selling; the life of
    an organism; a process of sleeping; a process of cell-division; a
    beating of the heart; a process of meiosis; the taxiing of an
    aircraft; the programming of a computer [BF0]</rdfs:comment>
</rdf:Description>

```

```

<!-- http://purl.obolibrary.org/obo/bfo#Function -->

<rdf:Description rdf:about="http://purl.obolibrary.org/obo/bfo#Function
">
<rdfs:comment xml:lang="en"> A function is a disposition that exists in
virtue of its bearer's physical make-up & this physical make
-up is something the bearer possesses because it came into being
either through evolution (in the case of natural biological entities)
or through intentional design (in the case of artefacts) in order to
realize processes of a certain sort such as The function of a hammer
to drive in nails; the function of a heart pacemaker to regulate the
beating of a heart through electricity [BFO]</rdfs:comment>
</rdf:Description>

<!-- http://purl.obolibrary.org/obo/bfo#Quality -->

<rdf:Description rdf:about="http://purl.obolibrary.org/obo/bfo#Quality"
>
<rdfs:comment xml:lang="en">A quality is a specifically dependent
continuant that, in contrast to roles and dispositions, does not
require any further process in order to be realized such as The
colour of a tomato; the ambient temperature of this portion of air;
the length of the circumference of your waist; the shape of your nose
; the shape of your nostril; the mass of this piece of gold. [BFO]
</rdfs:comment>
</rdf:Description>

<!-- http://www.w3id.org/hsu-aut/css#Capability -->

<rdf:Description rdf:about="http://www.w3id.org/hsu-aut/css#Capability">
<rdfs:subClassOf rdf:resource="http://purl.obolibrary.org/obo/bfo#
Function"/>
</rdf:Description>

<!-- http://www.w3id.org/hsu-aut/css#Resource -->

<rdf:Description rdf:about="http://www.w3id.org/hsu-aut/css#Resource">
<rdfs:subClassOf rdf:resource="http://purl.obolibrary.org/obo/bfo#
Object"/>
</rdf:Description>

<!-- http://www.w3id.org/hsu-aut/css#Process -->

<rdf:Description rdf:about="http://www.w3id.org/hsu-aut/css#Process">
<rdfs:subClassOf rdf:resource="http://purl.obolibrary.org/obo/bfo#
Process"/>
</rdf:Description>

```

```

<!-- http://www.example.org/iso25010#QualityAttribute -->

<owl:Class rdf:about="http://www.example.org/iso25010#QualityAttribute"
>
  <rdfs:subClassOf rdf:resource="http://purl.obolibrary.org/obo/bfo#
    Quality"/>
<rdfs:comment xml:lang="en">Quality characteristics and
  subcharacteristics of the product quality model [ISO25010]</rdfs:
  comment>
</owl:Class>

<!-- http://www.semanticweb.org/eklaspad/ontologies/2022/6/untitled-
  ontology-5#SimulationModel -->

<rdf:Description rdf:about="http://www.semanticweb.org/eklaspad/
  ontologies/2022/6/untitled-ontology-5#SimulationModel">
  <rdfs:subClassOf rdf:resource="http://purl.obolibrary.org/obo/bfo#
    Object"/>
</rdf:Description>

<!-- https://raw.githubusercontent.com/JonathanReif/SiS/main/Resources/
  VDI3633#Simulation -->

<rdf:Description rdf:about="http://www.semanticweb.org/reif/ontologies/
  VDI3633#Simulation">
  <rdfs:subClassOf rdf:resource="http://purl.obolibrary.org/obo/bfo#
    Process"/>
<rdfs:comment xml:lang="en">Simulation as the representation of a
  system with its dynamic processes in an experimentable model with
  the aim of reaching findings which are transferable to reality [
  VDI3633]</rdfs:comment>
</rdf:Description>

<!-- Subclasses of QualityAttribute -->

<owl:Class rdf:about="http://www.example.org/iso25010#
  FunctionalSuitability">
  <rdfs:subClassOf rdf:resource="http://www.example.org/iso25010#
    QualityAttribute"/>
  <rdfs:label xml:lang="en">Functional Suitability</rdfs:label>
<rdfs:comment xml:lang="en">capability of a product to provide
  functions that meet stated and implied needs of intended users when
  it is used under specified conditions.
Note 1 to entry: Functional suitability is concerned with whether the
  functions meet not only stated and implied needs, but also the
  functional specification [ISO25010]
</rdfs:comment>

```

```

</owl:Class>

<owl:Class rdf:about="http://www.example.org/iso25010#Reliability">
  <rdfs:subClassOf rdf:resource="http://www.example.org/iso25010#
    QualityAttribute"/>
  <rdfs:label xml:lang="en">Reliability</rdfs:label>
  <rdfs:comment xml:lang="en">capability of a product to perform
    specified functions under specified conditions for a specified
    period of time without interruptions and failures
    Note 1 to entry: Wear does not occur in software. Limitations in
    reliability are due to results from faults in
    requirements, design and implementation, or from contextual changes.
    Note 2 to entry: Dependability is often used as a synonym for
    reliability. However, dependability has a larger
    scope in that it includes security (3.6), performance efficiency (3.2),
    and continuing support and others in addition
    to the subcharacteristics of reliability. [ISO25010]
  </rdfs:comment>
</owl:Class>

<owl:Class rdf:about="http://www.example.org/iso25010#Usability">
  <rdfs:subClassOf rdf:resource="http://www.example.org/iso25010#
    QualityAttribute"/>
  <rdfs:label xml:lang="en">Interaction Capability</rdfs:label>
  <rdfs:comment xml:lang="en">capability of a product to be interacted
    with by specified users to exchange information between a user and
    a system via the user interface to complete the intended task
    Note 1 to entry: Interaction capability in the product quality model
    and its subcharacteristics focus on a set of
    attributes that enable interaction by users (or operators) to complete
    specific tasks in a variety of contexts of use.
    On the other hand, usability as defined in the quality-in-use model (
    ISO/IEC 25019) comprehensively focuses on
    outcomes of use to determine whether tasks are achieved by users with
    effectiveness, efficiency and satisfaction
    in a specific context of use.
    Note 2 to entry: Interaction capability is a prerequisite for usability
    [ISO25010]
  </rdfs:comment>
</owl:Class>

<owl:Class rdf:about="http://www.example.org/iso25010#
  PerformanceEfficiency">
  <rdfs:subClassOf rdf:resource="http://www.example.org/iso25010#
    QualityAttribute"/>
  <rdfs:label xml:lang="en">Performance Efficiency</rdfs:label>
  <rdfs:comment xml:lang="en">capability of a product to perform its
    functions within specified time and throughput parameters and be
    efficient in the use of resources under specified conditions.
    Note 1 to entry: Resources can be CPU, memory, storage, and network
    devices.

```

Note 2 to entry: Resources can include other software products, the software and hardware configuration of the system, energy, and materials (e.g. print paper, storage media). [ISO25010]

```

</rdfs:comment>
</owl:Class>

<owl:Class rdf:about="http://www.example.org/iso25010#Maintainability">
  <rdfs:subClassOf rdf:resource="http://www.example.org/iso25010#
    QualityAttribute"/>
  <rdfs:label xml:lang="en">Maintainability</rdfs:label>
  <rdfs:comment xml:lang="en">capability of a product to be modified by
    the intended maintainers with effectiveness and efficiency
Note 1 to entry: Modifications can include corrections, improvements or
  adaptation of the product to changes in environment, and in
  requirements and functional specifications. Modifications include
  those carried out by specialized support staff, and those carried
  out by business or operational staff, or end users.
Note 2 to entry: Maintainability includes installation of updates and
  upgrades.
Note 3 to entry: Maintainability can be interpreted as either an
  inherent capability of the product to facilitate maintenance
  activities, or the quality-in-use experienced by the maintainers
  for the goal of maintaining the product. [ISO25010]
</rdfs:comment>
</owl:Class>

<owl:Class rdf:about="http://www.example.org/iso25010#Portability">
  <rdfs:subClassOf rdf:resource="http://www.example.org/iso25010#
    QualityAttribute"/>
  <rdfs:label xml:lang="en">Flexibility</rdfs:label>
  <rdfs:comment xml:lang="en">capability of a product to be adapted to
    changes in its requirements, contexts of use, or system environment
Note 1 to entry: Flexibility to context of use should consider two
  distinguished aspects, i.e. technical and
non-technical. The technical aspect is related with the execution
  environment of products, such as software,
hardware and communication facility; and the non-technical aspect is
  related with the social environment, such
as user and task, and the physical environment, such as climate and
  nature. [ISO25010]
</rdfs:comment>
</owl:Class>

<owl:Class rdf:about="http://www.example.org/iso25010#Security">
  <rdfs:subClassOf rdf:resource="http://www.example.org/iso25010#
    QualityAttribute"/>
  <rdfs:label xml:lang="en">Security</rdfs:label>
  <rdfs:comment xml:lang="en">capability of a product to protect
    information and data so that persons or other products have the
    degree of data access appropriate to their types and levels of

```

```

        authorization, and to defend against attackpatterns by malicious
        actors
    Note 1 to entry: As well as data stored in or by a product or system,
        security also applies to data in transmission. [ISO25010]
</rdfs:comment>
</owl:Class>

<owl:Class rdf:about="http://www.example.org/iso25010#Compatibility">
    <rdfs:subClassOf rdf:resource="http://www.example.org/iso25010#
        QualityAttribute"/>
    <rdfs:label xml:lang="en">Compatibility</rdfs:label>
    <rdfs:comment xml:lang="en">capability of a product to exchange
        information with other products, and/or to perform its required
        functions while sharing the same common environment and resources.
        [ISO25010]</rdfs:comment>
</owl:Class>

</rdf:RDF>

```

Listing 1: Mid-Level Ontology

A.2 Domain-level Ontology

```

<?xml version="1.0"?>
<rdf:RDF xmlns="http://example.org/UnifiedSimulationModelOntology#"
    xml:base="http://example.org/UnifiedSimulationModelOntology"
    xmlns:owl="http://www.w3.org/2002/07/owl#"
    xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
    xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
    xmlns:xsd="http://www.w3.org/2001/XMLSchema#">

    <owl:Ontology rdf:about="http://example.org/
        UnifiedSimulationModelOntology">
        <owl:imports rdf:resource="http://example.org/
            CANSimulationModelOntology"/>
        <owl:imports rdf:resource="http://example.org/
            LINSimulationModelOntology"/>
        <owl:imports rdf:resource="http://example.org/
            FlexRaySimulationModelOntology"/>
        <owl:imports rdf:resource="http://example.org/
            EthernetSimulationModelOntology"/>

    </owl:Ontology>

</rdf:RDF>

```

Listing 2: Domain-level Ontology

A.3 CAN Simulation Model Ontology

```

<?xml version="1.0"?>
<rdf:RDF xmlns="http://example.org/CANSimulationModelOntology#"
  xml:base="http://example.org/CANSimulationModelOntology"
  xmlns:dc="http://purl.org/dc/elements/1.1/"
  xmlns:bfo="http://purl.obolibrary.org/obo/bfo#"
  xmlns:css="https://raw.githubusercontent.com/CaSkade-Automation/CSS/
    main/CSS-Ontology#"
  xmlns:iso="http://www.example.org/iso25010#"
  xmlns:obo="http://purl.obolibrary.org/obo/"
  xmlns:ont="http://example.org/ontology#"
  xmlns:owl="http://www.w3.org/2002/07/owl#"
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:vdi="https://raw.githubusercontent.com/JonathanReif/SiS/main/
    Resources/VDI3633#"
  xmlns:idta="http://www.semanticweb.org/eklaspad/ontologies/2022/6/
    untitled-ontology-5#"
  xmlns:xml="http://www.w3.org/XML/1998/namespace"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema#"
  xmlns:foaf="http://xmlns.com/foaf/0.1/"
  xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
  xmlns:terms="http://purl.org/dc/terms/">
  <owl:Ontology rdf:about="http://example.org/CANSimulationModelOntology"
    >
    <owl:imports rdf:resource="http://example.org/ontology"/>
    <rdfs:comment></rdfs:comment>
  </owl:Ontology>

<!-- Classes -->

<!-- http://example.org/CANSimulationModelOntology#CAN_SimulationModel
-->

<owl:Class rdf:about="http://example.org/CANSimulationModelOntology#
  CAN_SimulationModel">
  <rdfs:subClassOf rdf:resource="http://www.semanticweb.org/eklaspad/
    ontologies/2022/6/untitled-ontology-5#SimulationModel"/>
  <rdfs:comment xml:lang="en">Simulation model for CAN network
    communication, supporting multi-node setups with message creation,
    Arbitration, fault injection, message frame transmission.</rdfs:
    comment>
</owl:Class>

<!-- Resources -->

<!-- http://example.org/CANSimulationModelOntology#CAN_Node -->

<owl:Class rdf:about="http://example.org/CANSimulationModelOntology#
  CAN_Node">
  <rdfs:subClassOf rdf:resource="http://www.w3id.org/hsu-aut/css#
    Resource"/>

```

```

<rdfs:comment xml:lang="en">Nodes are the individual devices or
    electronic control units (ECUs) connected to the CAN bus. Each node
    can both transmit and receive messages on the bus. In automotive
    applications, nodes can include components such as engine control
    units, airbag systems, anti-lock braking systems, instrument
    clusters, and more. Industrial applications may have nodes such as
    sensors, actuators, programmable logic controllers (PLCs), and
    human-machine interfaces (HMIs).</rdfs:comment>
</owl:Class>

<!-- http://example.org/CANSimulationModelOntology#CAN_Port -->

<owl:Class rdf:about="http://example.org/CANSimulationModelOntology#
    CAN_Port">
    <rdfs:subClassOf rdf:resource="http://www.w3id.org/hsu-aut/css#
        Resource"/>
    <owl:equivalentClass rdf:resource="http://www.semanticweb.org/eklaspad/
        ontologies/2022/6/untitled-ontology-5#portsConnector"/>
    <rdfs:comment xml:lang="en">Ports are for connecting several
        applications to the same network interface that can transmit and
        receive value events to and from the ApplicationInterface instance.
    </rdfs:comment>

</owl:Class>

<!-- Capabilities -->

<!-- http://example.org/CANSimulationModelOntology#FaultInjection -->

<owl:Class rdf:about="http://example.org/CANSimulationModelOntology#
    FaultInjection">
    <rdfs:subClassOf rdf:resource="http://www.w3id.org/hsu-aut/css#
        Capability"/>
    <owl:equivalentClass rdf:resource="http://www.semanticweb.org/eklaspad/
        ontologies/2022/6/untitled-ontology-5#simPurpose"/>
    <rdfs:comment xml:lang="en">CAN incorporates robust error detection and
        handling mechanisms through bitwise arbitration, cyclic redundancy
        check.</rdfs:comment>

</owl:Class>

<!-- http://example.org/CANSimulationModelOntology#
    MessageFrameTransmission -->

<owl:Class rdf:about="http://example.org/CANSimulationModelOntology#
    MessageFrameTransmission">
    <rdfs:subClassOf rdf:resource="http://www.w3id.org/hsu-aut/css#
        Capability"/>
    <owl:equivalentClass rdf:resource="http://www.semanticweb.org/eklaspad/
        ontologies/2022/6/untitled-ontology-5#simPurpose"/>

```

```

<rdfs:comment xml:lang="en">Communication on a CAN bus is message-
    oriented. Each message consists of an identifier representing the
    messages priority, content, and data payload. Messages are
    broadcast to all nodes on the bus, but only the intended recipient
    processes them based on the identifier.</rdfs:comment>
</owl:Class>

<!-- http://example.org/CANSimulationModelOntology#SimulationSettings -->

<owl:Class rdf:about="http://example.org/CANSimulationModelOntology#
    SimulationSettings">
    <rdfs:comment>Configuration parameters controlling simulation behavior
        and physical constraints.</rdfs:comment>
</owl:Class>

<!-- http://example.org/CANSimulationModelOntology#MessageTransmission
    -->

<owl:Class rdf:about="http://example.org/CANSimulationModelOntology#
    MessageTransmission">
    <rdfs:subClassOf rdf:resource="http://www.w3id.org/hsu-aut/css#
        Process"/>
    <rdfs:comment xml:lang="en"> Models the timing and behavior of CAN
        frame transmission on the bus.The data transmission process begins
        when a node (sender) creates a message to be transmitted onto the
        CAN bus. This message typically contains information about the data
        to be communicated, along with an identifier that determines its
        priority.</rdfs:comment>
</owl:Class>

<!-- http://example.org/CANSimulationModelOntology#
    AutomatedEmergencyBrake -->

<owl:Class rdf:about="http://example.org/CANSimulationModelOntology#
    AutomatedEmergencyBrake">
    <rdfs:subClassOf rdf:resource="https://raw.githubusercontent.com/
        JonathanReif/SiS/main/Resources/VDI3633#Simulation"/>
    <rdfs:comment xml:lang="en"> Initiating Simulation for Automated
        Emergency Braking for the CanBus.</rdfs:comment>
</owl:Class>

<!-- http://example.org/CANSimulationModelOntology#Reliability -->

<owl:Class rdf:about="http://example.org/CANSimulationModelOntology#
    Reliability">
    <owl:equivalentClass rdf:resource="http://www.example.org/iso25010#
        Reliability"/>
</owl:Class>

```

```

<!-- http://example.org/CANSimulationModelOntology#
TransmissionEfficiency -->

<owl:Class rdf:about="http://example.org/CANSimulationModelOntology#
TransmissionEfficiency">
  <rdfs:subClassOf rdf:resource="http://www.example.org/iso25010#
PerformanceEfficiency"/>
</owl:Class>

<!-- Object Properties -->

<!-- http://example.org/CANSimulationModelOntology#hasPort -->

<owl:ObjectProperty rdf:about="http://example.org/
CANSimulationModelOntology#hasPort">
  <rdfs:domain rdf:resource="http://example.org/
CANSimulationModelOntology#CAN_Node"/>
  <rdfs:range rdf:resource="http://example.org/
CANSimulationModelOntology#CAN_Port"/>
  <rdfs:comment xml:lang="en">A can node will have port for incoming
and outgoing message frames.</rdfs:comment>

</owl:ObjectProperty>

<owl:ObjectProperty rdf:about="http://example.org/
CANSimulationModelOntology#hasSettings">
  <rdfs:domain rdf:resource="http://example.org/
CANSimulationModelOntology#CAN_SimulationModel"/>
  <rdfs:range rdf:resource="http://example.org/
CANSimulationModelOntology#SimulationSettings"/>
  <rdfs:comment xml:lang="en">Can simulation model will need the
required simulation settings for conducting the simulation.</
rdfs:comment>

</owl:ObjectProperty>

<owl:ObjectProperty rdf:about="http://example.org/
CANSimulationModelOntology#communicatesWith">
  <rdfs:domain rdf:resource="http://example.org/
CANSimulationModelOntology#MessageFrameTransmission"/>
  <rdfs:range rdf:resource="http://example.org/CANSimulationModelOntology
#MessageFrameTransmission"/>
  <rdfs:comment xml:lang="en">Links a TX message frame to the
corresponding RX frame indicating communication between sender and
receiver.</rdfs:comment>
</owl:ObjectProperty>

<rdfs:Description rdf:about="http://example.org/CANSimulationModelOntology
#CAN_SimulationModel">

```

```

    <ont:usesResource rdf:resource="http://example.org/
        CANSimulationModelOntology#CAN_Node"/>
</rdf:Description>

<rdf:Description rdf:about="http://example.org/CANSimulationModelOntology
    #CAN_SimulationModel">
    <ont:describes rdf:resource="http://example.org/
        CANSimulationModelOntology#AutomatedEmergencyBrake"/>
</rdf:Description>

<rdf:Description rdf:about="http://example.org/CANSimulationModelOntology
    #CAN_SimulationModel">
    <ont:hasCapability rdf:resource="http://example.org/
        CANSimulationModelOntology#FaultInjection"/>
    <ont:hasCapability rdf:resource="http://example.org/
        CANSimulationModelOntology#MessageFrameTransmission"/>
</rdf:Description>

<rdf:Description rdf:about="http://example.org/CANSimulationModelOntology
    #CAN_Node">
    <ont:providesCapability rdf:resource="http://example.org/
        CANSimulationModelOntology#FaultInjection"/>
    <ont:providesCapability rdf:resource="http://example.org/
        CANSimulationModelOntology#MessageFrameTransmission"/>
</rdf:Description>

<!-- DataType Properties -->

<owl:DatatypeProperty rdf:about="http://example.org/
    CANSimulationModelOntology#hasBitRate">
    <rdfs:domain rdf:resource="http://example.org/
        CANSimulationModelOntology#CAN_SimulationModel"/>
    <rdfs:range rdf:resource="xsd:integer"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="http://example.org/
    CANSimulationModelOntology#hasBitStuffingMode">
    <rdfs:domain rdf:resource="http://example.org/
        CANSimulationModelOntology#CAN_SimulationModel"/>
    <rdfs:range rdf:resource="xsd:string"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="http://example.org/
    CANSimulationModelOntology#hasLogTransmissions">
    <rdfs:domain rdf:resource="http://example.org/
        CANSimulationModelOntology#CAN_SimulationModel"/>
    <rdfs:range rdf:resource="xsd:boolean"/>
</owl:DatatypeProperty>

```

```

<owl:DatatypeProperty rdf:about="http://example.org/
  CANSimulationModelOntology#hasName">
  <rdfs:domain rdf:resource="http://example.org/
    CANSimulationModelOntology#CAN_Node"/>
  <rdfs:range rdf:resource="xsd:string"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="http://example.org/
  CANSimulationModelOntology#hasDropDuplicatedFrames">
  <rdfs:domain rdf:resource="http://example.org/
    CANSimulationModelOntology#CAN_Node"/>
  <rdfs:range rdf:resource="xsd:boolean"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="http://example.org/
  CANSimulationModelOntology#hasStartTimeMillis">
  <rdfs:domain rdf:resource="http://example.org/
    CANSimulationModelOntology#FaultInjection"/>
  <rdfs:range rdf:resource="xsd:integer"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="http://example.org/
  CANSimulationModelOntology#hasEndTimeMillis">
  <rdfs:domain rdf:resource="http://example.org/
    CANSimulationModelOntology#FaultInjection"/>
  <rdfs:range rdf:resource="xsd:integer"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="http://example.org/
  CANSimulationModelOntology#hasErrorModel">
  <rdfs:domain rdf:resource="http://example.org/
    CANSimulationModelOntology#FaultInjection"/>
  <rdfs:range rdf:resource="xsd:string"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="http://example.org/
  CANSimulationModelOntology#hasErrorProcessor">
  <rdfs:domain rdf:resource="http://example.org/
    CANSimulationModelOntology#FaultInjection"/>
  <rdfs:range rdf:resource="xsd:string"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="http://example.org/
  CANSimulationModelOntology#hasAddress">
  <rdfs:domain rdf:resource="http://example.org/
    CANSimulationModelOntology#MessageFrameTransmission"/>
  <rdfs:range rdf:resource="xsd:string"/>
</owl:DatatypeProperty>

```

```

<owl:DatatypeProperty rdf:about="http://example.org/
  CANSimulationModelOntology#hasPortName">
  <rdfs:domain rdf:resource="http://example.org/
    CANSimulationModelOntology#MessageFrameTransmission"/>
  <rdfs:range rdf:resource="xsd:string"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="http://example.org/
  CANSimulationModelOntology#hasSimulatedPayloadSize">
  <rdfs:domain rdf:resource="http://example.org/
    CANSimulationModelOntology#MessageFrameTransmission"/>
  <rdfs:range rdf:resource="xsd:integer"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="http://example.org/
  CANSimulationModelOntology#hasType">
  <rdfs:domain rdf:resource="http://example.org/
    CANSimulationModelOntology#MessageFrameTransmission"/>
  <rdfs:range rdf:resource="xsd:string"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="http://example.org/
  CANSimulationModelOntology#hasSpeedMetersPerSecond">
  <rdfs:domain rdf:resource="http://example.org/
    CANSimulationModelOntology#SimulationSettings"/>
  <rdfs:range rdf:resource="xsd:float"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="http://example.org/
  CANSimulationModelOntology#hasStartingDistanceMeters">
  <rdfs:domain rdf:resource="http://example.org/
    CANSimulationModelOntology#SimulationSettings"/>
  <rdfs:range rdf:resource="xsd:float"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="http://example.org/
  CANSimulationModelOntology#hasSensorTimeStepNs">
  <rdfs:domain rdf:resource="http://example.org/
    CANSimulationModelOntology#SimulationSettings"/>
  <rdfs:range rdf:resource="xsd:integer"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="http://example.org/
  CANSimulationModelOntology#hasMinimumBrakingDistance">
  <rdfs:domain rdf:resource="http://example.org/
    CANSimulationModelOntology#SimulationSettings"/>
  <rdfs:range rdf:resource="xsd:float"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="http://example.org/
  CANSimulationModelOntology#hasSimulationDuration">

```

```

    <rdfs:domain rdf:resource="http://example.org/
      CANSimulationModelOntology#SimulationSettings"/>
    <rdfs:range rdf:resource="xsd:float"/>
  </owl:DatatypeProperty>

  <owl:DatatypeProperty rdf:about="http://example.org/
    CANSimulationModelOntology#hasBrakingDuration">
    <rdfs:domain rdf:resource="http://example.org/
      CANSimulationModelOntology#SimulationSettings"/>
    <rdfs:range rdf:resource="xsd:float"/>
  </owl:DatatypeProperty>

</rdf:RDF>

```

Listing 3: CAN Simulation Model Ontology

A.4 CAN use-case-level Ontology

```

<?xml version="1.0"?>
<rdf:RDF xmlns="http://example.org/CANUseCase#"
  xml:base="http://example.org/CANUseCase"
  xmlns:can="http://example.org/CANSimulationModelOntology#"
  xmlns:ont="http://example.org/ontology#"
  xmlns:css="https://raw.githubusercontent.com/CaSkade-Automation/CSS/
    main/CSS-Ontology#"
  xmlns:owl="http://www.w3.org/2002/07/owl#"
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema#">

  <owl:Ontology rdf:about="http://example.org/CANUseCase">
    <owl:imports rdf:resource="http://example.org/
      CANSimulationModelOntology"/>
  <owl:imports rdf:resource="http://example.org/ontology"/>
  </owl:Ontology>

  <!-- CAN_SimulationModel Instance -->
  <owl:NamedIndividual rdf:about="http://example.org/CANUseCase#
    CanVehicleControlConfig">
    <rdf:type rdf:resource="http://example.org/
      CANSimulationModelOntology#CAN_SimulationModel"/>
    <can:hasBitRate rdf:datatype="xsd:integer">100000</can:hasBitRate>
    <can:hasBitStuffingMode rdf:datatype="xsd:string">None</can:
      hasBitStuffingMode>
    <can:hasLogTransmissions rdf:datatype="xsd:boolean">true</can:
      hasLogTransmissions>
    <ont:usesResource rdf:resource="http://example.org/CANUseCase#
      BrakeNode"/>
    <ont:usesResource rdf:resource="http://example.org/CANUseCase#
      GatewayNode"/>
  </owl:NamedIndividual>

```

```

<ont:usesResource rdf:resource="http://example.org/CANUseCase#
  VehicleControllerNode"/>
<ont:describes rdf:resource="http://example.org/CANUseCase#
  EmergencyBraking"/>
<can:hasSettings rdf:resource="http://example.org/CANUseCase#
  EmergencyBrakingSettings"/>
<ont:hasCapability rdf:resource="http://example.org/CANUseCase#TxType"/
  >
<ont:hasCapability rdf:resource="http://example.org/CANUseCase#RxType3"
  />
<ont:hasCapability rdf:resource="http://example.org/CANUseCase#TxType4"
  />
<ont:hasCapability rdf:resource="http://example.org/CANUseCase#Fault1"/
  >
<ont:hasCapability rdf:resource="http://example.org/CANUseCase#RxType"/
  >
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="http://example.org/CANUseCase#
  CanSensorConfig">
  <rdf:type rdf:resource="http://example.org/
    CANSimulationModelOntology#CAN_SimulationModel"/>
  <can:hasBitRate rdf:datatype="xsd:integer">100000</can:hasBitRate>
  <can:hasBitStuffingMode rdf:datatype="xsd:string">None</can:
    hasBitStuffingMode>
  <can:hasLogTransmissions rdf:datatype="xsd:boolean">>false</can:
    hasLogTransmissions>
  <ont:usesResource rdf:resource="http://example.org/CANUseCase#
    SensorRedundancyNode"/>
  <ont:usesResource rdf:resource="http://example.org/CANUseCase#
    SensorMainNode"/>
  <ont:usesResource rdf:resource="http://example.org/CANUseCase#
    GatewayNode"/>
  <ont:usesResource rdf:resource="http://example.org/CANUseCase#
    DistanceEvaluatorNode"/>
  <ont:describes rdf:resource="http://example.org/CANUseCase#
    EmergencyBraking"/>
  <can:hasSettings rdf:resource="http://example.org/CANUseCase#
    EmergencyBrakingSettings"/>
  <ont:hasCapability rdf:resource="http://example.org/CANUseCase#TxType3"
    />
  <ont:hasCapability rdf:resource="http://example.org/CANUseCase#RxType2"
    />
  <ont:hasCapability rdf:resource="http://example.org/CANUseCase#RxType1"
    />
  <ont:hasCapability rdf:resource="http://example.org/CANUseCase#TxType2"
    />
  <ont:hasCapability rdf:resource="http://example.org/CANUseCase#TxType1"
    />
  <ont:hasCapability rdf:resource="http://example.org/CANUseCase#Fault2"/
    >

```

```

</owl:NamedIndividual>

<!-- CAN_Node Instances -->

<owl:NamedIndividual rdf:about="http://example.org/CANUseCase#
SensorRedundancyNode">
  <rdf:type rdf:resource="http://example.org/
    CANSimulationModelOntology#CAN_Node"/>
  <can:hasName rdf:datatype="xsd:string">sensor_redundancy_node</can:
    hasName>
  <can:hasDropDuplicatedFrames rdf:datatype="xsd:boolean">true</can:
    hasDropDuplicatedFrames>
  <ont:providesCapability rdf:resource="http://example.org/CANUseCase
    #TxType1"/>
  <ont:providesCapability rdf:resource="http://example.org/CANUseCase
    #Fault2"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="http://example.org/CANUseCase#
SensorMainNode">
  <rdf:type rdf:resource="http://example.org/
    CANSimulationModelOntology#CAN_Node"/>
  <can:hasName rdf:datatype="xsd:string">sensor_main_node</can:
    hasName>
  <can:hasDropDuplicatedFrames rdf:datatype="xsd:boolean">true</can:
    hasDropDuplicatedFrames>
  <ont:providesCapability rdf:resource="http://example.org/CANUseCase
    #TxType2"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="http://example.org/CANUseCase#GatewayNode
">
  <rdf:type rdf:resource="http://example.org/
    CANSimulationModelOntology#CAN_Node"/>
  <can:hasName rdf:datatype="xsd:string">gateway_node</can:hasName>
  <can:hasDropDuplicatedFrames rdf:datatype="xsd:boolean">true</can:
    hasDropDuplicatedFrames>
  <ont:providesCapability rdf:resource="http://example.org/CANUseCase
    #RxType1"/>
  <ont:providesCapability rdf:resource="http://example.org/CANUseCase#
    TxType4"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="http://example.org/CANUseCase#
DistanceEvaluatorNode">
  <rdf:type rdf:resource="http://example.org/
    CANSimulationModelOntology#CAN_Node"/>
  <can:hasName rdf:datatype="xsd:string">distance_evaluator_node</can:
    :hasName>
  <can:hasDropDuplicatedFrames rdf:datatype="xsd:boolean">true</can:
    hasDropDuplicatedFrames>

```

```

        <ont:providesCapability rdf:resource="http://example.org/CANUseCase#TxType3"/>
    <ont:providesCapability rdf:resource="http://example.org/CANUseCase#RxType2"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="http://example.org/CANUseCase#VehicleControllerNode">
    <rdf:type rdf:resource="http://example.org/CANSimulationModelOntology#CAN_Node"/>
    <can:hasName rdf:datatype="xsd:string">vehicle_controller_node</can:hasName>
    <can:hasDropDuplicatedFrames rdf:datatype="xsd:boolean">true</can:hasDropDuplicatedFrames>
    <ont:providesCapability rdf:resource="http://example.org/CANUseCase#TxType"/>
    <ont:providesCapability rdf:resource="http://example.org/CANUseCase#RxType3"/>
    <ont:providesCapability rdf:resource="http://example.org/CANUseCase#Fault1"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="http://example.org/CANUseCase#BrakeNode">
    <rdf:type rdf:resource="http://example.org/CANSimulationModelOntology#CAN_Node"/>
    <can:hasName rdf:datatype="xsd:string">brake_node</can:hasName>
    <can:hasDropDuplicatedFrames rdf:datatype="xsd:boolean">false</can:hasDropDuplicatedFrames>
    <ont:providesCapability rdf:resource="http://example.org/CANUseCase#RxType"/>
</owl:NamedIndividual>

<!-- Capabilities -->
<owl:NamedIndividual rdf:about="http://example.org/CANUseCase#TxType">
    <rdf:type rdf:resource="http://example.org/CANSimulationModelOntology#MessageFrameTransmission"/>
    <can:hasType rdf:datatype="xsd:string">TX</can:hasType>
    <can:hasAddress rdf:datatype="xsd:string">fd:4</can:hasAddress>
    <can:hasSimulatedPayloadSize rdf:datatype="xsd:integer">4</can:hasSimulatedPayloadSize>
    <can:hasPortName rdf:datatype="xsd:string">vehicle_controller_tx</can:hasPortName>
    <can:communicatesWith rdf:resource="http://example.org/CANUseCase#RxType"/>
    <rdfs:label xml:lang="en">Transmission of Vehicle Controller node</rdfs:label>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="http://example.org/CANUseCase#TxType1">

```

```

<rdf:type rdf:resource="http://example.org/
    CANSimulationModelOntology#MessageFrameTransmission"/>
<can:hasType rdf:datatype="xsd:string">TX</can:hasType>
<can:hasAddress rdf:datatype="xsd:string">fd:3</can:hasAddress>
<can:hasSimulatedPayloadSize rdf:datatype="xsd:integer">4</can:
    hasSimulatedPayloadSize>
<can:hasPortName rdf:datatype="xsd:string">sensor_redundancy_tx</
    can:hasPortName>
<can:communicatesWith rdf:resource="http://example.org/CANUseCase#
    RxType2"/>
<rdfs:label xml:lang="en">Transmission of Sensor Redundancy node</rdfs:
    label>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="http://example.org/CANUseCase#TxType2">
    <rdf:type rdf:resource="http://example.org/
        CANSimulationModelOntology#MessageFrameTransmission"/>
    <can:hasType rdf:datatype="xsd:string">TX</can:hasType>
    <can:hasAddress rdf:datatype="xsd:string">fd:3</can:hasAddress>
    <can:hasSimulatedPayloadSize rdf:datatype="xsd:integer">4</can:
        hasSimulatedPayloadSize>
    <can:hasPortName rdf:datatype="xsd:string">sensor_main_tx</can:
        hasPortName>
    <can:communicatesWith rdf:resource="http://example.org/CANUseCase#
        RxType2"/>
    <rdfs:label xml:lang="en">Transmission of Sensor Main node</rdfs:label>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="http://example.org/CANUseCase#TxType3">
    <rdf:type rdf:resource="http://example.org/
        CANSimulationModelOntology#MessageFrameTransmission"/>
    <can:hasType rdf:datatype="xsd:string">TX</can:hasType>
    <can:hasAddress rdf:datatype="xsd:string">fd:2</can:hasAddress>
    <can:hasSimulatedPayloadSize rdf:datatype="xsd:integer">4</can:
        hasSimulatedPayloadSize>
    <can:hasPortName rdf:datatype="xsd:string">distance_evaluator_tx</
        can:hasPortName>
    <can:communicatesWith rdf:resource="http://example.org/CANUseCase#
        RxType1"/>
    <rdfs:label xml:lang="en">Transmission of Distance Evaluator node</rdfs:
        label>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="http://example.org/CANUseCase#TxType4">
    <rdf:type rdf:resource="http://example.org/
        CANSimulationModelOntology#MessageFrameTransmission"/>
    <can:hasType rdf:datatype="xsd:string">TX</can:hasType>
    <can:hasAddress rdf:datatype="xsd:string">fd:2</can:hasAddress>
    <can:hasSimulatedPayloadSize rdf:datatype="xsd:integer">4</can:
        hasSimulatedPayloadSize>

```

```

    <can:hasPortName rdf:datatype="xsd:string">gateway_tx</can:
      hasPortName>
  <can:communicatesWith rdf:resource="http://example.org/CANUseCase#
    RxType3"/>
  <rdfs:label xml:lang="en">Transmission of Gateway node</rdfs:label>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="http://example.org/CANUseCase#RxType">
  <rdf:type rdf:resource="http://example.org/
    CANSimulationModelOntology#MessageFrameTransmission"/>
  <can:hasType rdf:datatype="xsd:string">RX</can:hasType>
  <can:hasAddress rdf:datatype="xsd:string">fd:4</can:hasAddress>
  <can:hasSimulatedPayloadSize rdf:datatype="xsd:integer">4</can:
    hasSimulatedPayloadSize>
  <can:hasPortName rdf:datatype="xsd:string">brake_rx</can:
    hasPortName>
  <rdfs:label xml:lang="en">Reception of Brake node</rdfs:label>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="http://example.org/CANUseCase#RxType1">
  <rdf:type rdf:resource="http://example.org/
    CANSimulationModelOntology#MessageFrameTransmission"/>
  <can:hasType rdf:datatype="xsd:string">RX</can:hasType>
  <can:hasAddress rdf:datatype="xsd:string">fd:2</can:hasAddress>
  <can:hasSimulatedPayloadSize rdf:datatype="xsd:integer">4</can:
    hasSimulatedPayloadSize>
  <can:hasPortName rdf:datatype="xsd:string">gateway_rx</can:
    hasPortName>
  <rdfs:label xml:lang="en">Reception of Gateway node</rdfs:label>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="http://example.org/CANUseCase#RxType2">
  <rdf:type rdf:resource="http://example.org/
    CANSimulationModelOntology#MessageFrameTransmission"/>
  <can:hasType rdf:datatype="xsd:string">RX</can:hasType>
  <can:hasAddress rdf:datatype="xsd:string">fd:3</can:hasAddress>
  <can:hasSimulatedPayloadSize rdf:datatype="xsd:integer">4</can:
    hasSimulatedPayloadSize>
  <can:hasPortName rdf:datatype="xsd:string">distance_evaluator_rx</
    can:hasPortName>
  <rdfs:label xml:lang="en">Reception of Distance Evaluator node</rdfs:
    label>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="http://example.org/CANUseCase#RxType3">
  <rdf:type rdf:resource="http://example.org/
    CANSimulationModelOntology#MessageFrameTransmission"/>
  <can:hasType rdf:datatype="xsd:string">RX</can:hasType>
  <can:hasAddress rdf:datatype="xsd:string">fd:2</can:hasAddress>
  <can:hasSimulatedPayloadSize rdf:datatype="xsd:integer">4</can:
    hasSimulatedPayloadSize>

```

```

        <can:hasPortName rdf:datatype="xsd:string">vehicle_controller_rx</
            can:hasPortName>
    <rdfs:label xml:lang="en">Reception of Vehicle Controller node</rdfs:
        label>
</owl:NamedIndividual>

<!-- Fault Injection -->

<owl:NamedIndividual rdf:about="http://example.org/CANUseCase#Fault1">
    <rdf:type rdf:resource="http://example.org/
        CANSimulationModelOntology#FaultInjection"/>
    <can:hasErrorModel rdf:datatype="xsd:string">SenderLinkFailure</can:
        hasErrorModel>
    <can:hasErrorProcessor rdf:datatype="xsd:string">DropFrame</can:
        hasErrorProcessor>
    <can:hasStartTimeMillis rdf:datatype="xsd:integer">300</can:
        hasStartTimeMillis>
    <can:hasEndTimeMillis rdf:datatype="xsd:integer">600</can:
        hasEndTimeMillis>
    <rdfs:label xml:lang="en">Drop Frame Fault</rdfs:label>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="http://example.org/CANUseCase#Fault2">
    <rdf:type rdf:resource="http://example.org/
        CANSimulationModelOntology#FaultInjection"/>
    <can:hasErrorModel rdf:datatype="xsd:string">SenderLinkFailure</can:
        hasErrorModel>
    <can:hasErrorProcessor rdf:datatype="xsd:string">ChangeSignDouble</
        can:hasErrorProcessor>
    <can:hasStartTimeMillis rdf:datatype="xsd:integer">11000</can:
        hasStartTimeMillis>
    <can:hasEndTimeMillis rdf:datatype="xsd:integer">12000</can:
        hasEndTimeMillis>
    <rdfs:label xml:lang="en">Change Sign Fault</rdfs:label>
</owl:NamedIndividual>

<!-- Simulation Scenario -->

<owl:NamedIndividual rdf:about="http://example.org/CANUseCase#
    EmergencyBraking">
    <rdf:type rdf:resource="http://example.org/CANSimulationModelOntology#
        AutomatedEmergencyBrake"/>
    <!-- <rdf:type rdf:resource="https://raw.githubusercontent.com/
        JonathanReif/SiS/main/Resources/VDI3633#Simulation"/>-->
    <rdfs:label xml:lang="en">Emergency Braking scenario</rdfs:label>
</owl:NamedIndividual>

<!-- Simulation Settings -->

<owl:NamedIndividual rdf:about="http://example.org/CANUseCase#
    EmergencyBrakingSettings">

```

```

<rdf:type rdf:resource="http://example.org/CANSimulationModelOntology#
SimulationSettings"/>
<can:hasSpeedMetersPerSecond rdf:datatype="xsd:float">15.0</can:
hasSpeedMetersPerSecond>
<can:hasStartingDistanceMeters rdf:datatype="xsd:float">200.0</can:
hasStartingDistanceMeters>
<can:hasSensorTimeStepNs rdf:datatype="xsd:integer">10000000</can:
hasSensorTimeStepNs>
<can:hasMinimumBrakingDistance rdf:datatype="xsd:float">30.16</can:
hasMinimumBrakingDistance>
<can:hasSimulationDuration rdf:datatype="xsd:float">13.33</can:
hasSimulationDuration>
<can:hasBrakingDuration rdf:datatype="xsd:float">2.01</can:
hasBrakingDuration>
</owl:NamedIndividual>

</rdf:RDF>

```

Listing 4: CAN use-case-level Ontology

A.5 LIN Simulation Model Ontology

```

<?xml version="1.0"?>
<rdf:RDF xmlns="http://example.org/LINSimulationModelOntology#"
xml:base="http://example.org/LINSimulationModelOntology"
xmlns:dc="http://purl.org/dc/elements/1.1/"
xmlns:bfo="http://purl.obolibrary.org/obo/bfo#"
xmlns:css="https://raw.githubusercontent.com/CaSkade-Automation/CSS/
main/CSS-Ontology#"
xmlns:obo="http://purl.obolibrary.org/obo/"
xmlns:ont="http://example.org/ontology#"
xmlns:owl="http://www.w3.org/2002/07/owl#"
xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
xmlns:xml="http://www.w3.org/XML/1998/namespace"
xmlns:xsd="http://www.w3.org/2001/XMLSchema#"
xmlns:foaf="http://xmlns.com/foaf/0.1/"
xmlns:idta="http://www.semanticweb.org/eklaspad/ontologies/2022/6/
untitled-ontology-5#"
xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
xmlns:terms="http://purl.org/dc/terms/"
xmlns:vdi3633="https://raw.githubusercontent.com/JonathanReif/SiS/main
/Resources/VDI3633#"
xmlns:iso25010="http://www.example.org/iso25010#">

<owl:Ontology rdf:about="http://example.org/LINSimulationModelOntology"
>
  <owl:imports rdf:resource="http://example.org/ontology"/>
</owl:Ontology>

```

```

<!-- LIN Simulation Model -->

<owl:Class rdf:about="http://example.org/LINSimulationModelOntology#
  LIN_SimulationModel">
  <rdfs:subClassOf rdf:resource="http://www.semanticweb.org/eklaspad/
    ontologies/2022/6/untitled-ontology-5#SimulationModel"/>
  <rdfs:comment xml:lang="en">Simulation model for LIN master-slave
    communication.</rdfs:comment>
</owl:Class>

<!-- Capabilities -->

<owl:Class rdf:about="http://example.org/LINSimulationModelOntology#
  MasterSlaveScheduling">
  <rdfs:subClassOf rdf:resource="http://www.w3id.org/hsu-aut/css#
    Capability"/>
  <owl:equivalentClass rdf:resource="http://www.semanticweb.org/eklaspad/
    ontologies/2022/6/untitled-ontology-5#simPurpose"/>
  <rdfs:comment xml:lang="en">Validation of LIN master-slave scheduling and
    communication.</rdfs:comment>
</owl:Class>

<owl:Class rdf:about="http://example.org/LINSimulationModelOntology#
  FrameTransmission">
  <rdfs:subClassOf rdf:resource="http://www.w3id.org/hsu-aut/css#
    Capability"/>
  <owl:equivalentClass rdf:resource="http://www.semanticweb.org/eklaspad/
    ontologies/2022/6/untitled-ontology-5#simPurpose"/>
  <rdfs:comment xml:lang="en">Simulating transmission of different frame
    types such as unconditional, sporadic, and event triggered.</rdfs:
    comment>
</owl:Class>

<!--
<owl:Class rdf:about="http://example.org/LINSimulationModelOntology#
  TimingConstraintEvaluation">
  <rdfs:subClassOf rdf:resource="http://www.w3id.org/hsu-aut/css#
    Capability"/>
</owl:Class> -->

<!-- Resources -->

<owl:Class rdf:about="http://example.org/LINSimulationModelOntology#
  LIN_MasterNode">
  <rdfs:subClassOf rdf:resource="http://www.w3id.org/hsu-aut/css#Resource
    "/>
  <rdfs:comment xml:lang="en">Always controls the bus scheduling and
    initiates all frame transfers in the LIN network.</rdfs:comment>
</owl:Class>

```

```

<owl:Class rdf:about="http://example.org/LINSimulationModelOntology#
  LIN_SlaveNode">
  <rdfs:subClassOf rdf:resource="http://www.w3id.org/hsu-aut/css#Resource
    "/>
  <rdfs:comment xml:lang="en">Responds to master requests (cannot initiate
    frames) in the LIN network.</rdfs:comment>
</owl:Class>

<owl:Class rdf:about="http://example.org/LINSimulationModelOntology#
  LIN_Port">
  <rdfs:subClassOf rdf:resource="http://www.w3id.org/hsu-aut/css#Resource
    "/>
  <owl:equivalentClass rdf:resource="http://www.semanticweb.org/eklaspad/
    ontologies/2022/6/untitled-ontology-5#portsConnector"/>
  <rdfs:comment xml:lang="en">Ports are for enabling bidirectional
    communication between master and slave nodes in the same network
    interface that can transmit and receive frames.</rdfs:comment>
</owl:Class>

<!-- Processes -->

<owl:Class rdf:about="http://example.org/LINSimulationModelOntology#
  MasterScheduledCommunication">
  <rdfs:subClassOf rdf:resource="http://www.w3id.org/hsu-aut/css#Process"
    />
  <rdfs:comment xml:lang="en"> Master controlled communication through the
    scheduling mechanism defined in the LDF or manual configuration.</
    rdfs:comment>
</owl:Class>

<!-- Quality Attributes -->

<owl:Class rdf:about="http://example.org/LINSimulationModelOntology#
  SchedulingAccuracy">
  <rdfs:subClassOf rdf:resource="http://www.example.org/iso25010#
    PerformanceEfficiency"/>
</owl:Class>

<!-- Object Properties -->

<rdf:Description rdf:about="http://example.org/LINSimulationModelOntology
  #LIN_SimulationModel">
  <ont:usesResource rdf:resource="http://example.org/
    LINSimulationModelOntology#LIN_MasterNode"/>
  <ont:usesResource rdf:resource="http://example.org/
    LINSimulationModelOntology#LIN_SlaveNode"/>
</rdf:Description>

<rdf:Description rdf:about="http://example.org/LINSimulationModelOntology
  #LIN_SimulationModel">

```

```

    <ont:hasCapability rdf:resource="http://example.org/
      LINSimulationModelOntology#MasterSlaveScheduling"/>
    <ont:hasCapability rdf:resource="http://example.org/
      LINSimulationModelOntology#FrameTransmission"/>
  </rdf:Description>

<!-- DataType Properties -->

<owl:DatatypeProperty rdf:about="http://example.org/
  LINSimulationModelOntology#hasBitRate">
  <rdfs:domain rdf:resource="http://example.org/
    LINSimulationModelOntology#LIN_SimulationModel"/>
  <rdfs:range rdf:resource="xsd:integer"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="http://example.org/
  LINSimulationModelOntology#hasTimeBase">
  <rdfs:domain rdf:resource="http://example.org/
    LINSimulationModelOntology#LIN_SimulationModel"/>
  <rdfs:range rdf:resource="xsd:integer"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="http://example.org/
  LINSimulationModelOntology#hasScheduleStartDelay">
  <rdfs:domain rdf:resource="http://example.org/
    LINSimulationModelOntology#LIN_SimulationModel"/>
  <rdfs:range rdf:resource="xsd:integer"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="http://example.org/
  LINSimulationModelOntology#hasPayloadSize">
  <rdfs:domain rdf:resource="http://example.org/
    LINSimulationModelOntology#LIN_Port"/>
  <rdfs:range rdf:resource="xsd:integer"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="http://example.org/
  LINSimulationModelOntology#hasFrameID">
  <rdfs:domain rdf:resource="http://example.org/
    LINSimulationModelOntology#LIN_Port"/>
  <rdfs:range rdf:resource="xsd:integer"/>
</owl:DatatypeProperty>

</rdf:RDF>

```

Listing 5: LIN Simulation Model Ontology

A.6 Ethernet Simulation Model Ontology

```
<?xml version="1.0"?>
```

```

<rdf:RDF xmlns="http://example.org/EthernetSimulationModelOntology#"
  xml:base="http://example.org/EthernetSimulationModelOntology"
  xmlns:dc="http://purl.org/dc/elements/1.1/"
  xmlns:bfo="http://purl.obolibrary.org/obo/bfo#"
  xmlns:css="https://raw.githubusercontent.com/CaSkade-Automation/CSS/
    main/CSS-Ontology#"
  xmlns:obo="http://purl.obolibrary.org/obo/"
  xmlns:ont="http://example.org/ontology#"
  xmlns:owl="http://www.w3.org/2002/07/owl#"
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:xml="http://www.w3.org/XML/1998/namespace"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema#"
  xmlns:foaf="http://xmlns.com/foaf/0.1/"
  xmlns:idta="http://www.semanticweb.org/eklaspad/ontologies/2022/6/
    untitled-ontology-5#"
  xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
  xmlns:terms="http://purl.org/dc/terms/"
  xmlns:vd13633="https://raw.githubusercontent.com/JonathanReif/SiS/main
    /Resources/VDI3633#"
  xmlns:iso25010="http://www.example.org/iso25010#">

  <owl:Ontology rdf:about="http://example.org/
    EthernetSimulationModelOntology">
    <owl:imports rdf:resource="http://example.org/ontology"/>
  </owl:Ontology>

  <!-- Ethernet Simulation Model -->

  <owl:Class rdf:about="http://example.org/EthernetSimulationModelOntology#
    Ethernet_SimulationModel">
    <rdfs:subClassOf rdf:resource="http://www.semanticweb.org/eklaspad/
      ontologies/2022/6/untitled-ontology-5#SimulationModel"/>
    <rdfs:comment xml:lang="en">Simulation model for Ethernet communication
      .</rdfs:comment>
  </owl:Class>

  <!-- Capabilities -->

  <owl:Class rdf:about="http://example.org/EthernetSimulationModelOntology#
    StoreAndForwardCommunication">
    <rdfs:subClassOf rdf:resource="http://www.w3id.org/hsu-aut/css#
      Capability"/>
    <owl:equivalentClass rdf:resource="http://www.semanticweb.org/eklaspad/
      ontologies/2022/6/untitled-ontology-5#simPurpose"/>
    <rdfs:comment xml:lang="en">Simulation of switched Ethernet networks for
      store-and-forward communication.</rdfs:comment>
  </owl:Class>

  <owl:Class rdf:about="http://example.org/EthernetSimulationModelOntology#
    MulticastCommunication">

```

```

    <rdfs:subClassOf rdf:resource="http://www.w3id.org/hsu-aut/css#
      Capability"/>
  <owl:equivalentClass rdf:resource="http://www.semanticweb.org/eklaspad/
    ontologies/2022/6/untitled-ontology-5#simPurpose"/>
  <rdfs:comment xml:lang="en">Simulation of unswitched Ethernet networks
    for multicast communication.</rdfs:comment>
</owl:Class>

<!-- Resources -->

<owl:Class rdf:about="http://example.org/EthernetSimulationModelOntology#
  Ethernet_Node">
  <rdfs:subClassOf rdf:resource="http://www.w3id.org/hsu-aut/css#Resource
    "/>
  <rdfs:comment xml:lang="en">Nodes are the individual devices connected to
    the Ethernet network. Each node can both transmit and receive
    messages on the network. </rdfs:comment>
</owl:Class>

<owl:Class rdf:about="http://example.org/EthernetSimulationModelOntology#
  Ethernet_Port">
  <rdfs:subClassOf rdf:resource="http://www.w3id.org/hsu-aut/css#Resource
    "/>
  <owl:equivalentClass rdf:resource="http://www.semanticweb.org/eklaspad/
    ontologies/2022/6/untitled-ontology-5#portsConnector"/>
  <rdfs:comment xml:lang="en">Ports are for connecting several applications
    to the same network interface that can transmit and receive value
    events to and from the ApplicationInterface instance.</rdfs:comment>
</owl:Class>

<!-- Processes -->

<owl:Class rdf:about="http://example.org/EthernetSimulationModelOntology#
  Ethernet_PacketTransmission">
  <rdfs:subClassOf rdf:resource="http://www.w3id.org/hsu-aut/css#Process"
    />
  <rdfs:comment xml:lang="en">Models packet transmission over Ethernet
    medium (switched/unswitched). </rdfs:comment>
</owl:Class>

<!-- Quality Attributes -->

<owl:Class rdf:about="http://example.org/EthernetSimulationModelOntology#
  Efficiency">
  <rdfs:subClassOf rdf:resource="http://www.example.org/iso25010#
    PerformanceEfficiency"/>
</owl:Class>

```

```

<owl:Class rdf:about="http://example.org/EthernetSimulationModelOntology#
    Reliability">
    <owl:equivalentClass rdf:resource="http://www.example.org/iso25010#
        Reliability"/>
</owl:Class>

<!-- Object Properties -->

<rdf:Description rdf:about="http://example.org/
    EthernetSimulationModelOntology#Ethernet_SimulationModel">
    <ont:usesResource rdf:resource="http://example.org/
        EthernetSimulationModelOntology#Ethernet_Node"/>
</rdf:Description>

<rdf:Description rdf:about="http://example.org/
    EthernetSimulationModelOntology#Ethernet_SimulationModel">
    <ont:hasCapability rdf:resource="http://example.org/
        EthernetSimulationModelOntology#StoreAndForwardCommunication"/>
    <ont:hasCapability rdf:resource="http://example.org/
        EthernetSimulationModelOntology#MulticastCommunication"/>

</rdf:Description>

<!-- DataType Properties -->

<owl:DatatypeProperty rdf:about="http://example.org/
    EthernetSimulationModelOntology#hasPayloadSize">
    <rdfs:domain rdf:resource="http://example.org/
        EthernetSimulationModelOntology#Ethernet_Port"/>
    <rdfs:range rdf:resource="xsd:integer"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="http://example.org/
    EthernetSimulationModelOntology#hasMACAddress">
    <rdfs:domain rdf:resource="http://example.org/
        EthernetSimulationModelOntology#Ethernet_Port"/>
    <rdfs:range rdf:resource="xsd:string"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="http://example.org/
    EthernetSimulationModelOntology#hasBitRate">
    <rdfs:domain rdf:resource="http://example.org/
        EthernetSimulationModelOntology#Ethernet_SimulationModel"/>
    <rdfs:range rdf:resource="xsd:integer"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="http://example.org/
    EthernetSimulationModelOntology#hasLogTransmissions">
    <rdfs:domain rdf:resource="http://example.org/
        EthernetSimulationModelOntology#Ethernet_SimulationModel"/>

```

```

    <rdfs:range rdf:resource="xsd:boolean"/>
</owl:DatatypeProperty>

</rdf:RDF>

```

Listing 6: Ethernet Simulation Model Ontology

A.7 FlexRay Simulation Model Ontology

```

<?xml version="1.0"?>
<rdf:RDF xmlns="http://example.org/FlexRaySimulationModelOntology#"
  xml:base="http://example.org/FlexRaySimulationModelOntology"
  xmlns:dc="http://purl.org/dc/elements/1.1/"
  xmlns:bfo="http://purl.obolibrary.org/obo/bfo#"
  xmlns:css="https://raw.githubusercontent.com/CaSkade-Automation/CSS/
    main/CSS-Ontology#"
  xmlns:obo="http://purl.obolibrary.org/obo/"
  xmlns:ont="http://example.org/ontology#"
  xmlns:owl="http://www.w3.org/2002/07/owl#"
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:xml="http://www.w3.org/XML/1998/namespace"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema#"
  xmlns:foaf="http://xmlns.com/foaf/0.1/"
  xmlns:ida="http://www.semanticweb.org/eklaspad/ontologies/2022/6/
    untitled-ontology-5#"
  xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
  xmlns:terms="http://purl.org/dc/terms/"
  xmlns:vdi3633="https://raw.githubusercontent.com/JonathanReif/SiS/main
    /Resources/VDI3633#"
  xmlns:iso25010="http://www.example.org/iso25010#">

  <owl:Ontology rdf:about="http://example.org/
    FlexRaySimulationModelOntology">
    <owl:imports rdf:resource="http://example.org/ontology"/>
  </owl:Ontology>

  <!-- FlexRay Simulation Model -->

  <owl:Class rdf:about="http://example.org/FlexRaySimulationModelOntology#
    FlexRay_SimulationModel">
    <rdfs:subClassOf rdf:resource="http://www.semanticweb.org/eklaspad/
      ontologies/2022/6/untitled-ontology-5#SimulationModel"/>
    <rdfs:comment xml:lang="en">Simulation model for FlexRay deterministic
      communication.</rdfs:comment>
  </owl:Class>

  <!-- Capabilities -->

  <owl:Class rdf:about="http://example.org/FlexRaySimulationModelOntology#
    TimeTriggeredCommunication">

```

```

    <rdfs:subClassOf rdf:resource="http://www.w3id.org/hsu-aut/css#
      Capability"/>
  <owl:equivalentClass rdf:resource="http://www.semanticweb.org/eklaspad/
    ontologies/2022/6/untitled-ontology-5#simPurpose"/>
  <rdfs:comment xml:lang="en">Time-Triggered behaviour enables
    deterministic communication with its static segment for periodic,
    time-critical data.</rdfs:comment>
</owl:Class>

<owl:Class rdf:about="http://example.org/FlexRaySimulationModelOntology#
  EventDrivenCommunication">
  <rdfs:subClassOf rdf:resource="http://www.w3id.org/hsu-aut/css#
    Capability"/>
  <owl:equivalentClass rdf:resource="http://www.semanticweb.org/eklaspad/
    ontologies/2022/6/untitled-ontology-5#simPurpose"/>
  <rdfs:comment xml:lang="en">Event-Driven communication enables flexible
    bandwidth allocation with its dynamic segment for event-driven, less
    time-critical data.</rdfs:comment>
</owl:Class>

<owl:Class rdf:about="http://example.org/FlexRaySimulationModelOntology#
  DualChannelCommunication">
  <rdfs:subClassOf rdf:resource="http://www.w3id.org/hsu-aut/css#
    Capability"/>
  <owl:equivalentClass rdf:resource="http://www.semanticweb.org/eklaspad/
    ontologies/2022/6/untitled-ontology-5#simPurpose"/>
  <rdfs:comment xml:lang="en">FlexRay's Dual Channel Architecture allows
    parallel transmission on both channels simultaneously.</rdfs:comment>
</owl:Class>

<!-- Resources -->

<owl:Class rdf:about="http://example.org/FlexRaySimulationModelOntology#
  FlexRay_Node">
  <rdfs:subClassOf rdf:resource="http://www.w3id.org/hsu-aut/css#Resource
    "/>
  <rdfs:comment xml:lang="en">Nodes are the individual devices connected to
    the FlexRay network. Each node can both transmit and receive
    messages on the network using dual channels.</rdfs:comment>
</owl:Class>

<owl:Class rdf:about="http://example.org/FlexRaySimulationModelOntology#
  FlexRay_Port">
  <rdfs:subClassOf rdf:resource="http://www.w3id.org/hsu-aut/css#Resource
    "/>
  <owl:equivalentClass rdf:resource="http://www.semanticweb.org/eklaspad/
    ontologies/2022/6/untitled-ontology-5#portsConnector"/>
  <rdfs:comment xml:lang="en">Ports are for connecting several applications
    to the same network interface that can transmit and receive value
    events to and from the ApplicationInterface instance.</rdfs:comment>
</owl:Class>

```

```

<owl:Class rdf:about="http://example.org/FlexRaySimulationModelOntology#
  HybridCommunication">
  <rdfs:subClassOf rdf:resource="http://www.w3id.org/hsu-aut/css#Process"
    />
  <rdfs:comment xml:lang="en">FlexRay's hybrid communication combines
    deterministic time-triggered communication with flexible bandwidth
    allocation, all supported by dual redundant channels for high
    reliability.</rdfs:comment>
</owl:Class>

<owl:Class rdf:about="http://example.org/FlexRaySimulationModelOntology#
  Flexibility">
  <rdfs:subClassOf rdf:resource="http://www.example.org/iso25010#
    Portability"/>
</owl:Class>

<owl:Class rdf:about="http://example.org/FlexRaySimulationModelOntology#
  Reliability">
  <owl:equivalentClass rdf:resource="http://www.example.org/iso25010#
    Reliability"/>
</owl:Class>

<!-- Object Properties -->

<rdf:Description rdf:about="http://example.org/
  FlexRaySimulationModelOntology#FlexRay_SimulationModel">
  <ont:usesResource rdf:resource="http://example.org/
    FlexRaySimulationModelOntology#FlexRay_Port"/>
</rdf:Description>

<rdf:Description rdf:about="http://example.org/
  FlexRaySimulationModelOntology#FlexRay_SimulationModel">
  <ont:hasCapability rdf:resource="http://example.org/
    FlexRaySimulationModelOntology#TimeTriggeredCommunication"/>
  <ont:hasCapability rdf:resource="http://example.org/
    FlexRaySimulationModelOntology#EventDrivenCommunication"/>
  <ont:hasCapability rdf:resource="http://example.org/
    FlexRaySimulationModelOntology#DualChannelCommunication"/>
</rdf:Description>

<owl:DatatypeProperty rdf:about="http://example.org/
  FlexRaySimulationModelOntology#hasPayloadSize">
  <rdfs:domain rdf:resource="http://example.org/
    FlexRaySimulationModelOntology#FlexRay_Port"/>
  <rdfs:range rdf:resource="xsd:integer"/>
</owl:DatatypeProperty>

```

```

<owl:DatatypeProperty rdf:about="http://example.org/
  FlexRaySimulationModelOntology#hasSlotValue">
  <rdfs:domain rdf:resource="http://example.org/
    FlexRaySimulationModelOntology#FlexRay_Port"/>
  <rdfs:range rdf:resource="xsd:integer"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="http://example.org/
  FlexRaySimulationModelOntology#hasSegmentType">
  <rdfs:domain rdf:resource="http://example.org/
    FlexRaySimulationModelOntology#FlexRay_Port"/>
  <rdfs:range rdf:resource="xsd:string"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="http://example.org/
  FlexRaySimulationModelOntology#hasBitRate">
  <rdfs:domain rdf:resource="http://example.org/
    FlexRaySimulationModelOntology#FlexRay_SimulationModel"/>
  <rdfs:range rdf:resource="xsd:integer"/>
</owl:DatatypeProperty>

<owl:DatatypeProperty rdf:about="http://example.org/
  FlexRaySimulationModelOntology#hasGdStaticSlot">
  <rdfs:domain rdf:resource="http://example.org/
    FlexRaySimulationModelOntology#FlexRay_SimulationModel"/>
  <rdfs:range rdf:resource="xsd:integer"/>
</owl:DatatypeProperty>

</rdf:RDF>

```

Listing 7: FlexRay Simulation Model Ontology

A.8 Additional Ontology files

The following ontology files are available online due to their size and complexity:

The complete CSS ontology file can be accessed at:

<https://github.com/CaSkade-Automation/CSS/blob/main/CSS/CSS-Ontology.owl>

The complete BFO ontology file can be accessed at:

<https://github.com/BFO-ontology/BFO-2020/blob/master/src/owl/bfo-core.owl>

The complete IDTA ontology file can be accessed at:

<http://www.semanticweb.org/eklaspad/ontologies/2022/6/untitled-ontology-5/SimulationModels.ontology.owl>

The complete VDI3633 ontology file can be accessed at:

<https://github.com/JonathanReif/SiS/blob/main/Resources/VDI3633.owl>

Declaration

I hereby declare and confirm that this Master thesis is entirely the result of my own original work. Where other sources of information have been used, they have been indicated as such and properly acknowledged. I declare that I have written this thesis independently and used only the sources and resources listed. I further declare that this work has not been submitted in whole or in part for any other degree or qualification at this or any other institution.

Kaiserslautern, May 31, 2025

Anuja Chakraborty

(Signature)



Acknowledgment

Firstly, I would like to express my gratitude to Fraunhofer Institute For Experimental Software Engineering (IESE), Kaiserslautern, for providing me with the necessary tools and support for completion of this thesis.

I would like to express my deepest gratitude to my supervisors, Prof. Dr.-Ing. Peter Liggesmeyer and Siwara Schmitt, for their invaluable guidance, unwavering support, and insightful feedback throughout this research journey.

I am also grateful to my alma mater RPTU, Kaiserslautern for providing access to relevant resources and data that made this research possible. Special thanks also go to some of my Fraunhofer IESE colleagues for their collaborative spirit, stimulating discussions, and constructive feedback that enriched this work. My heartfelt appreciation extends to my family as well for their encouragement and understanding during this thesis period.